UNIVERSIDAD AUTONOMA GABRIEL RENE MORENO

FACULTAD DE INGENIERÍA DE CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

Proyecto Integrador – Primer Parcial

ReviewIQ: Plataforma de Diagnóstico de Reputación y Evaluación de Experiencia del Usuario mediante NLP y Machine Learning

DOCENTE: Ing. Kenji Kawaida

MATERIA: Tópicos y Programación Avanzada

INTEGRANTES:

- Rojas Cruz German Lino 221045740
- Pimentel Guarena Jose Carlos 221089373
- Quispe Antonio Kevin 222009500

Santa Cruz – Bolivia

Gestión II – 2026

ÍNDICE GENERAL

ÍNDICE GENERAL 2
1.2. Antecedentes 5
1.3. Planteamiento del Problema 6
1.3.1. Situación Problemática 6
1.3.2. Formulación del Problema 7
1.4. Objetivos 7
1.4.1. Objetivo General 7
1.4.2. Objetivos Específicos 7
1.5. Justificación 8
1.6. Alcance y Límites 8
1.6.1. Alcance 8
1.6.2. Límites 9
1.7. Recursos a Usar 9
Software 9
Hardware 9
1.8. Metodología de Desarrollo 10
1.8.1. Proceso Unificado de Desarrollo de Software (PUDS) 10
1.8.2. Lenguaje Unificado de Modelado (UML) 11
2. PROCESO DE DESARROLLO 12
2.1. FLUJO DE TRABAJO: CAPTURA DE REQUISITOS 12
2.1.1. Identificar actores y casos de uso 12
2.1.2 Priorizar casos de uso 14
2.1.3 Detallar casos de uso 15
1. Diagrama General de Casos de Uso 39
2.2.2 Relación de Paquetes 42
2. Análisis de casos de uso 47
2.1. CU01 Iniciar sesión 47
2.2. CU02 Cerrar sesión 48
2.3. CU03 Registrar usuario 48
2.4. CU04 Explorar catálogo de aplicaciones por categoría 49
2.5. CU05 Ver diagnóstico y calificación de una aplicación 49
2.6. CU06 Consultar opiniones y reseñas de una aplicación 49
2.7. CU07 Escribir reseña y calificar una aplicación 50
2.8. CU08 Comparar dos aplicaciones en modo Versus 50
2.9. CU09 Buscar aplicaciones en Google Play en tiempo real 51
2.10. CU10 Sincronizar opiniones desde Google Play 51
2.11. CU11 Estimar calificación de estrellas para una reseña libre 52
2.12. CU12 Inspeccionar pipeline lingüístico en el Laboratorio NLP 52

2.13. CU13 Consultar métricas del modelo y matriz de confusión.....53
2.14. CU14 Comparar modelo de Regresión Logística con Naive Bayes.....53
2.15. CU15 Consultar metodología del corpus y particiones de datos.....53
2.16. CU16 Consultar documentación interactiva de la API (Swagger).....54
2.17. CU17 Procesar lote de reseñas de forma masiva.....54
2.18. CU18 Implementar pipeline de preprocesamiento de texto.....55
2.19. CU19 Implementar flujo de entrenamiento del modelo de clasificación.....56
2.20. CU20 Monitorear disponibilidad y estado del servicio.....56
3. Análisis de paquete.....57
2.2.3 FLUJO DE TRABAJO: DISEÑO.....58
2.2.3.1 Diseño Físico (Diagrama de Despliegue).....58
Diagrama de Clase.....59
Diseño de Datos.....59
Diseño Lógico.....59
Diseño de Caso de Uso.....65
4. Diseño de Interfaces.....75
5. Procesamiento De Lenguaje Natural Y Modelos De Machine Learning.....76
5.1 Pipeline de Procesamiento de Lenguaje Natural.....77
RECOMENDACIONES.....82
Bibliografía.....83
1. Anexos.....85

1.1. Introducción

En la actualidad, las opiniones y reseñas publicadas por los usuarios en plataformas digitales (como tiendas de comercio electrónico, servicios de entrega, plataformas financieras y tiendas de aplicaciones como Google Play Store) representan una de las fuentes de información más valiosas para conocer la percepción real sobre un producto o servicio. Diariamente se generan millones de comentarios en texto libre donde los consumidores expresan sus felicitaciones, dudas o quejas específicas.

Sin embargo, este gran volumen de datos no estructurados presenta una limitación crítica: su análisis manual es humanamente inviable para las organizaciones y los usuarios. Aunque muchas plataformas ofrecen una calificación numérica simple (de 1 a 5 estrellas), esta nota promedio suele ocultar los verdaderos motivos de satisfacción o descontento del cliente. Una aplicación o producto puede tener 4 estrellas globales, pero sufrir fallos constantes en sus entregas o una pésima atención al cliente, aspectos que pasan desapercibidos a simple vista.

Frente a esta problemática surge ReviewIQ, una plataforma integral de software basada en Procesamiento de Lenguaje Natural (NLP) y Machine Learning. El sistema está diseñado para procesar texto no estructurado en español, predecir de forma automatizada la puntuación que merece una opinión (en una escala de 1 a 5 estrellas con sus respectivas probabilidades) y desagregar el contenido de las reseñas en dimensiones clave de experiencia:

- Atención al cliente.
- Envíos y logística.
- Calidad del producto o servicio.
- Relación precio y valor.
- Experiencia y funcionamiento de la aplicación (UX).

La solución abarca desde la recolección y preprocesamiento riguroso de texto crudo (normalización de caracteres, gestión de modismos y emojis, filtrado de palabras vacías y vectorización TF-IDF), el entrenamiento y evaluación comparativa de modelos de clasificación supervisada (Regresión Logística multiclase y Naive Bayes), hasta su despliegue operativo a través de una API REST en Django y una interfaz web interactiva en React. De este modo, ReviewIQ no solo ofrece un predictor académico de estrellas y un laboratorio pedagógico para inspeccionar el comportamiento del texto, sino también una herramienta práctica de diagnóstico comercial, comparativas directas (modo Versus) e ingesta en tiempo real desde Google Play Store.

1.2. Antecedentes

Históricamente, las organizaciones dependían de métodos tradicionales como encuestas presenciales, llamadas telefónicas o grupos focales (*focus groups*) para medir la satisfacción de sus clientes. Con la masificación del comercio electrónico y los teléfonos inteligentes, la retroalimentación de los consumidores se trasladó masivamente hacia canales digitales públicos como Google Play Store, foros y redes sociales.

En el campo del Procesamiento de Lenguaje Natural (NLP) y la minería de opiniones, las primeras soluciones se basaban en listas manuales de palabras positivas y negativas (enfoques léxicos basados en diccionarios). Si bien eran fáciles de implementar, presentaban serias deficiencias al analizar opiniones complejas, sarcasmo o contextos donde intervienen negaciones (por ejemplo, la diferencia entre *"no es malo"* y *"es muy malo"*).

Con el avance del Machine Learning, el análisis de opiniones evolucionó hacia modelos de aprendizaje supervisado apoyados en representaciones vectoriales como TF-IDF (Term Frequency – Inverse Document Frequency) combinados con algoritmos de clasificación como Naive Bayes y Regresión Logística multiclase. Diversos estudios han demostrado que estos modelos lineales ofrecen un equilibrio sobresaliente entre precisión predictiva, bajo costo computacional y alta interpretabilidad de las características del texto, resultando ideales para entornos de producción donde se requiere responder en pocos milisegundos.

A nivel de datos de entrenamiento en español, la disponibilidad de conjuntos de datos estandarizados era limitada hasta la publicación del corpus Amazon Reviews Multi ES (Keung et al., 2020), el cual recopila cientos de miles de opiniones reales en español etiquetadas rigurosamente de 1 a 5 estrellas de manera perfectamente balanceada entre clases.

En el ámbito regional y local (particularmente en Bolivia y Latinoamérica), servicios digitales esenciales como entrega a domicilio (*delivery*), transporte urbano, banca móvil y comercio electrónico acumulan diariamente miles de valoraciones en Google Play Store. A pesar de la abundancia de estos datos públicos, no existen herramientas accesibles que analicen sistemáticamente estas opiniones en tiempo real ni que permitan a los usuarios comparar de forma objetiva las fortalezas y debilidades operativas entre aplicaciones competidoras.

1.3. Planteamiento del Problema

1.3.1. Situación Problemática

La creciente dependencia de las plataformas digitales en la vida cotidiana ha provocado que la reputación de una empresa o aplicación móvil dependa directamente de las opiniones emitidas por sus usuarios. No obstante, en la gestión y análisis de estas opiniones se identifican los siguientes problemas fundamentales:

1. **Inviabilidad del análisis manual por saturación de texto**:
   Las aplicaciones populares acumulan miles de opiniones por semana. Leer, clasificar y extraer conclusiones de cada reseña de forma humana requiere una inversión desmedida de tiempo y personal, provocando que la mayoría de los comentarios queden archivados sin ser analizados.

2. **Opacidad de la calificación numérica tradicional**:
   Una puntuación promedio (por ejemplo, 3.8 estrellas) es un indicador genérico que no explica las causas del rendimiento. No permite distinguir si los clientes están insatisfechos por cobros excesivos (precio), demoras en el reparto (logística), mala atención del personal o constantes fallos y cierres inesperados de la aplicación móvil (UX).

3. **Particularidades y desafíos del idioma español**:
   El texto informal de las opiniones suele incluir modismos, errores ortográficos, uso intensivo de emojis y estructuras de negación complejas (*"no me gustó"*, *"nunca más vuelvo a pedir"*). Si el sistema no cuenta con un preprocesamiento lingüístico adecuado, estas particularidades distorsionan la interpretación del sentimiento.

4. **Desconexión entre modelos de Machine Learning y software productivo**:
   Con frecuencia, los experimentos de inteligencia artificial se limitan a entornos aislados (como cuadernos de Jupyter), sin ser integrados en una arquitectura de software completa y reproducible que cuente con una API segura, una base de datos relacional y una interfaz intuitiva para el usuario final.

1.3.2. Formulación del Problema

A partir de la situación descrita, se formula la siguiente pregunta central de investigación:

¿De qué manera se puede desarrollar una plataforma web integral que combine Procesamiento de Lenguaje Natural (NLP) y modelos de Machine Learning supervisados para estimar de forma precisa la calificación de 1 a 5 estrellas en reseñas escritas en español, y desagregar el análisis en dimensiones clave de calidad y experiencia del usuario?

1.4. Objetivos

1.4.1. Objetivo General

Desarrollar e implementar una plataforma web completa de software (ReviewIQ) que integre un pipeline de Procesamiento de Lenguaje Natural (NLP) y modelos de Machine Learning supervisados, expuestos mediante una API REST en el backend y consumidos por un frontend interactivo, para estimar la calificación de 1 a 5 estrellas a partir de reseñas en español y evaluar la experiencia del usuario a través de un diagnóstico multidimensional de calidad.

1.4.2. Objetivos Específicos

1. Construir y validar un pipeline lingüístico de preprocesamiento de texto en español que incorpore normalización de caracteres (NFKC, minúsculas y tildes), mapeo léxico de emojis, filtrado de palabras vacías (*stop words*) preservando negaciones clave, y extracción de características mediante n-gramas y vectorización TF-IDF.
2. Entrenar, calibrar y comparar modelos de Machine Learning de clasificación supervisada (Regresión Logística multiclase y Naive Bayes) sobre un corpus estandarizado de reseñas en español (Amazon Reviews Multi ES), evaluando su rendimiento mediante métricas formales (Macro-F1, Accuracy, MAE y Matriz de Confusión de 5x5) para seleccionar y congelar el modelo óptimo.
3. Empaquetar el pipeline de procesamiento y el modelo entrenado en un artefacto serializado y reproducible, asegurando que exactamente las mismas transformaciones del entrenamiento se ejecuten de manera determinista durante la inferencia en producción.
4. Desarrollar una API REST robusta en Django y Django REST Framework que exponga endpoints seguros para inferencia individual, procesamiento por lotes (*Batch*), trazabilidad lingüística paso a paso, consulta de métricas y verificación de salud del servicio.

5. Implementar un conector de extracción (*Scraper*) para Google Play Store que permita buscar aplicaciones por nombre y sincronizar opiniones de usuarios reales en tiempo real con almacenamiento persistente en una base de datos relacional PostgreSQL.
6. Diseñar un motor de diagnóstico multidimensional que clasifique las menciones textuales en 5 aspectos esenciales de experiencia: *Atención al cliente, Envíos y logística, Calidad del producto, Relación precio/valor y Experiencia de uso (UX)*.
7. Construir una interfaz de usuario moderna y reactiva en React y TypeScript que incluya un probador interactivo de reseñas, un laboratorio visual de NLP, un panel analítico de métricas científicas, un catálogo de aplicaciones organizado por categorías y un módulo comparativo.

1.5. Justificación

El desarrollo de ReviewIQ se justifica por la necesidad apremiante de transformar grandes volúmenes de opiniones digitales no estructuradas en información comprensible y accionable para la toma de decisiones. Actualmente, la calificación numérica tradicional de las plataformas digitales resulta insuficiente, ya que una simple nota promedio oculta los verdaderos motivos de satisfacción o queja del cliente.

Frente a esta limitación, el proyecto propone una solución tecnológica basada en Procesamiento de Lenguaje Natural (NLP) y Machine Learning, utilizando un pipeline con TF-IDF y Regresión Logística multiclase. Esta combinación técnica permite estimar con alta precisión calificaciones de 1 a 5 estrellas y responder en menos de 15 milisegundos, logrando una alta eficiencia computacional e interpretabilidad sin requerir costosos servidores con GPU.

Asimismo, la plataforma conecta la rigurosidad académica con la aplicabilidad en el mundo real, al integrar la extracción en vivo de reseñas desde Google Play Store y desagregar la experiencia del usuario en 5 aspectos esenciales (atención, logística, producto, precio y funcionamiento de la app). De este modo, se dota a los usuarios de una herramienta transparente para comparar aplicaciones y a los desarrolladores de un diagnóstico claro para priorizar mejoras en sus servicios.

1.6. Alcance y Límites

1.6.1. Alcance

- Dominio lingüístico: Procesamiento exclusivo de textos en idioma español, adaptado a expresiones informales, tildes, signos de puntuación y emojis comunes en reseñas.
- Tarea predictiva: Clasificación supervisada multiclase en una escala ordinal de 1 a 5 estrellas, reportando la clase estimada junto con el vector completo de probabilidades.

- Diagnóstico multidimensional: Clasificación léxico-semántica de las menciones en 5 aspectos: *Atención al cliente, Envíos y logística, Calidad del producto, Relación precio/valor y Experiencia de uso (UX)*.
- Extracción de datos: Búsqueda en vivo y extracción de opiniones públicas desde Google Play Store mediante scraping automatizado.
- Plataforma web: Interfaz interactiva para consulta de catálogo por categorías (Compras, Delivery, Finanzas, Juegos), comparativa Versus, predictor libre, laboratorio de trazas de NLP y panel de métricas científicas.

1.6.2. Límites

- El sistema analiza exclusivamente información textual no estructurada; no procesa imágenes, audios ni videos adjuntos a las opiniones.
- La sincronización con Google Play Store está orientada a la extracción de reseñas públicas en lotes de hasta 100 opiniones por consulta para respetar las tasas de consulta del servicio externo.
- El corpus de entrenamiento base corresponde al estándar de comercio electrónico (*Amazon Reviews Multi ES*), aplicando aprendizaje transferido y ajuste dimensional para el análisis de aplicaciones móviles.

1.7. Recursos a Usar

Software

- **Lenguajes de programación**: Python 3.11 (backend, NLP y Machine Learning) y TypeScript / JavaScript (frontend).
- **Entorno y Framework Web**: Django 5 y Django REST Framework para la construcción de la API REST.
- **Librerías de NLP y Machine Learning**: Scikit-learn (vectorización TF-IDF, Regresión Logística y Naive Bayes) y Joblib (serialización del pipeline).
- **Extracción de datos**: Google-play-scraper para la obtención en vivo de opiniones y metadatos de aplicaciones.
- **Base de datos**: PostgreSQL para el almacenamiento relacional de entidades, reseñas y usuarios.
- **Frontend**: React 18, empaquetado con Vite y estilizado con CSS modular.
- **Control de versiones**: Git y GitHub (trabajo en ramas colaborativas con estándar Conventional Commits).

Hardware

- **Estaciones de trabajo (PC / Laptop)**: Procesador multi-núcleo (Intel Core i5/i7 o AMD Ryzen), memoria RAM mínima de 8 GB (16 GB recomendado) y almacenamiento en disco de estado sólido (SSD).
- **Entorno de ejecución**: Despliegue local y ejecución de inferencias en CPU sin requerir tarjetas gráficas dedicadas (GPU), demostrando alta eficiencia y bajo costo operativo.

1.8. Metodología de Desarrollo

Para el desarrollo del proyecto ReviewIQ se adoptó el Proceso Unificado de Desarrollo de Software (PUDS) asistido por el Lenguaje Unificado de Modelado (UML) como estándar de análisis, diseño y documentación visual.

1.8.1. Proceso Unificado de Desarrollo de Software (PUDS)

Características del PUDS

El Proceso Unificado de Desarrollo de Software es un proceso iterativo; un enfoque iterativo propone una comprensión incremental del problema a través de refinamientos sucesivos y un crecimiento incremental de una solución efectiva a través de varios ciclos.

El desarrollo de un producto software comercial supone un gran esfuerzo que puede durar entre varios meses hasta posiblemente un año o más. Es práctico dividir el producto en partes más pequeñas o mini proyectos. Cada mini proyecto es una iteración que resulta en un incremento. Las iteraciones hacen referencia a pasos en el flujo, y los incrementos, al crecimiento del producto. En cada iteración, los desarrolladores identifican y especifican los casos de uso relevantes, crean un diseño utilizando la arquitectura seleccionada como guía, implementan el diseño mediante componentes y verifican que los componentes satisfagan los casos de uso. Cada uno de ellos sigue el esquema:

- **Análisis**: Identificar y especificar casos de uso relevantes.
- **Diseño**: Crear un diseño utilizando la arquitectura seleccionada.
- **Implementación**: Implementar el diseño mediante componentes.
- **Pruebas**: Verificación de estos componentes.

El Proceso Unificado de Desarrollo de Software (PUDS) es un marco de desarrollo de software que se caracteriza por:

- **Enfoque Iterativo e Incremental**: El PUDS se basa en ciclos iterativos y entregas incrementales. En lugar de desarrollar todo el software de una sola vez, el proyecto se divide en iteraciones, cada una de las cuales produce una versión parcial o incremento del sistema. Esto permite una retroalimentación temprana y la adaptación a cambios de requisitos.

- **Enfoque en la Arquitectura**: Se presta una atención especial a la arquitectura del sistema desde el principio. Se define una arquitectura sólida que sirve como base para el desarrollo posterior y permite una mejor gestión de riesgos.
- **Roles Definidos**: El PUDS define roles específicos para los miembros del equipo, como analistas, diseñadores, programadores y *testers*. Cada rol tiene responsabilidades claras en diferentes etapas del proceso de desarrollo.
- **Gestión de Riesgos**: El PUDS se enfoca en la identificación y gestión temprana de riesgos. Se realiza un análisis de riesgos continuo a lo largo del proyecto y se toman medidas para mitigarlos.
- **Documentación y Modelado**: El proceso enfatiza la documentación y el modelado adecuados. Se utilizan técnicas como UML para crear modelos visuales que describen el sistema de manera completa y comprensible.

1.8.2. Lenguaje Unificado de Modelado (UML)

Características de UML

El Lenguaje Unificado de Modelado (UML) es un lenguaje de modelado visual de propósito general que se emplea para especificar, visualizar, construir y documentar los elementos de un sistema de software. Se utiliza para comprender, diseñar, examinar, configurar, mantener y supervisar la información acerca de dichos sistemas. UML está diseñado para ser aplicado en conjunción con todos los métodos de desarrollo, en todas las etapas del ciclo de vida del software, en diversos dominios de aplicación y en diferentes contextos. A continuación, se destacan algunas de sus características principales:

- **Estandarización**: UML es un estándar de la industria respaldado por el *Object Management Group* (OMG). Esto significa que UML proporciona una notación y un conjunto de reglas comunes que pueden ser entendidos y utilizados por desarrolladores de software en todo el mundo.
- **Modelado Visual**: UML se basa en un enfoque visual para el modelado de sistemas. Utiliza diagramas gráficos para representar diferentes aspectos de un sistema, lo que facilita la comprensión y la comunicación entre los miembros del equipo y los *stakeholders*.
- **Versatilidad**: UML es versátil y admite una amplia gama de aplicaciones, desde el modelado de sistemas de software hasta el modelado de procesos de negocios y sistemas en tiempo real. Ofrece varios tipos de diagramas, como diagramas de casos de uso, de clases, de secuencia, de componentes y de despliegue, que se adaptan a diferentes necesidades de modelado.

- **Abstracción**: UML permite representar sistemas a diferentes niveles de abstracción. Puede utilizarse para modelar la estructura estática de un sistema (clases, objetos) o su comportamiento dinámico (secuencia de eventos e interacciones).
- **Reutilización**: UML fomenta la reutilización de componentes y patrones de diseño. Los modelos UML sirven como documentación que ayuda a comprender cómo se pueden utilizar los componentes y cómo encajan dentro de la arquitectura global del sistema.

2. PROCESO DE DESARROLLO

2.1. FLUJO DE TRABAJO: CAPTURA DE REQUISITOS

2.1.1. Identificar actores y casos de uso

Actor	Descripción
Administrador	Usuario con privilegios completos de gestión sobre el sistema. Es responsable de administrar las aplicaciones registradas en el catálogo, gestionar usuarios y roles, autorizar la sincronización e ingesta masiva de reseñas desde Google Play Store y monitorear la salud y disponibilidad de los modelos de Machine Learning y la base de datos PostgreSQL. Accede a través de la plataforma web.
Usuario/Analista	Usuario final o analista comercial que utiliza la plataforma web para explorar aplicaciones por categoría, evaluar el diagnóstico de calidad en 5 aspectos de experiencia, publicar opiniones con calificación automática de estrellas, comparar dos aplicaciones cara a cara (Versus) y experimentar con el laboratorio interactivo de NLP.

Figura 2. Identificación de Actores

Casos de uso

Código	Nombre del Caso de Uso	Plataforma
CU01	Iniciar sesión	Web
CU02	Cerrar sesión	Web
CU03	Registrar usuario	Web
CU04	Explorar catálogo de aplicaciones por categoría	Web
CU05	Ver diagnóstico y calificación de una aplicación	Web
CU06	Consultar opiniones y reseñas de una aplicación	Web
CU07	Escribir reseña y calificar una aplicación	Web
CU08	Comparar dos aplicaciones en modo Versus	Web
CU09	Buscar aplicaciones en Google Play en tiempo real	Web
CU10	Sincronizar opiniones desde Google Play	Web
CU11	Estimar calificación de estrellas para una reseña libre	Web
CU12	Inspeccionar pipeline lingüístico en el Laboratorio NLP	Web
CU13	Consultar métricas del modelo y matriz de confusión	Web
CU14	Comparar modelo de Regresión Logística con Naive Bayes	Web
CU15	Consultar metodología del corpus y particiones de datos	Web
CU16	Consultar documentación interactiva de la API (Swagger)	Web
CU17	Procesar lote de reseñas de forma masiva	Web / API
CU18	Implementar pipeline de preprocesamiento de texto	Backend / CLI
CU19	Implementar flujo de entrenamiento del modelo de clasificación	Backend / CLI
CU20	Monitorear disponibilidad y estado del servicio	Web / API

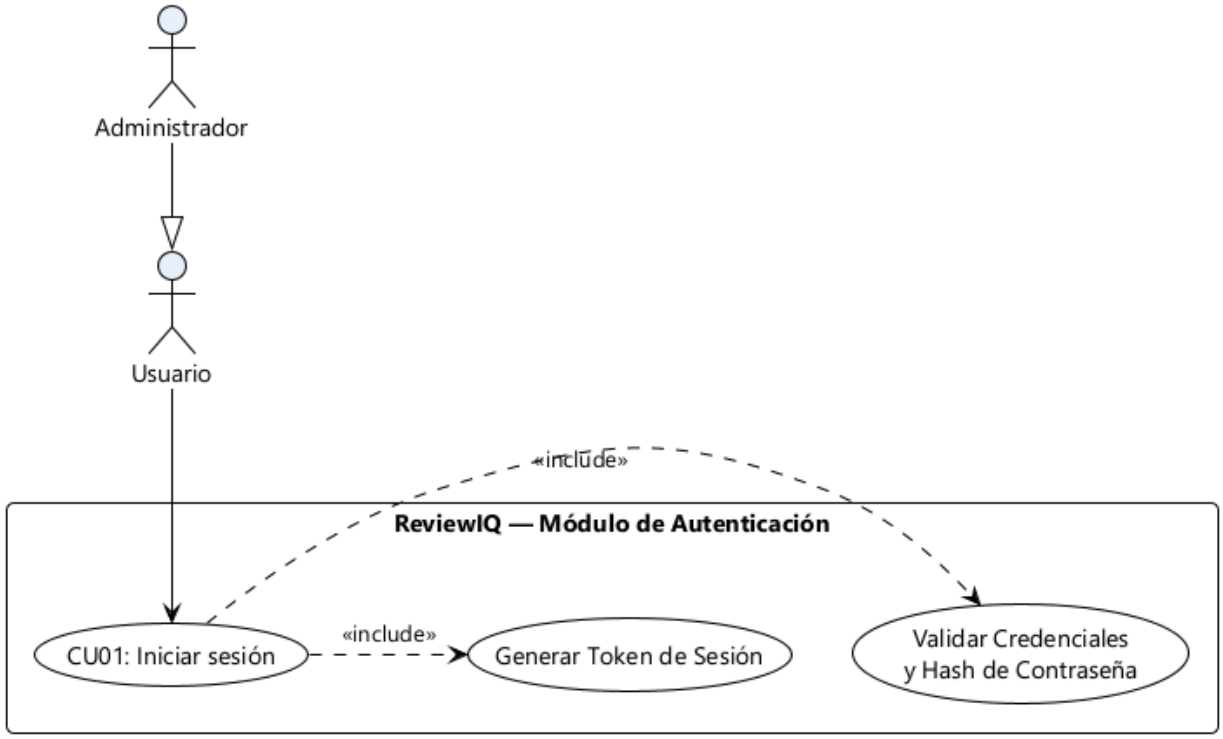
2.1.2 Priorizar casos de uso

CÓDIGO	LISTA DE CASOS DE USO	PLATAFORMA	ESTADO	PRIORIDAD	RIESGO	ACTORES
CU01	Iniciar sesión	Web	Aprobado	Crítico	Crítico	Administrador, Usuario
CU02	Cerrar sesión	Web	Aprobado	Medio	Bajo	Administrador, Usuario
CU03	Registrar usuario	Web	Aprobado	Alto	Medio	Usuario
CU04	Explorar catálogo de aplicaciones por categoría	Web	Aprobado	Alto	Medio	Usuario, Administrador
CU05	Ver diagnóstico y calificación de una aplicación	Web	Aprobado	Crítico	Crítico	Usuario, Administrador
CU06	Consultar opiniones y reseñas de una aplicación	Web	Aprobado	Medio	Bajo	Usuario, Administrador
CU07	Escribir reseña y calificar una aplicación	Web	Aprobado	Alto	Significativo	Usuario
CU08	Comparar dos aplicaciones en modo Versus	Web	Aprobado	Alto	Significativo	Usuario, Administrador
CU09	Buscar aplicaciones en Google Play en tiempo real	Web	Aprobado	Alto	Significativo	Usuario, Administrador
CU10	Sincronizar opiniones desde Google Play	Web	Aprobado	Crítico	Crítico	Administrador, Usuario, Google Play Store
CU11	Estimar calificación de estrellas para una reseña libre	Web	Aprobado	Crítico	Crítico	Usuario, Administrador
CU12	Inspeccionar pipeline lingüístico en el Laboratorio NLP	Web	Aprobado	Medio	Medio	Usuario, Administrador
CU13	Consultar métricas del modelo y matriz de confusión	Web	Aprobado	Medio	Bajo	Usuario, Administrador
CU14	Comparar modelo de Regresión Logística con Naive Bayes	Web	Aprobado	Medio	Bajo	Usuario, Administrador
CU15	Consultar metodología del corpus y particiones de datos	Web	Aprobado	Bajo	Bajo	Usuario, Administrador
CU16	Consultar documentación interactiva de la API	Web	Aprobado	Bajo	Bajo	Administrador, Usuario
CU17	Procesar lote de reseñas de forma masiva	Web / API	Aprobado	Alto	Significativo	Administrador, Usuario

CU18	Implementar pipeline de preprocesamiento de texto	Backend / CLI	Aprobado	Medio	Medio	Sistema
CU19	Implementar flujo de entrenamiento del modelo de clasificación	Backend / CLI	Aprobado	Medio	Medio	Sistema
CU20	Monitorear disponibilidad y estado del servicio	Web / API	Aprobado	Crítico	Crítico	Administrador, Sistema

2.1.3 Detallar casos de uso

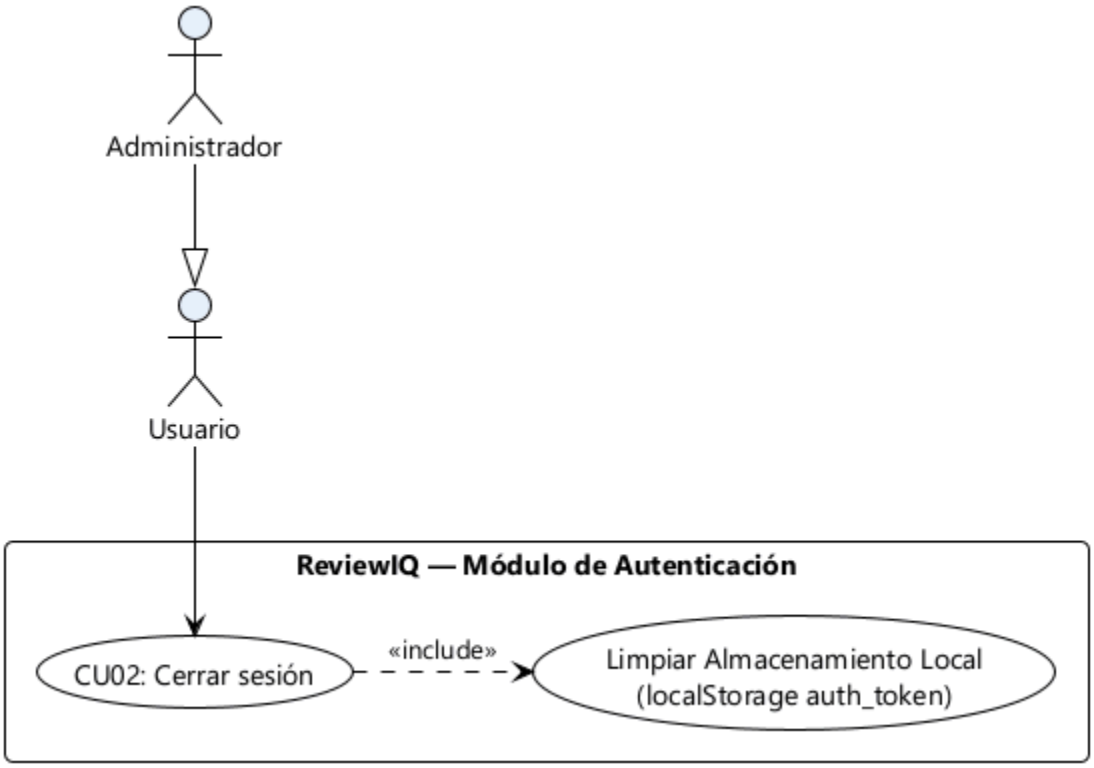
CU01: Iniciar sesión



CASO DE USO:	**CU-01 Iniciar sesión**
Propósito:	Permitir a un Usuario o a un Administrador acceder al sistema con una sesión autenticada.
Resumen:	El usuario introduce su nombre de usuario y contraseña para identificarse y recibir acceso a las funcionalidades según su rol.
Actores:	Administrador, Usuario

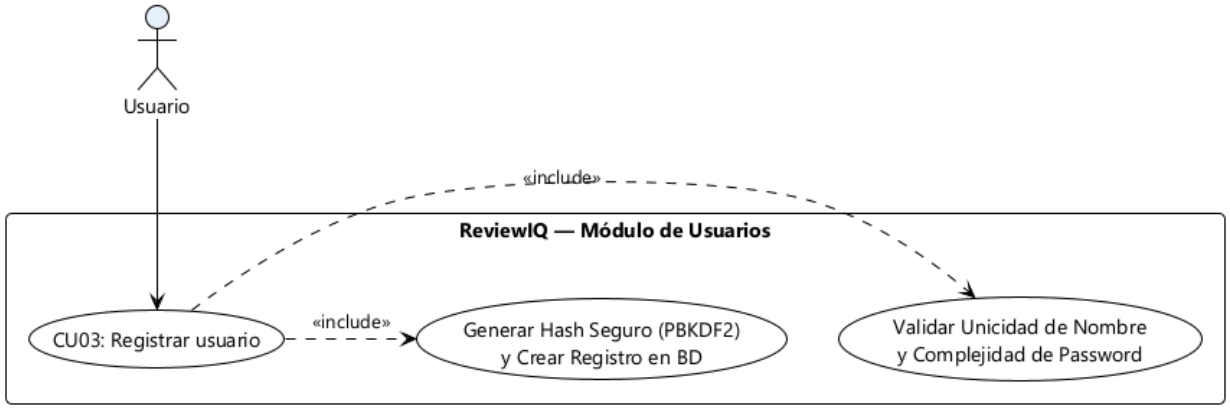
Actor Iniciador:	Usuario o Administrador
Pre-Condición:	El usuario debe tener una cuenta registrada previamente en el sistema.
Flujo de Sucesos:	1. El actor hace clic en 'Iniciar sesión'. 2. El sistema muestra el formulario de ingreso solicitando usuario y contraseña. 3. El actor ingresa sus credenciales y hace clic en 'Entrar'. 4. El sistema valida las credenciales (POST /api/v1/reputation/auth/login/) contra el hash PBKDF2 almacenado en la base de datos PostgreSQL. 5. El sistema devuelve el token de sesión y el rol correspondiente (staff o analista). 6. La interfaz actualiza el encabezado mostrando el nombre y rol del usuario autenticado.
Post-Condición:	Sesión activa; credenciales y token almacenados localmente para las siguientes peticiones.
Excepciones:	Credenciales incorrectas (usuario o contraseña no coinciden). Campos requeridos vacíos.

CU-02 Cerrar sesión



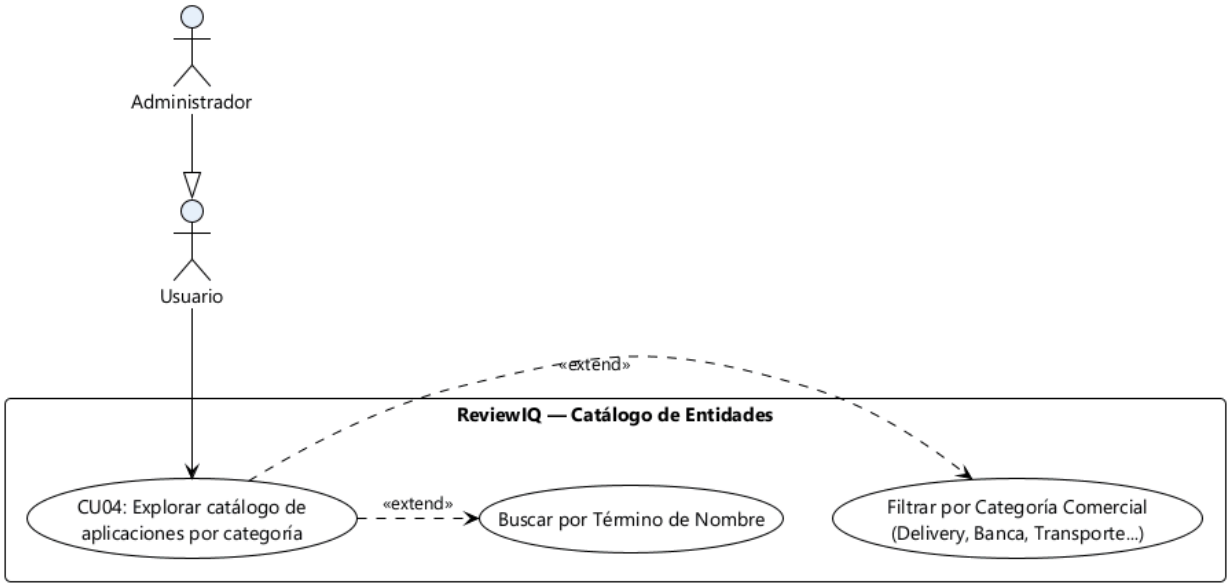
CASO DE USO:	**CU-02 Cerrar sesión**
Propósito:	Permitir al usuario finalizar su sesión activa de manera segura.
Resumen:	El usuario desconecta su cuenta para proteger su privacidad y revocar el acceso a las funciones protegidas.
Actores:	Administrador, Usuario
Actor Iniciador:	Usuario o Administrador
Pre-Condición:	Tener una sesión activa en la aplicación.
Flujo de Sucesos:	1. El actor hace clic en el botón 'Salir' ubicado en la barra superior junto a su nombre. 2. El sistema elimina los datos de sesión y el token almacenados en el navegador (localStorage). 3. El sistema redirige automáticamente al espacio público de consulta de aplicaciones. 4. La interfaz oculta las opciones reservadas para usuarios autenticados.
Post-Condición:	Sesión finalizada; estado de autenticación reiniciado a visitante público.
Excepciones:	Pérdida de conectividad (la sesión se destruye localmente en el cliente sin bloquear la navegación).

CU-03 Registrar usuario



CASO DE USO:	CU-03 Registrar usuario
Propósito:	Permitir a una persona crear una nueva cuenta de analista en la plataforma.
Resumen:	El usuario completa el formulario con su nombre de usuario, correo electrónico y contraseña para darse de alta.
Actores:	Usuario
Actor Iniciador:	Usuario
Pre-Condición:	El nombre de usuario o correo electrónico no deben estar registrados previamente en el sistema.
Flujo de Sucesos:	1. El actor hace clic en 'Registrarse'. 2. El sistema despliega el formulario de registro solicitando datos de cuenta. 3. El actor introduce nombre de usuario, correo electrónico y contraseña. 4. Hace clic en 'Crear cuenta'. 5. El sistema valida la unicidad de los datos y registra la cuenta (POST /api/v1/reputation/auth/register/) en la tabla auth_user. 6. El sistema inicia la sesión automáticamente con la nueva cuenta creada.
Post-Condición:	Nueva cuenta creada con rol de analista estándar y sesión iniciada en la plataforma.
Excepciones:	El nombre de usuario o correo ya existen en el sistema. Contraseñas no coinciden o longitud insuficiente.

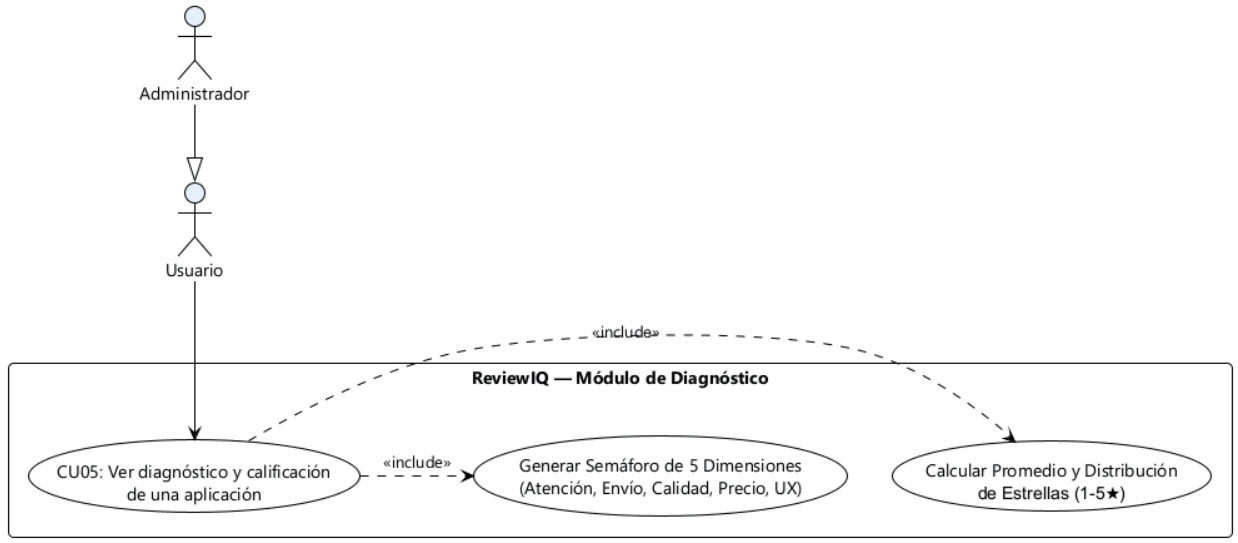
CU-04 Explorar catálogo de aplicaciones por categoría



CASO DE USO:	**CU-04 Explorar catálogo de aplicaciones por categoría**
Propósito:	Permitir al usuario consultar las aplicaciones registradas filtradas por su rubro comercial.
Resumen:	El usuario navega por las categorías de Compras, Delivery, Finanzas y Juegos, visualizando las métricas generales de cada app.
Actores:	Usuario, Administrador
Actor Iniciador:	Usuario
Pre-Condición:	Debe existir al menos una aplicación registrada en la base de datos.
Flujo de Sucesos:	1. El actor accede a la sección principal de aplicaciones. 2. Selecciona un filtro por categoría (Compras, Delivery, Finanzas o Juegos). 3. El sistema realiza una petición GET a /api/v1/reputation/entities/?categoria=... 4. La interfaz renderiza las tarjetas de las aplicaciones con su logo, nombre, calificación promedio y total de reseñas analizadas. 5. El actor hace clic en una aplicación para consultar su diagnóstico completo.
Post-Condición:	La aplicación seleccionada queda activa en pantalla para visualizar su detalle analítico.

Excepciones:	No existen aplicaciones en la categoría seleccionada (el sistema presenta un estado vacío informativo).

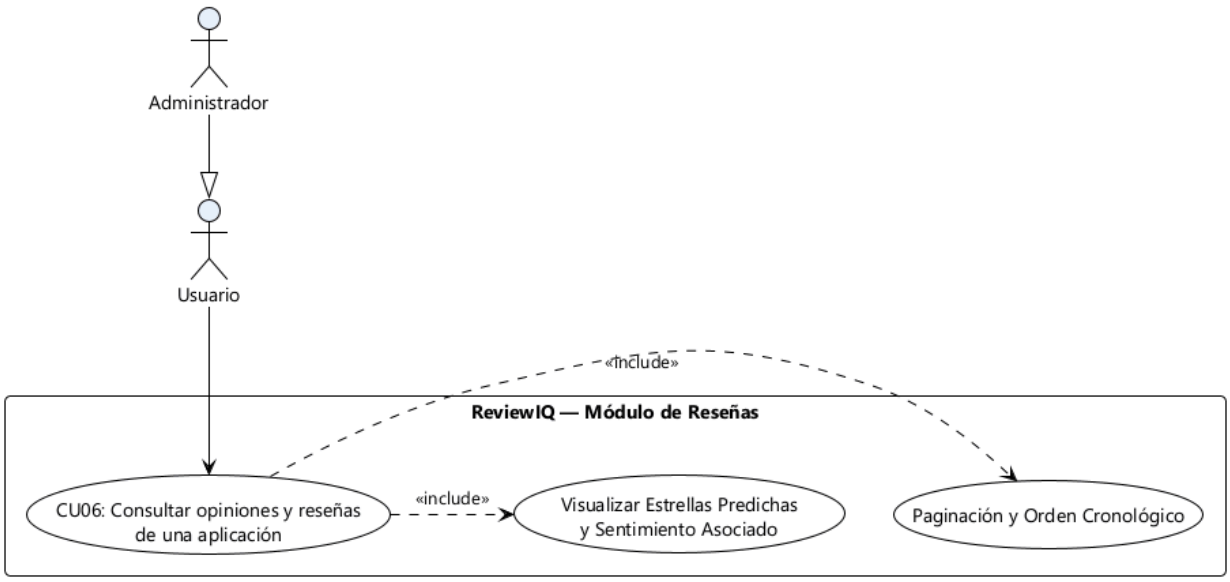
CU-05 Ver diagnóstico y calificación de una aplicación



CASO DE USO:	**CU-05 Ver diagnóstico y calificación de una aplicación**
Propósito:	Mostrar un informe analítico completo sobre la calidad de una aplicación evaluada en 5 aspectos de experiencia.
Resumen:	El sistema calcula y muestra porcentajes y etiquetas (Óptimo, En Atención, Crítico) para: Atención al cliente, Envíos y logística, Calidad del producto, Relación precio/valor y Experiencia de uso (UX).
Actores:	Usuario, Administrador
Actor Iniciador:	Usuario
Pre-Condición:	La aplicación debe estar registrada y contar con opiniones analizadas en la base de datos.
Flujo de Sucesos:	1. El actor hace clic en una aplicación del catálogo. 2. El sistema consulta el endpoint GET /api/v1/reputation/entities/<id>/diagnostic/.

	3. El servidor agrega las menciones y computa los puntajes por aspecto. 4. Muestra la calificación promedio general (1.0 a 5.0) y el total de opiniones evaluadas. 5. Renderiza las barras de porcentaje y badges de estado para cada dimensión con menciones. 6. Presenta el desglose porcentual de 1 a 5 estrellas y la lista cronológica de opiniones recientes.
Post-Condición:	El usuario visualiza el informe cualitativo y cuantitativo detallado de la aplicación.
Excepciones:	Dimensión sin menciones (0 opiniones): el sistema oculta esa barra automáticamente. Aplicación sin opiniones: se sugiere sincronizar reseñas desde Google Play.

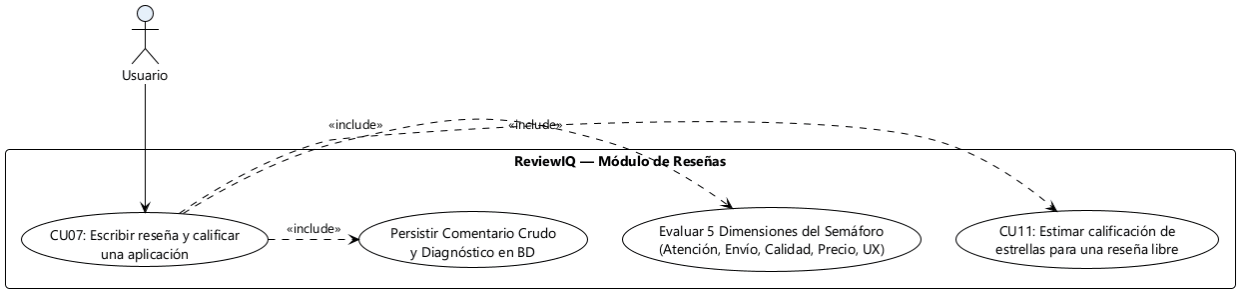
CU-06 Consultar opiniones y reseñas de una aplicación



CASO DE USO:	**CU-06 Consultar opiniones y reseñas de una aplicación**

Propósito:	Permitir al usuario leer las reseñas reales recopiladas para la aplicación seleccionada.
Resumen:	El usuario revisa la lista de comentarios de otros usuarios, observando el texto, la fecha y la calificación asignada a cada uno.
Actores:	Usuario, Administrador
Actor Iniciador:	Usuario
Pre-Condición:	Tener seleccionada una aplicación en la pantalla de detalle.
Flujo de Sucesos:	1. El actor se desplaza a la sección inferior de la ficha de la aplicación. 2. El sistema realiza una consulta GET a /api/v1/reputation/entities/<id>/reviews/. 3. El servidor retorna las reseñas ordenadas descendentemente por fecha de publicación. 4. La interfaz renderiza cada comentario mostrando el texto de la opinión, autor y estrellas asignadas. 5. El actor puede leer y auditar las valoraciones históricas recopiladas.
Post-Condición:	Listado de opiniones de la aplicación desplegado en pantalla.
Excepciones:	La aplicación aún no cuenta con reseñas registradas (se muestra aviso informativo).

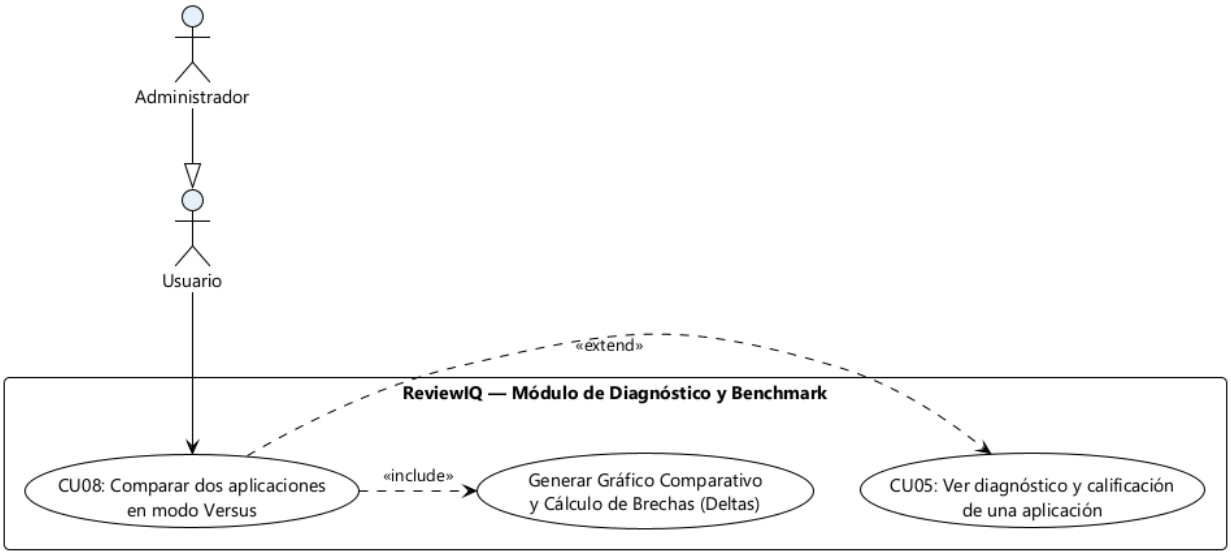
CU-07 Escribir reseña y calificar una aplicación



CASO DE USO:	**CU-07 Escribir reseña y calificar una aplicación**
Propósito:	Permitir al usuario agregar una opinión manual para una aplicación y calcular automáticamente cuántas estrellas merece.

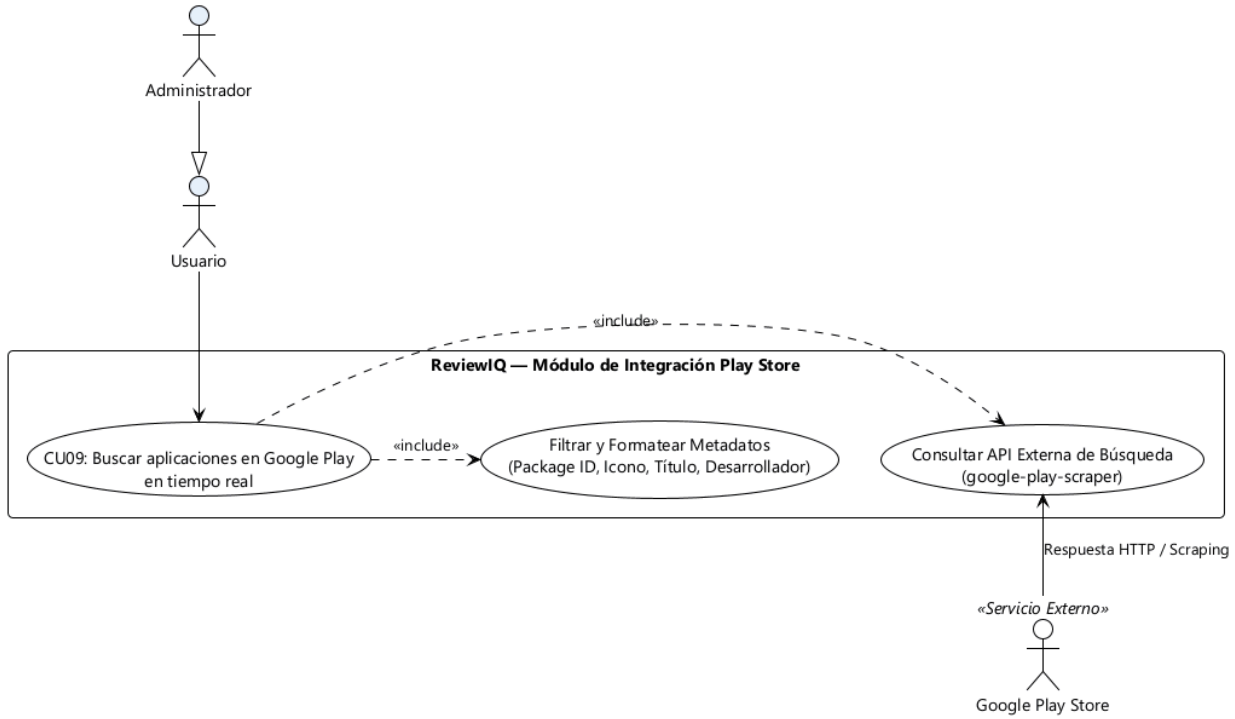
Resumen:	El usuario redacta un comentario; el modelo de Machine Learning infiere su calificación y actualiza el diagnóstico de la aplicación.
Actores:	Usuario
Actor Iniciador:	Usuario
Pre-Condición:	Haber seleccionado una aplicación en la pantalla de detalle.
Flujo de Sucesos:	1. El actor hace clic en el botón 'Escribir Reseña'. 2. Redacta su comentario expresando su experiencia personal sobre el servicio. 3. Hace clic en 'Publicar'. 4. El backend recibe el texto, clasifica la opinión mediante el modelo de NLP (POST /api/v1/reputation/entities/<id>/reviews/) y asigna las estrellas estimadas (1 a 5). 5. El comentario y el diagnóstico generado se guardan en la base de datos PostgreSQL. 6. El diagnóstico global de la aplicación se recalcula y se actualiza en pantalla en tiempo real.
Post-Condición:	La nueva opinión queda guardada y sumada a las métricas e historial de la aplicación.
Excepciones:	Texto con menos de 3 caracteres (se solicita ingresar un comentario más descriptivo).

CU-08 Comparar dos aplicaciones



CASO DE USO:	**CU-08 Comparar dos aplicaciones**
Propósito:	Realizar una comparación analítica lado a lado entre dos aplicaciones para identificar fortalezas y debilidades competitivas.
Resumen:	El actor selecciona dos entidades comerciales; el sistema calcula sus diferenciales en cada uno de los 5 aspectos y declara a la ganadora.
Actores:	Usuario, Administrador
Actor Iniciador:	Usuario
Pre-Condición:	Deben existir al menos dos aplicaciones registradas con opiniones en el sistema.
Flujo de Sucesos:	1. El actor ingresa a la pestaña 'Comparativa Versus'. 2. Selecciona la 'Aplicación A' en el primer menú desplegable (ej. PedidosYa). 3. Selecciona la 'Aplicación B' en el segundo menú desplegable (ej. Yango). 4. Hace clic en 'Comparar aplicaciones'. 5. El backend procesa la solicitud en /api/v1/reputation/versus/?app_a=...&app_b=... 6. El sistema compara puntajes dimensionales y resalta visualmente cuál domina en cada aspecto de calidad.
Post-Condición:	Se genera el cuadro comparativo multidimensional y el veredicto general de satisfacción.
Excepciones:	Se selecciona la misma aplicación en ambos campos (el sistema deshabilita la comparación y solicita seleccionar dos distintas).

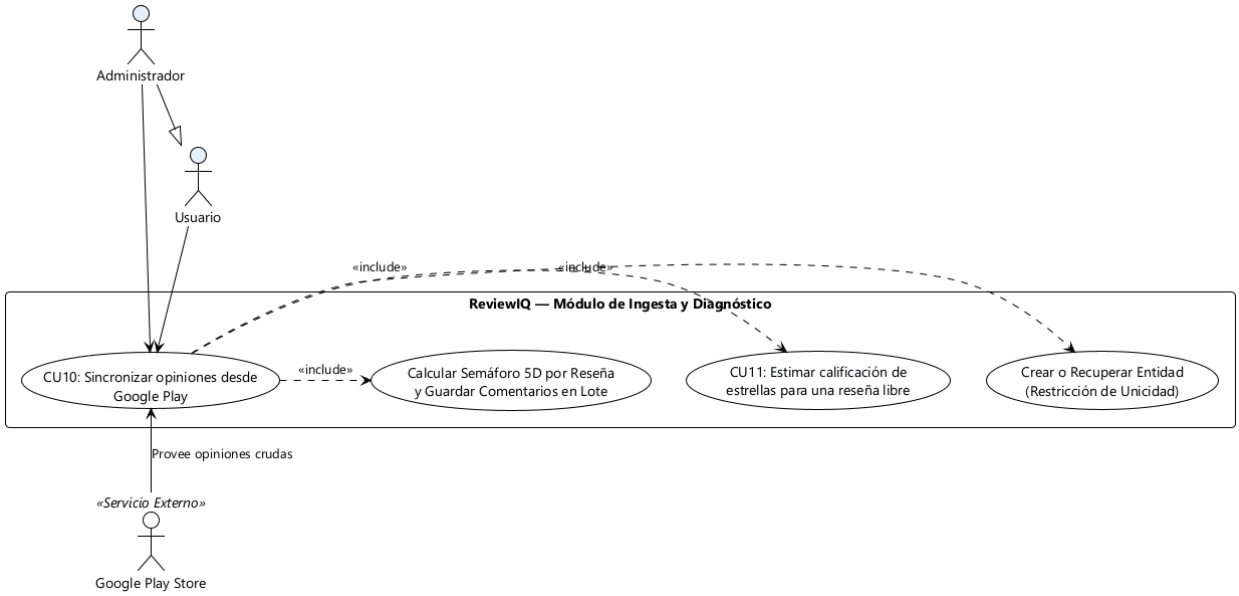
CU-09 Buscar aplicaciones en Google Play en tiempo real



CASO DE USO:	**CU-09 Buscar aplicaciones en Google Play en tiempo real**
Propósito:	Permitir al usuario buscar aplicaciones directamente en la tienda oficial Google Play Store por nombre comercial.
Resumen:	El usuario escribe un término en el buscador del modal y el sistema devuelve las aplicaciones coincidentes en tiempo real.
Actores:	Usuario, Administrador
Actor Iniciador:	Usuario o Administrador
Pre-Condición:	Conexión activa a Internet en el servidor backend.
Flujo de Sucesos:	1. El actor abre el modal de sincronización de Play Store. 2. Introduce el nombre de la app (ej. 'Uber', 'PedidosYa', 'Clash Royale'). 3. Hace clic en 'Buscar en Google Play'.

	4. El backend invoca el conector scraper (POST /api/v1/reputation/search/playstore/) consultando la API de Google Play. 5. El sistema devuelve el listado con icono oficial, título, desarrollador y package_id. 6. El actor selecciona la app requerida haciendo clic sobre su tarjeta para preparar la sincronización.
Post-Condición:	Aplicación seleccionada con identificador oficial de Android cargado en el formulario.
Excepciones:	No existen coincidencias para el término buscado. Error de comunicación con Google Play.

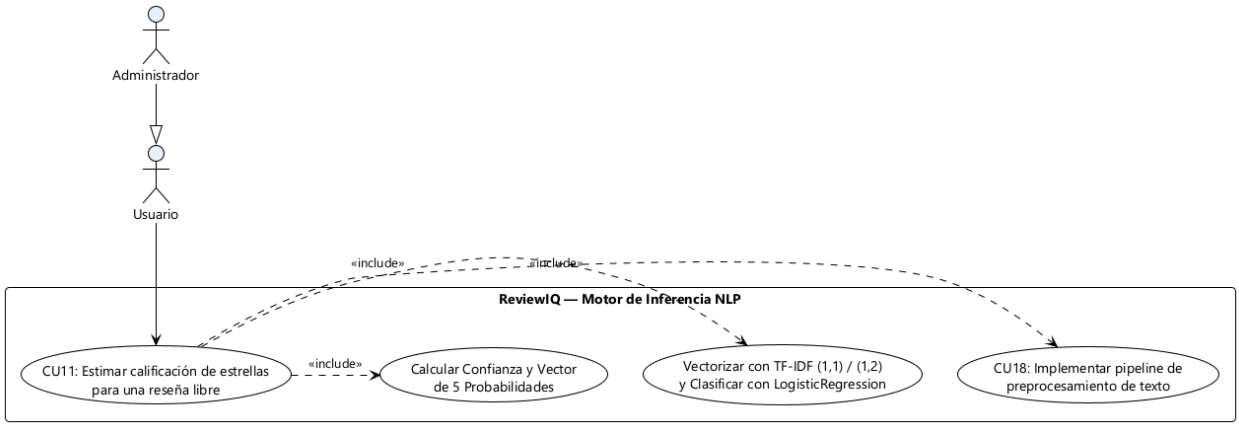
CU-10 Sincronizar opiniones desde Google Play



CASO DE USO:	**CU-10 Sincronizar opiniones desde Google Play**

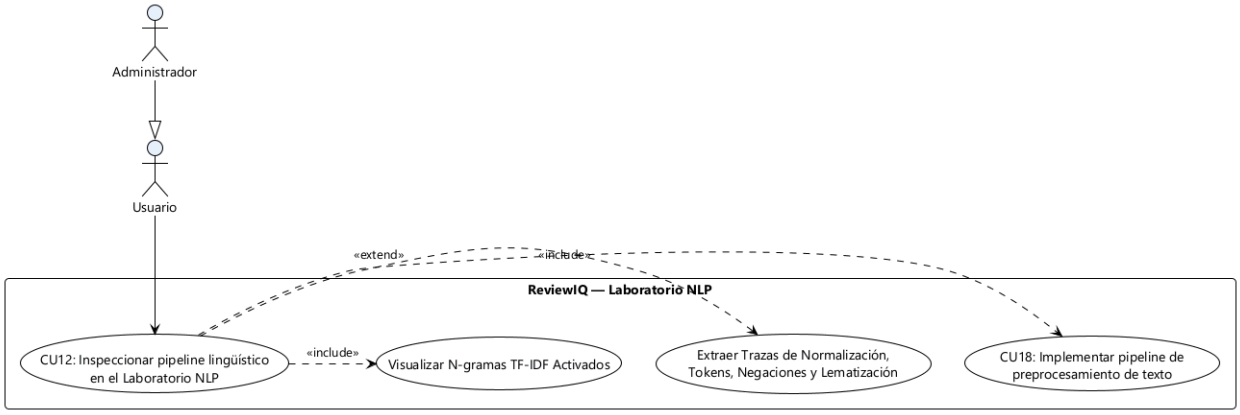
Propósito:	Extraer opiniones recientes de Google Play Store para una aplicación y analizarlas automáticamente con el clasificador de texto.
Resumen:	El sistema extrae hasta 100 opiniones reales mediante scraping en vivo, calcula sus estrellas con inteligencia artificial y las almacena.
Actores:	Administrador, Usuario, Google Play Store
Actor Iniciador:	Usuario o Administrador
Pre-Condición:	Conexión a Internet activa y Package ID de Android válido (ej. com.ubercab).
Flujo de Sucesos:	1. El actor abre el modal de sincronización e introduce o busca la aplicación deseada. 2. Define la cantidad de opiniones a importar (20, 50 o 100). 3. Hace clic en 'Iniciar Sincronización'. 4. El backend invoca el scraper para extraer las reseñas y metadatos oficiales en tiempo real (POST /api/v1/reputation/sync/playstore/). 5. El modelo clasifica cada reseña en estrellas (1 a 5) y categoriza los aspectos mencionados. 6. Las opiniones se guardan en PostgreSQL y se recalcula el diagnóstico de la aplicación. 7. El sistema confirma la sincronización exitosa y limpia los campos del modal.
Post-Condición:	Las opiniones quedan registradas en la base de datos y se actualiza el diagnóstico de la app.
Excepciones:	Aplicación inexistente en Google Play. Fallo de conexión a Internet durante la descarga.

CU-11 Estimar calificación de estrellas para una reseña libre



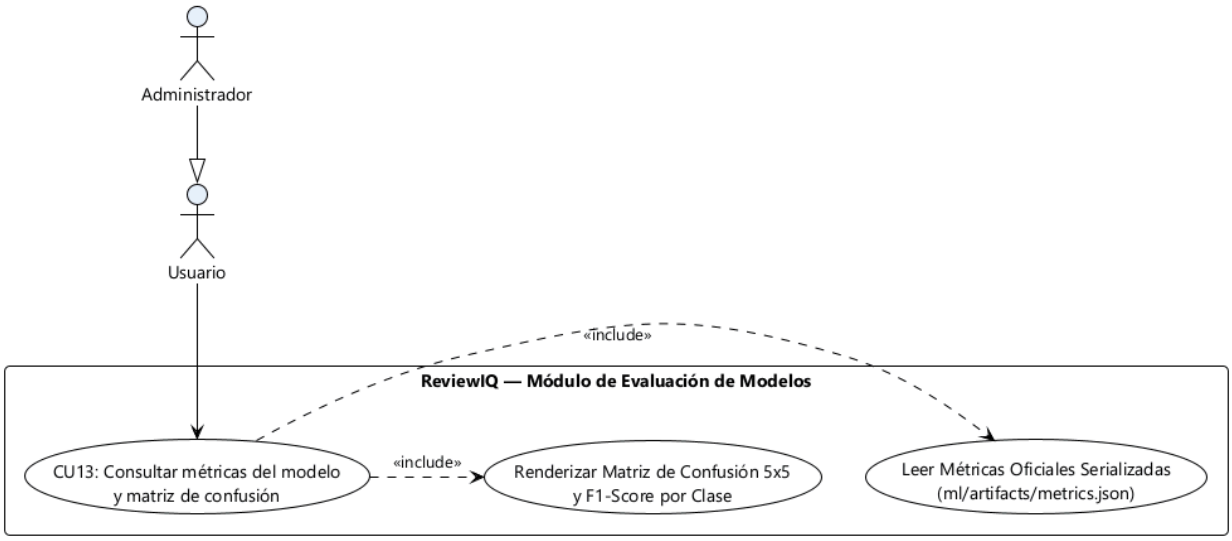
CASO DE USO:	**CU-11 Estimar calificación de estrellas para una reseña libre**
Propósito:	Evaluar cualquier texto libre sobre productos, servicios o aplicaciones y predecir de 1 a 5 estrellas con su distribución de probabilidades.
Resumen:	El usuario escribe o pega un comentario; el pipeline de NLP y el modelo de Regresión Logística calculan las estrellas y probabilidades.
Actores:	Usuario, Administrador
Actor Iniciador:	Usuario
Pre-Condición:	El artefacto del modelo serializado (joblib) debe estar cargado en memoria en el servidor.
Flujo de Sucesos:	1. El actor accede a la sección 'Predicción Clásica'. 2. Escribe una reseña en el cuadro de texto. 3. Hace clic en 'Analizar texto'. 4. El sistema valida la longitud y envía una petición POST a /api/v1/predict/. 5. El motor de NLP normaliza el texto, extrae n-gramas TF-IDF y la Regresión Logística predice las probabilidades. 6. La interfaz muestra la cantidad de estrellas predicha (de 1 a 5), las probabilidades por clase y el tiempo de respuesta en milisegundos. 7. El actor puede presionar 'Inspeccionar pipeline' para visualizar el análisis lingüístico en el laboratorio.
Post-Condición:	Se presenta el resultado inferido en pantalla con sus probabilidades asociadas.
Excepciones:	Texto vacío o que supera el límite de longitud permitido (retorna HTTP 400).

CU-12 Inspeccionar pipeline lingüístico en el Laboratorio NLP



CASO DE USO:	**CU-12 Inspeccionar pipeline lingüístico en el Laboratorio NLP**
Propósito:	Permitir al usuario observar detalladamente cómo la computadora limpia, transforma y tokeniza el texto de una opinión.
Resumen:	El sistema expone de forma interactiva cada etapa: minúsculas, emojis convertidos a palabras, palabras de negación protegidas y tokens finales.
Actores:	Usuario, Administrador
Actor Iniciador:	Usuario
Pre-Condición:	Servicio de trazas activo en /api/v1/nlp/trace/.
Flujo de Sucesos:	1. El actor accede a la pestaña 'Laboratorio NLP'. 2. Escribe una oración con expresiones informales o emojis (ej. '¡No me gustó la entrega! 😡'). 3. Hace clic en 'Analizar traza'. 4. El servidor ejecuta cada etapa de transformación registrando estados intermedios. 5. La interfaz gráfica muestra la secuencia de etapas y los tokens finales analizados junto con la latencia en milisegundos.
Post-Condición:	El usuario comprende el impacto del preprocesamiento en el vector de características.
Excepciones:	Texto vacío al presionar el botón de análisis.

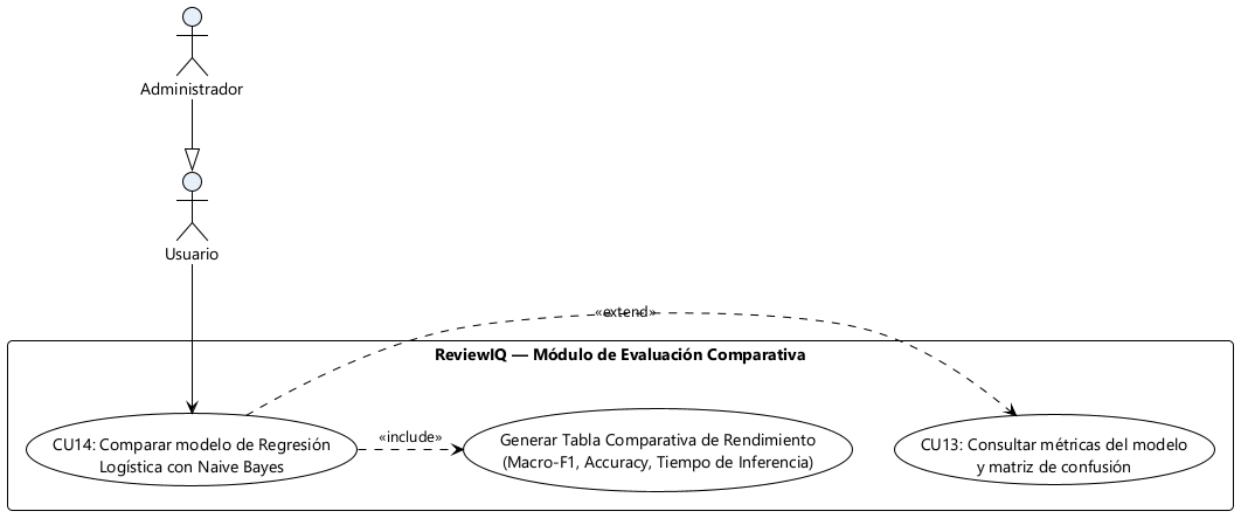
CU-13 Consultar métricas del modelo y matriz de confusión



CASO DE USO:	CU-13 Consultar métricas del modelo y matriz de confusión
Propósito:	Exponer las métricas de rendimiento del clasificador entrenado (Macro-F1, Accuracy, MAE) y su matriz de confusión formal.
Resumen:	El sistema presenta las métricas del modelo entrenado sobre el corpus Amazon Reviews Multi ES y la comparativa contra Naive Bayes.
Actores:	Usuario, Administrador
Actor Iniciador:	Usuario
Pre-Condición:	Archivo de reporte metrics.json generado durante el entrenamiento.
Flujo de Sucesos:	1. El actor solicita la información de métricas mediante GET a /api/v1/metrics/. 2. El servidor lee el manifiesto de evaluación. 3. Se presenta en pantalla el Macro-F1 (0.5131), Accuracy (51.54%), MAE (0.61 estrellas) y la matriz de confusión de 5x5 clases. 4. Muestra la tabla comparativa con el modelo base (Naive Bayes).
Post-Condición:	El usuario verifica el respaldo estadístico y nivel de error del modelo en producción.

Excepciones:	Archivo de métricas no disponible (se reporta estado degraded con código 503).

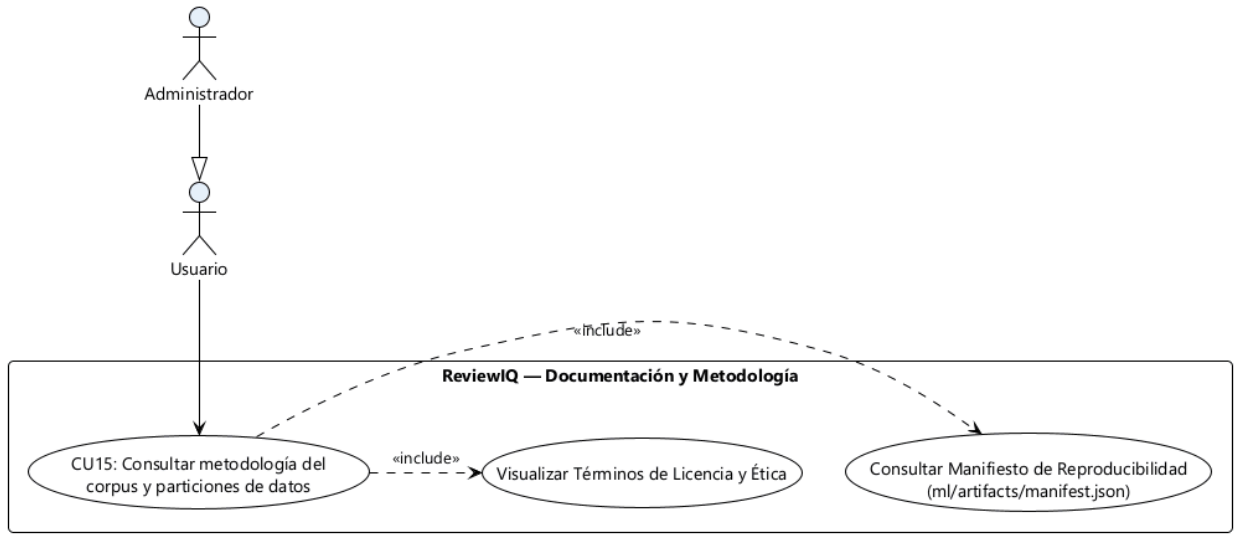
CU-14 Comparar modelo de Regresión Logística con Naive Bayes



CASO DE USO:	**CU-14 Comparar modelo de Regresión Logística con Naive Bayes**
Propósito:	Permitir contrastar el rendimiento del modelo seleccionado contra el algoritmo base (baseline).
Resumen:	El sistema muestra una comparativa técnica entre Naive Bayes y Regresión Logística multiclase evaluados sobre el mismo conjunto de datos.

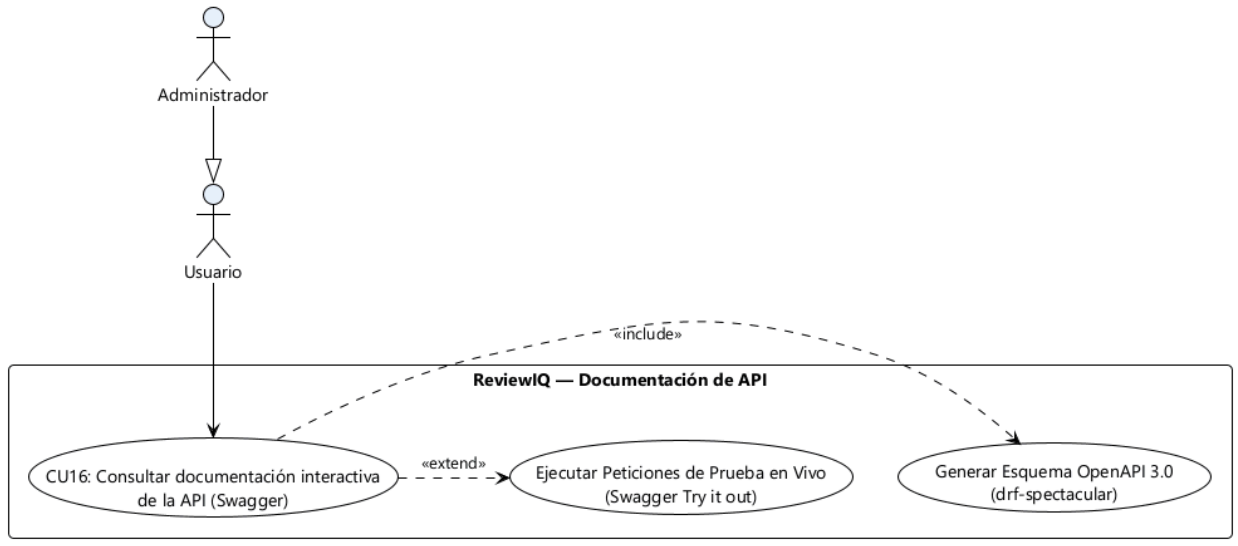
Actores:	Usuario, Administrador
Actor Iniciador:	Usuario
Pre-Condición:	Estar en la vista de Modelos y Métricas.
Flujo de Sucesos:	1. El actor selecciona la sección de comparación de modelos. 2. El sistema recupera los reportes de validación de ambos clasificadores. 3. Muestra una tabla comparativa con los valores de Accuracy, Macro-F1, MAE y tiempos de entrenamiento de ambos algoritmos. 4. El actor identifica por qué la Regresión Logística fue seleccionada para el entorno operativo final.
Post-Condición:	El usuario comprende la justificación experimental de la elección del modelo.
Excepciones:	Datos de comparación no disponibles.

CU-15 Consultar metodología del corpus y particiones de dato



CASO DE USO:	CU-15 Consultar metodología del corpus y particiones de datos
Propósito:	Proporcionar información sobre el conjunto de datos de entrenamiento (Amazon Reviews Multi ES) y la metodología de experimentación.
Resumen:	El usuario consulta los antecedentes teóricos del proyecto, la distribución de datos en español y las particiones de entrenamiento, validación y prueba.
Actores:	Usuario, Administrador
Actor Iniciador:	Usuario
Pre-Condición:	Estar en la plataforma web.
Flujo de Sucesos:	1. El actor ingresa a la pestaña 'Metodología'. 2. Lee los detalles de recolección y partición de las reseñas (200,000 entrenamiento, 5,000 validación, 5,000 prueba). 3. Consulta la distribución perfectamente balanceada (20% por cada clase de 1 a 5 estrellas). 4. Revisa la justificación matemática y lingüística del preprocesamiento.
Post-Condición:	El usuario conoce los fundamentos metodológicos del proyecto.
Excepciones:	Sección no accesible temporalmente.

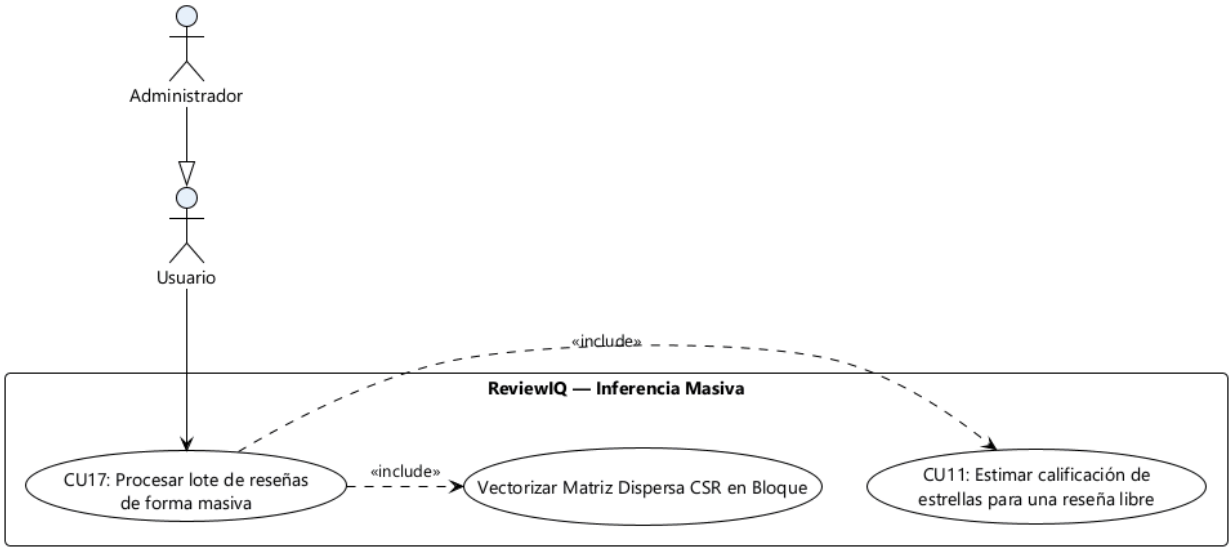
CU-16 Consultar documentación interactiva de la API



CASO DE USO:	CU-16 Consultar documentación interactiva de la API
Propósito:	Permitir a desarrolladores y evaluadores explorar los endpoints y especificaciones de la API REST mediante Swagger UI.

Resumen:	El usuario consulta los contratos de entrada y salida de cada servicio web directamente en la interfaz de Swagger / OpenAPI.
Actores:	Usuario, Administrador
Actor Iniciador:	Usuario o Administrador
Pre-Condición:	Servicio de documentación activo en /api/schema/swagger-ui/.
Flujo de Sucesos:	1. El actor hace clic en el enlace a la Documentación de la API. 2. El navegador despliega la interfaz interactiva de Swagger UI. 3. El actor puede inspeccionar cada endpoint, parámetros requeridos, esquemas JSON y códigos de respuesta (200, 400, 500). 4. El actor puede ejecutar llamadas de prueba en vivo contra el servidor.
Post-Condición:	El usuario conoce la especificación formal y contratos de la API.
Excepciones:	Servidor backend fuera de línea.

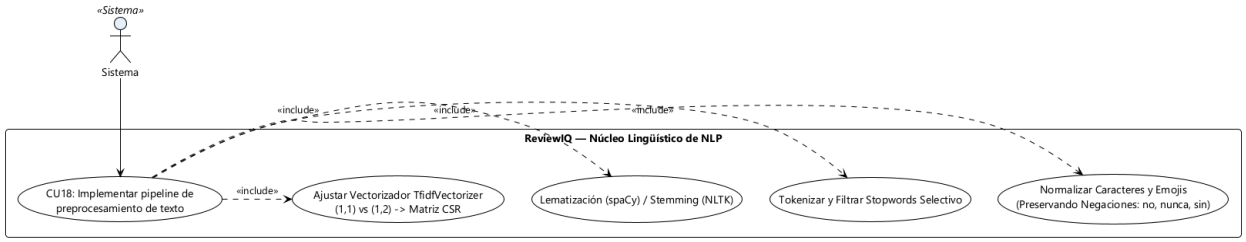
CU-17 Procesar lote de reseñas de forma masiva



CASO DE USO:	**CU-17 Procesar lote de reseñas de forma masiva**
Propósito:	Permitir analizar hasta 100 opiniones de una sola vez mediante procesamiento por lotes (Batch).
Resumen:	El sistema procesa una lista de textos de manera simultánea en una sola llamada a la API, optimizando el tiempo de cómputo.
Actores:	Usuario, Administrador, Sistema Externo
Actor Iniciador:	Usuario o Administrador
Pre-Condición:	Disponer de una lista de textos de opiniones con longitud válida.

Flujo de Sucesos:	1. El cliente envía un payload JSON a /api/v1/predict/batch/ con el array de textos. 2. El serializador valida que la lista contenga entre 1 y 100 elementos. 3. El pipeline vectoriza todos los textos en lote de forma optimizada en memoria. 4. La Regresión Logística predice estrellas y probabilidades simultáneamente mediante álgebra matricial. 5. Se retorna el arreglo de predicciones y el tiempo total de inferencia en ms.
Post-Condición:	El cliente recibe las predicciones individuales para cada reseña del lote.
Excepciones:	El arreglo supera los 100 elementos o contiene textos vacíos (retorna HTTP 400).

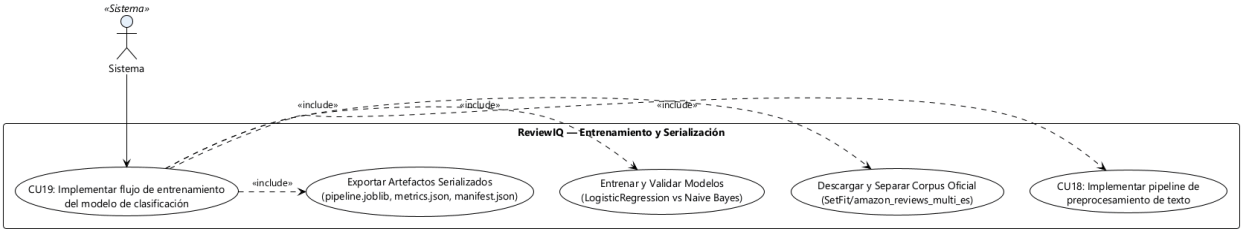
CU-18 Implementar pipeline de preprocesamiento de texto



CASO DE USO:	**CU-18 Implementar pipeline de preprocesamiento de texto**
Propósito:	Construir y aplicar la secuencia ordenada de transformaciones lingüísticas sobre el texto de las reseñas para convertir texto no estructurado en representaciones matemáticas aptas para el modelo.
Resumen:	El sistema ejecuta secuencialmente la normalización Unicode NFKC, conversión a minúsculas, eliminación de tildes, traducción léxica de emojis a texto, preservación estricta de palabras vacías de negación (no, nunca, jamás) y la extracción de características mediante unigramas y bigramas en una matriz TF-IDF.
Actores:	Ingeniero de ML, Administrador
Actor Iniciador:	Ingeniero de ML
Pre-Condición:	Disponer de textos de reseñas en español en memoria o archivo de datos crudos.

Flujo de Sucesos:	1. El actor invoca el pipeline de preprocesamiento pasando el texto crudo de la reseña. 2. El módulo aplica normalización Unicode NFKC y conversión de caracteres a minúsculas. 3. El sistema identifica emojis y los traduce a sus equivalentes semánticos en texto (ej. 👍 a 'bueno', 😡 a 'enojado'). 4. El sistema remueve acentos y caracteres especiales sin alterar la estructura léxica esencial. 5. Se filtran las palabras vacías (stop words) en español, protegiendo estrictamente las partículas de negación (no, nunca, jamás, tampoco). 6. Se realiza la tokenización y extracción combinada de unigramas y bigramas. 7. El vectorizador TF-IDF transforma los tokens en una matriz de frecuencias ponderadas por documento.
Post-Condición:	El texto queda transformado en un vector numérico TF-IDF normalizado listo para la inferencia del clasificador.
Excepciones:	Texto con caracteres no interpretables o cadena vacía (el pipeline maneja entradas anómalas asignando un vector disperso de ceros).

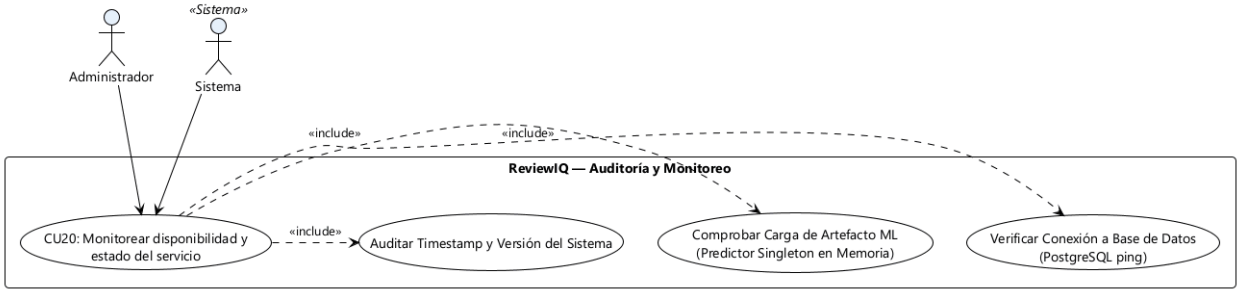
CU-19 Implementar flujo de entrenamiento del modelo de clasificación



CASO DE USO:	**CU-19 Implementar flujo de entrenamiento del modelo de clasificación**
Propósito:	Construir el flujo automatizado y reproducible que entrena los algoritmos supervisados, evalúa su desempeño estadístico y empaqueta el artefacto final congelado.
Resumen:	El script reproducible toma el corpus balanceado de Amazon Reviews Multi ES, alimenta el clasificador de Regresión Logística multiclase (y el clasificador base Naive Bayes), calcula las métricas de evaluación (Macro-F1, Accuracy, MAE, Matriz de Confusión) y exporta el modelo serializado (predictor.joblib) y sus métricas (metrics.json) para el backend.
Actores:	Ingeniero de ML, Administrador
Actor Iniciador:	Ingeniero de ML

Pre-Condición:	Disponer del corpus particionado y preprocesado en disco (200,000 entrenamiento, 5,000 validación, 5,000 prueba).
Flujo de Sucesos:	1. El actor ejecuta el script reproducible de entrenamiento desde la terminal (python -m ml.train o script equivalente). 2. El flujo carga las particiones del dataset y aplica el pipeline de preprocesamiento (CU-18) sobre cada partición. 3. El sistema entrena el modelo de Regresión Logística multiclase ajustando el hiperparámetro de regularización (C=1.0). 4. El sistema entrena el clasificador base Multinomial Naive Bayes para fines de comparación técnica. 5. Se ejecuta la evaluación sobre el conjunto de prueba, calculando Macro-F1 (0.5131), Accuracy (51.54%), MAE (0.61) y la matriz de confusión multiclase de 5x5. 6. El flujo serializa y congela el objeto predictor en disco como predictor.joblib. 7. Se genera el archivo de reporte metrics.json con los metadatos y resultados para su exposición vía API REST.
Post-Condición:	Modelo empaquetado en disco y métricas generadas, listos para ser consumidos por el backend en producción.
Excepciones:	Falta de memoria RAM o archivo de corpus no encontrado en la ruta especificada (el flujo aborta emitiendo un mensaje de error descriptivo en consola).

CU-20 Monitorear disponibilidad y estado del servicio



CASO DE USO:	**CU-20 Monitorear disponibilidad y estado del servicio**
Propósito:	Verificar de forma automática el estado operativo del servidor web, la base de datos y el modelo de inteligencia artificial.
Resumen:	El sistema expone un punto de verificación de salud (/api/v1/health/) para comprobar que todos los componentes respondan oportunamente.
Actores:	Administrador, Sistema
Actor Iniciador:	Administrador o Monitor

Pre-Condición:	Servidor backend en funcionamiento.
Flujo de Sucesos:	1. El actor o sistema monitor envía una petición GET a /api/v1/health/. 2. El servidor comprueba la conectividad con la base de datos PostgreSQL. 3. Verifica que el objeto predictor de Machine Learning esté cargado en memoria RAM. 4. Retorna HTTP 200 con payload: {status: 'ok', model_loaded: true, model_id: '...'}. 5. La interfaz actualiza el indicador de conexión en verde.
Post-Condición:	Se confirma el correcto funcionamiento de toda la infraestructura.
Excepciones:	El artefacto del modelo no se encuentra en disco o memoria (retorna HTTP 503).

1. Diagrama General de Casos de Uso

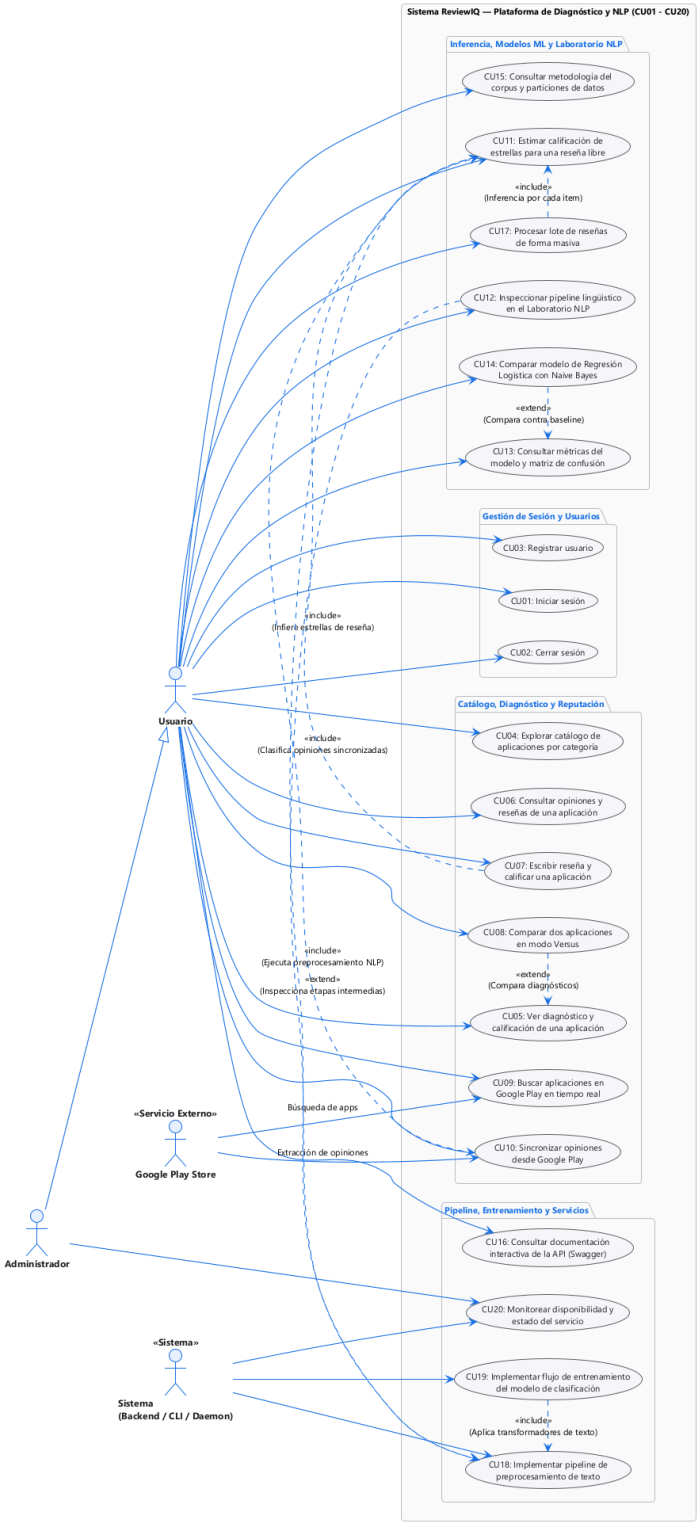
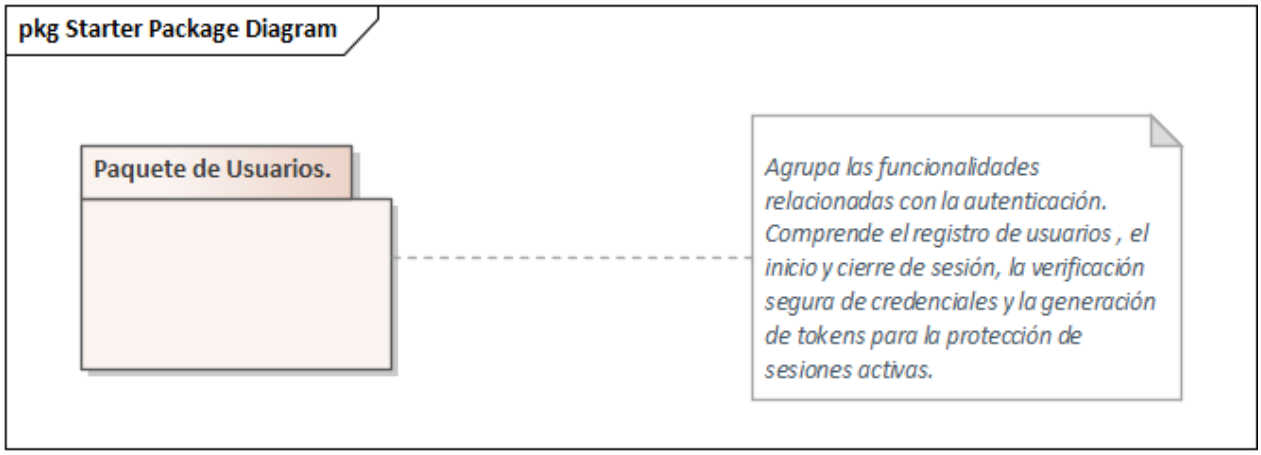


Figura 3. Diagrama General de Casos de Uso

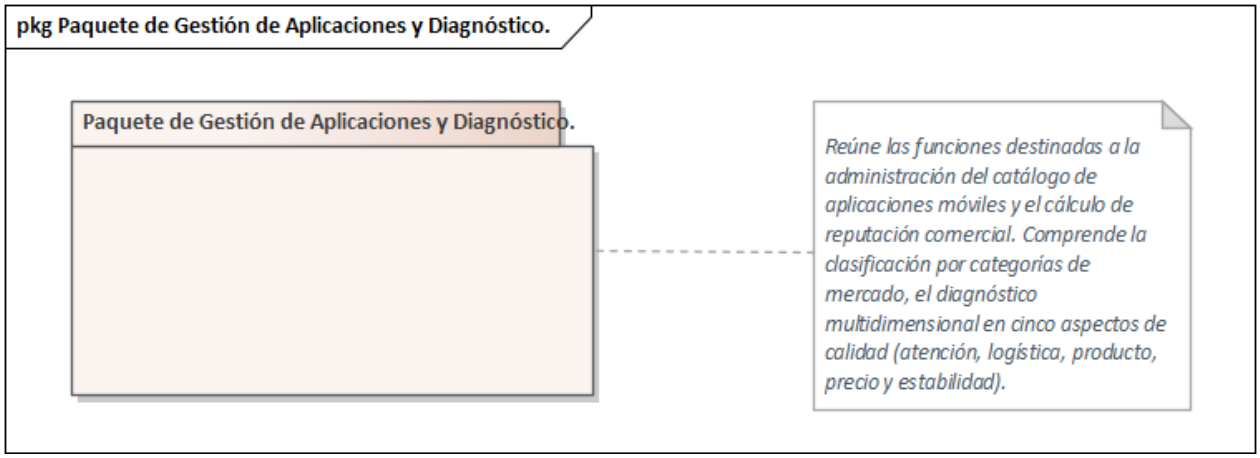
2.2 Flujo de Trabajo: Análisis

2.2.1 Análisis de la Arquitectura

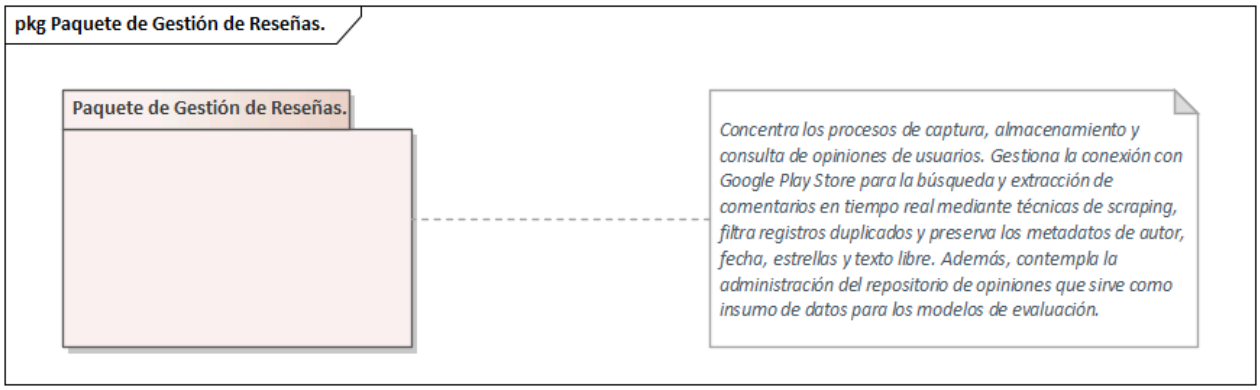
Paquete de Usuarios.



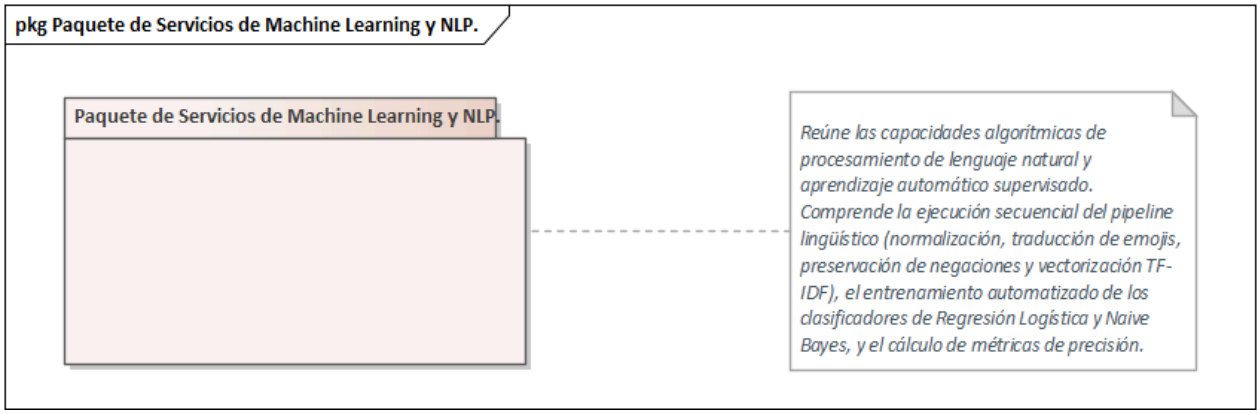
Paquete de Gestión de Aplicaciones y Diagnóstico.



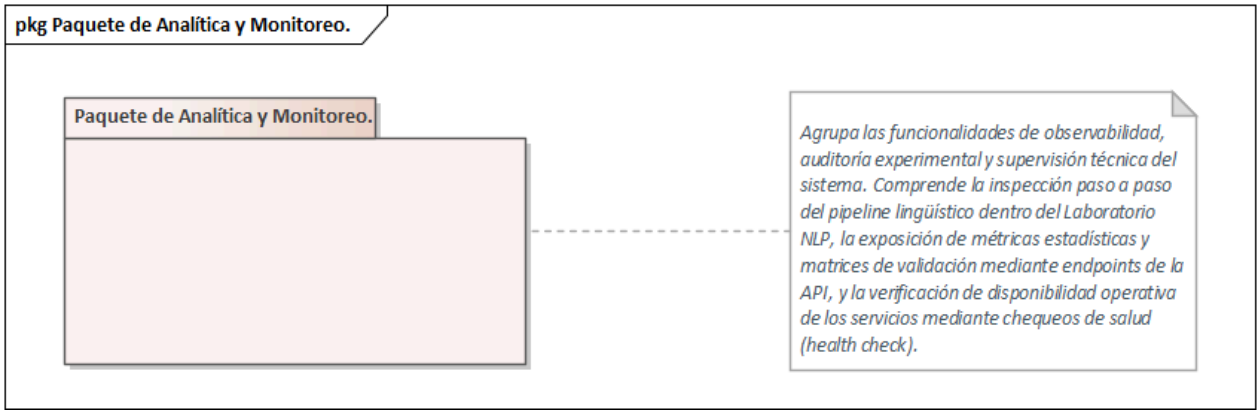
Paquete de Gestión de Reseñas.



Paquete de Servicios de Machine Learning y NLP.

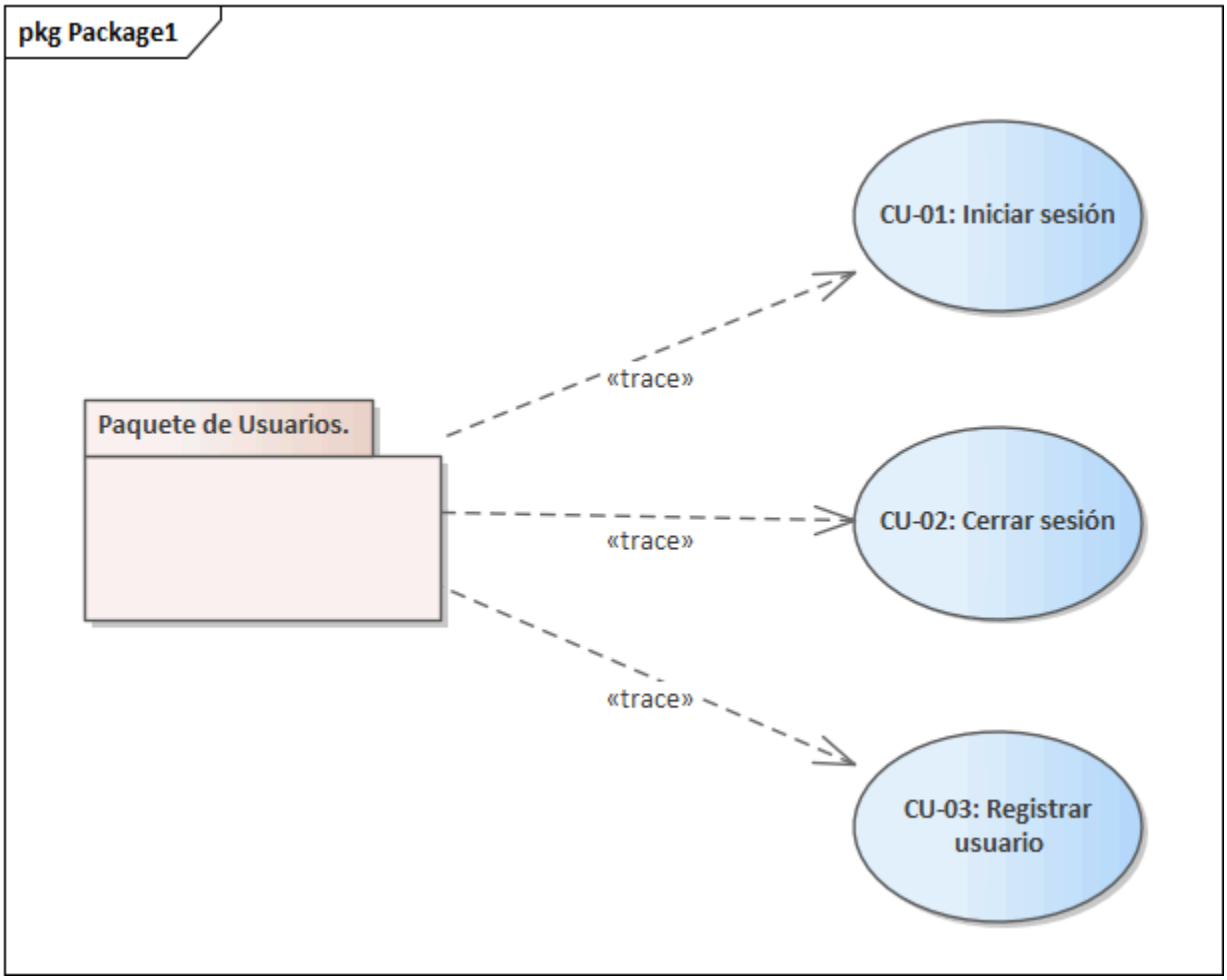


Paquete de Analítica y Monitoreo.

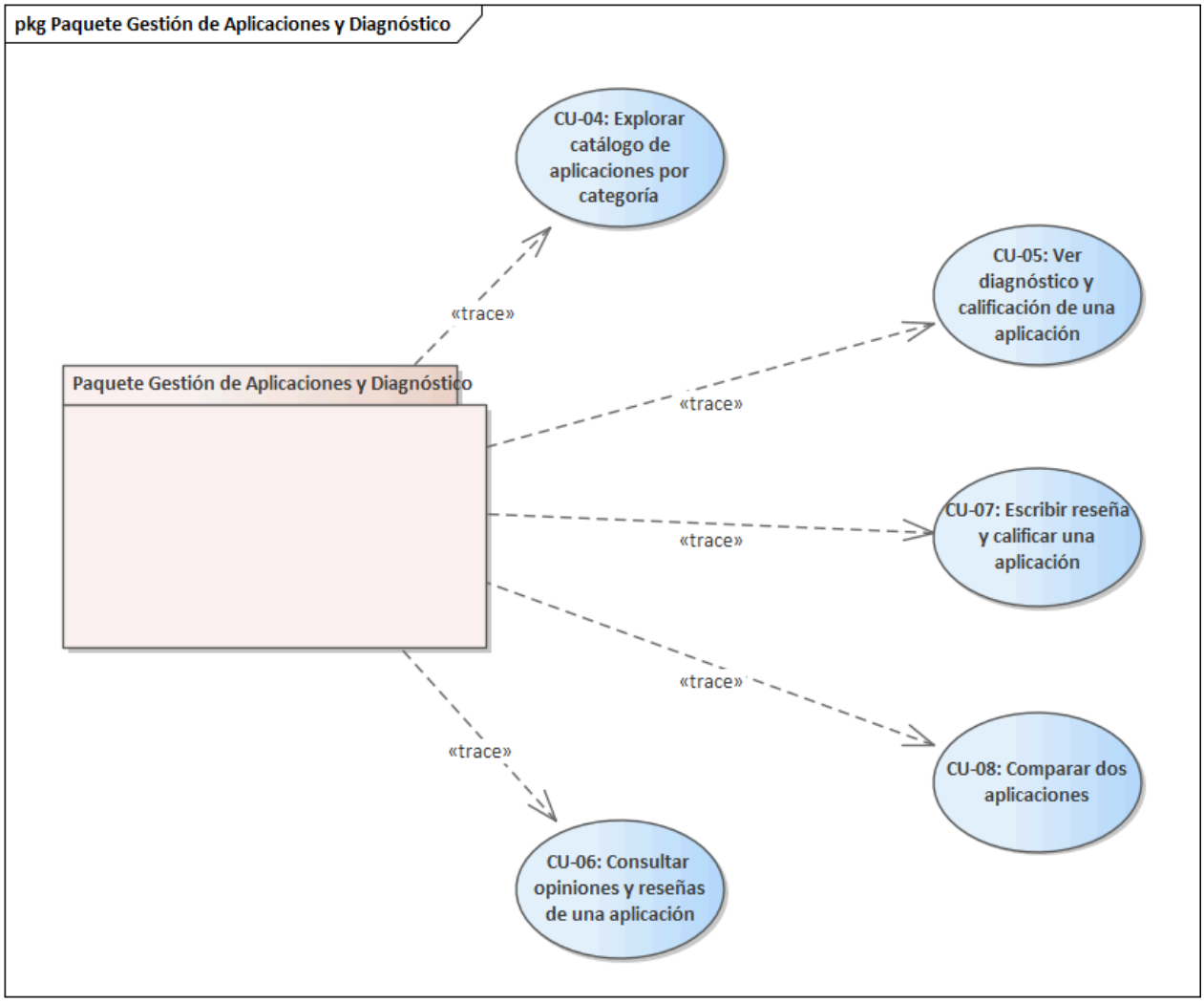


2.2.2 Relación de Paquetes

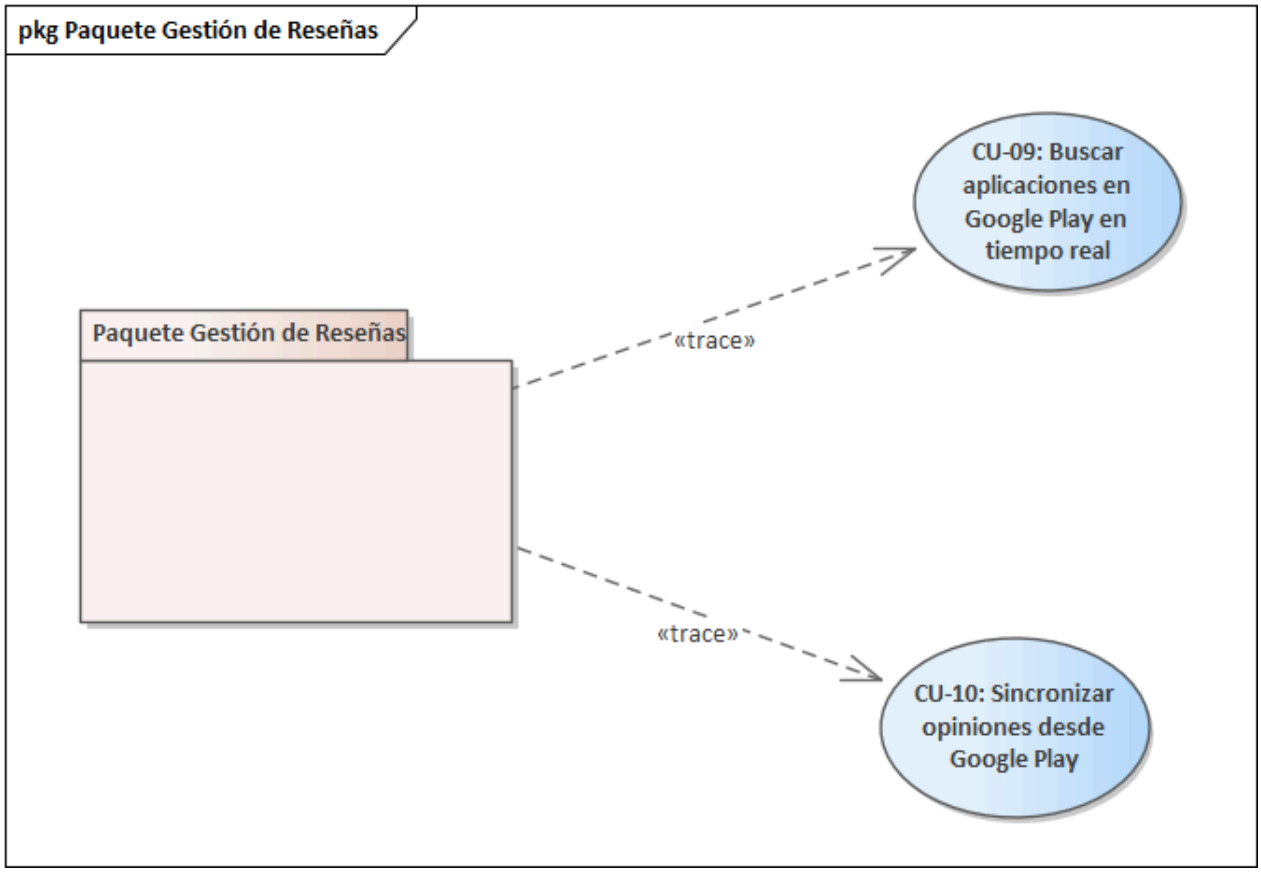
Paquete de Usuarios



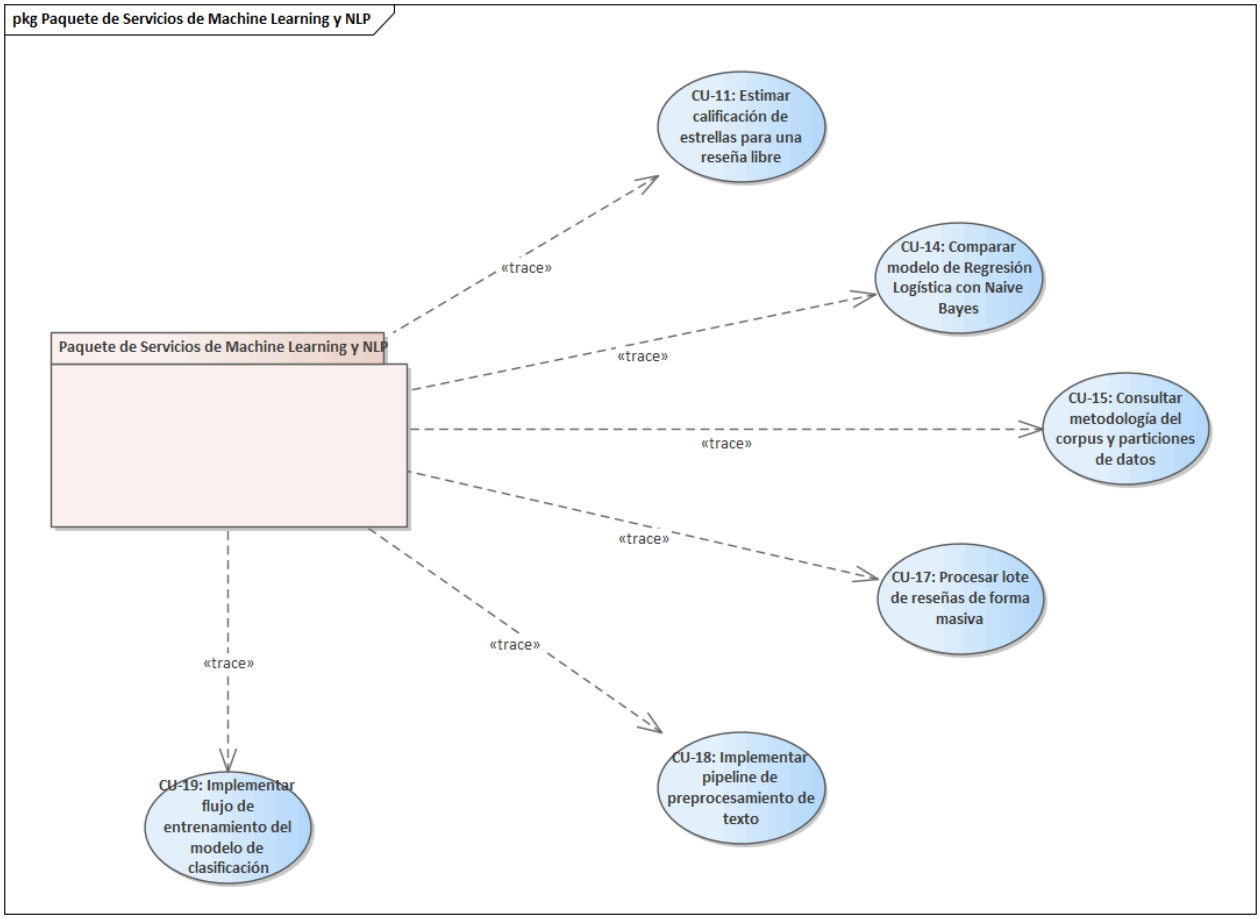
Paquete de Gestión de Aplicaciones y Diagnóstico



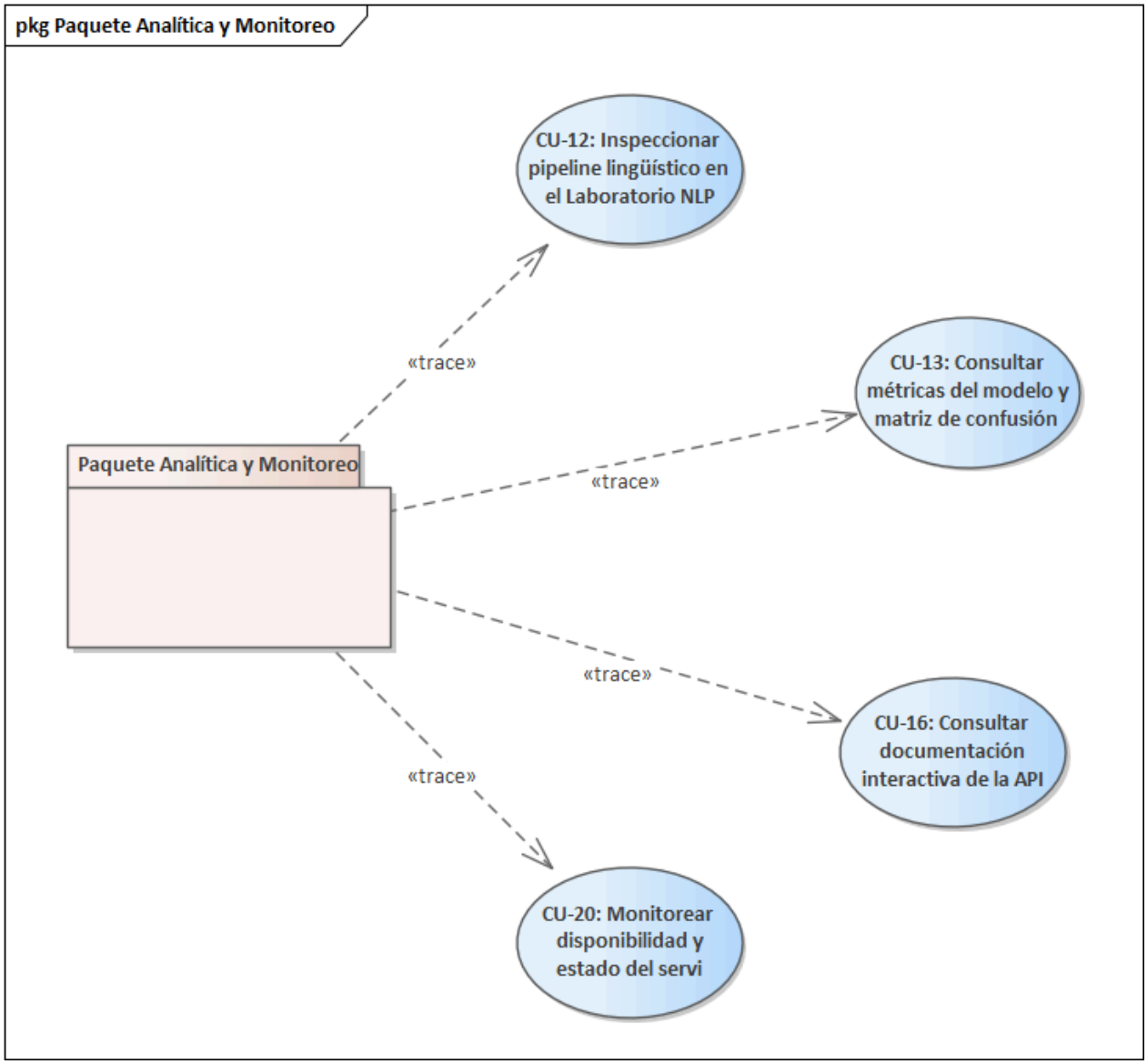
Paquete de Gestión de Reseñas



Paquete de Servicios de Machine Learning y NLP



Paquete de Analítica y Monitoreo


pkg Paquete Analítica y Monitoreo
CU-12: Inspeccionar pipeline lingüístico en el Laboratorio NLP
CU-13: Consultar métricas del modelo y matriz de confusión
Paquete Analítica y Monitoreo
«trace»
«trace»
«trace»
«trace»
CU-16: Consultar documentación interactiva de la API
CU-20: Monitorear disponibilidad y estado del servi

2. Análisis de casos de uso

2.1.CU01 Iniciar sesión

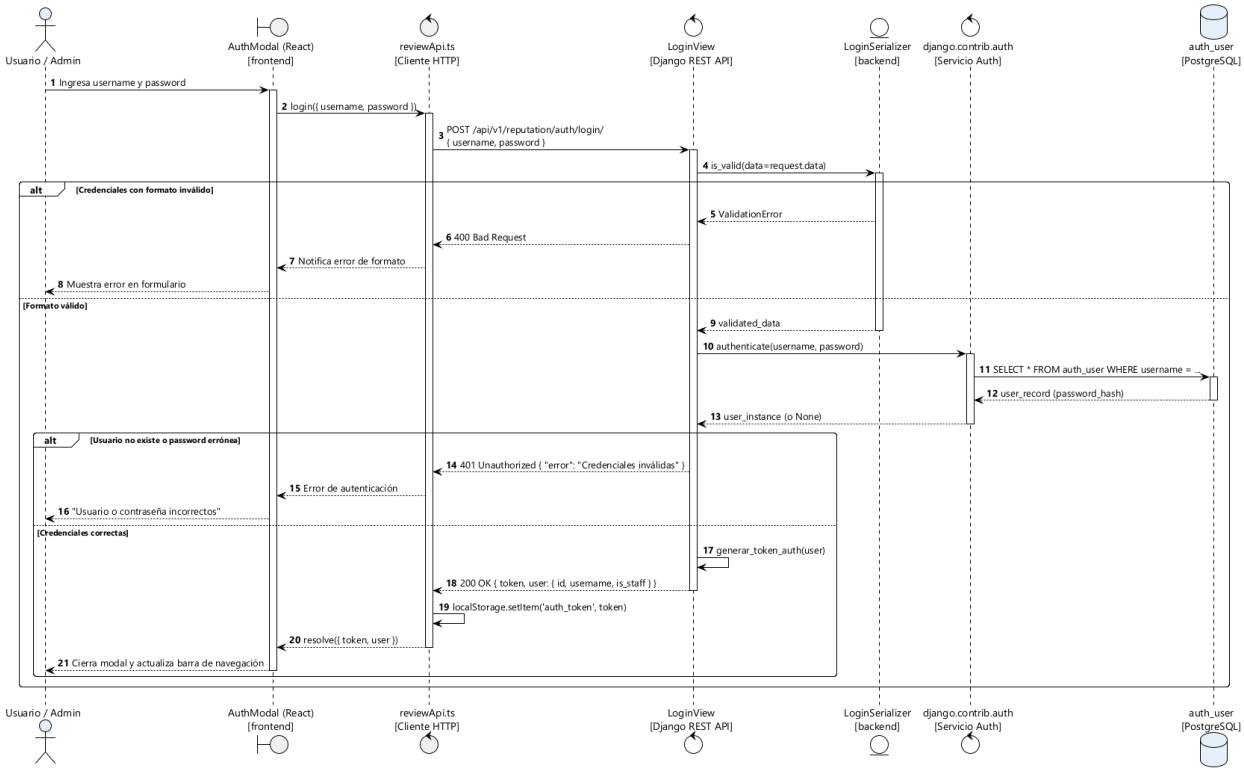


Figura 9. Análisis de casos de uso de Iniciar sesión

2.2.CU02 Cerrar sesión

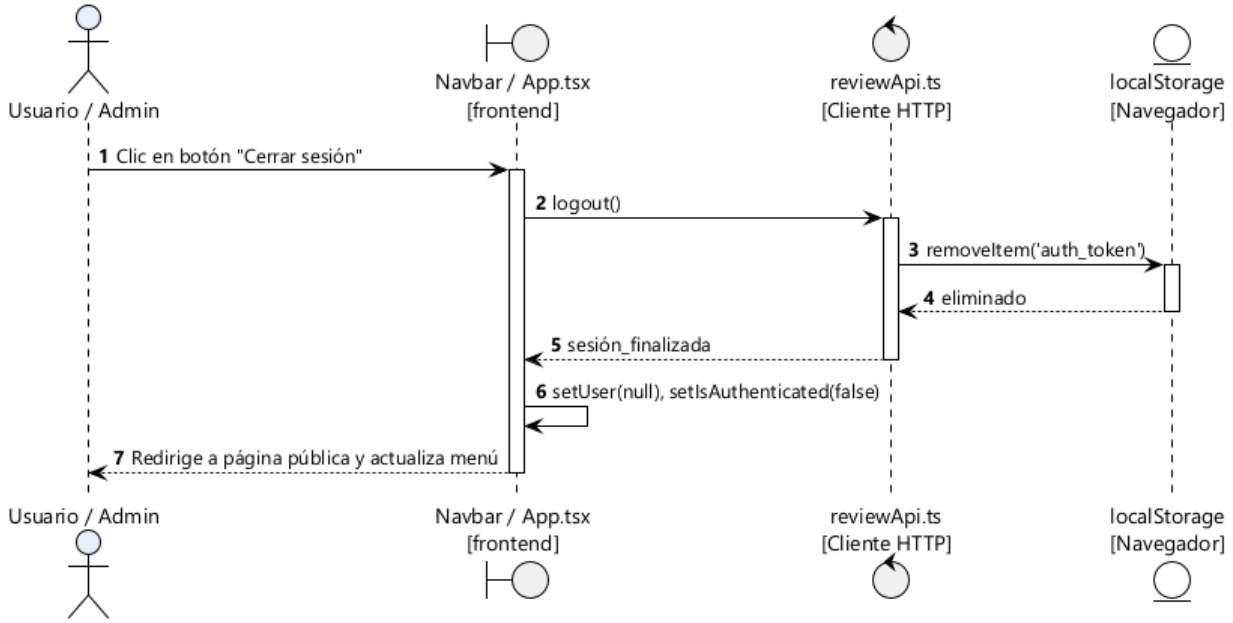


Figura 10. Análisis de casos de uso de Cerrar sesión

2.3.CU03 Registrar usuario

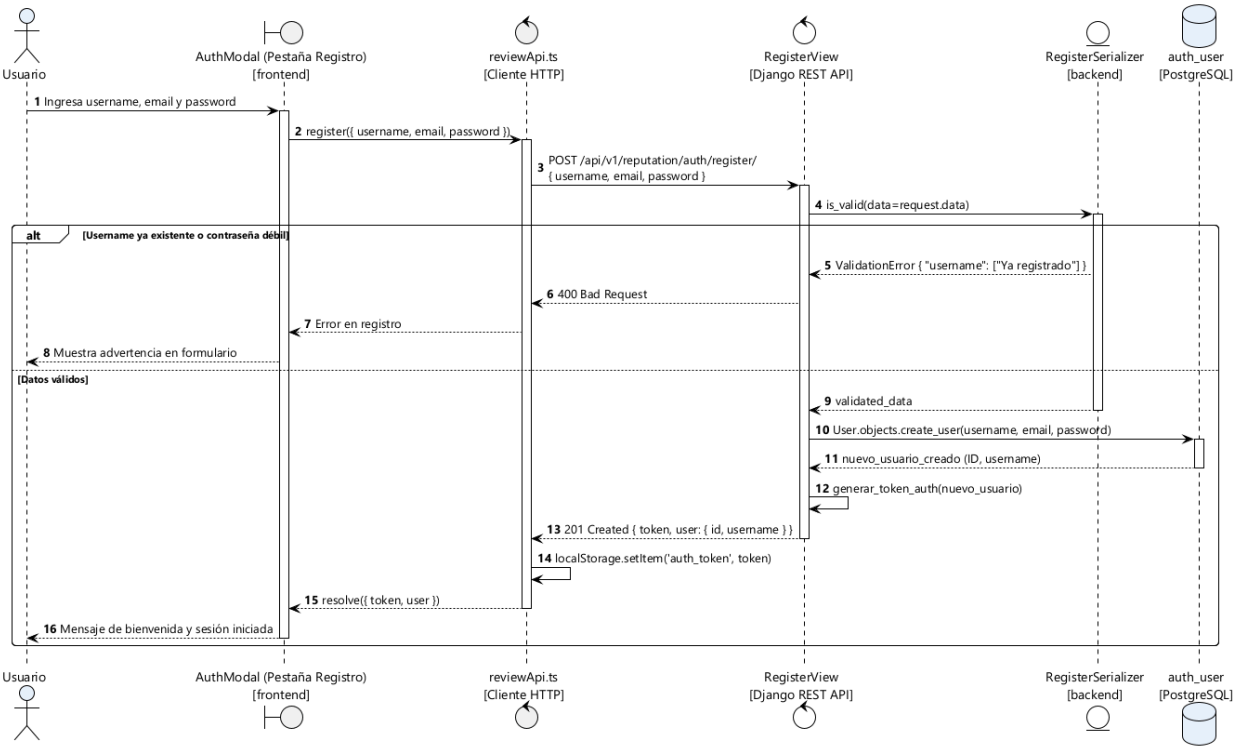


Figura 11. Análisis de casos de uso de Registrar usuario

2.4.CU04 Explorar catálogo de aplicaciones por categoría

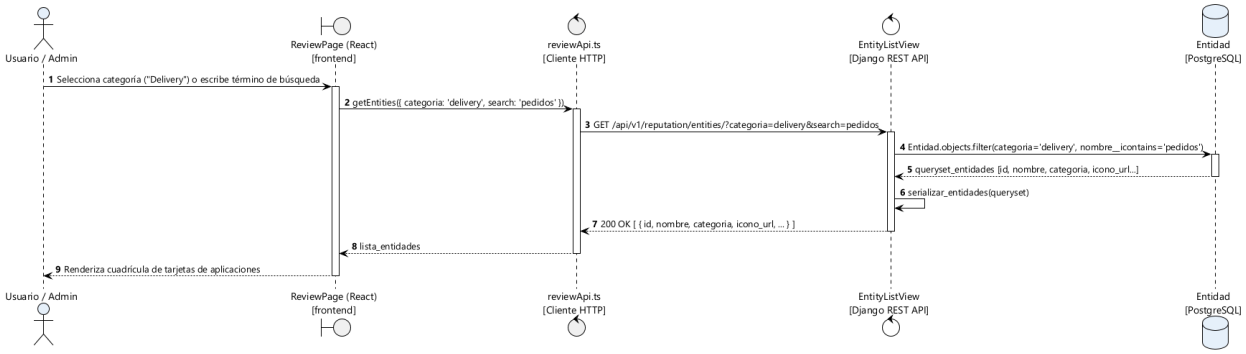


Figura 12. Análisis de casos de uso de Explorar catálogo de aplicaciones por categoría

2.5.CU05 Ver diagnóstico y calificación de una aplicación

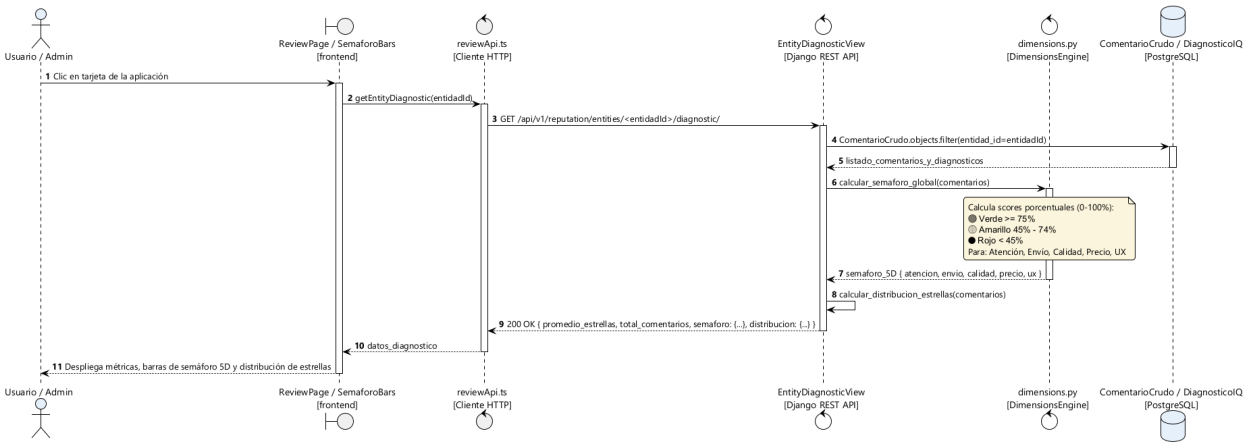


Figura 13. Análisis de casos de uso de Ver diagnóstico y calificación de una aplicación

2.6.CU06 Consultar opiniones y reseñas de una aplicación

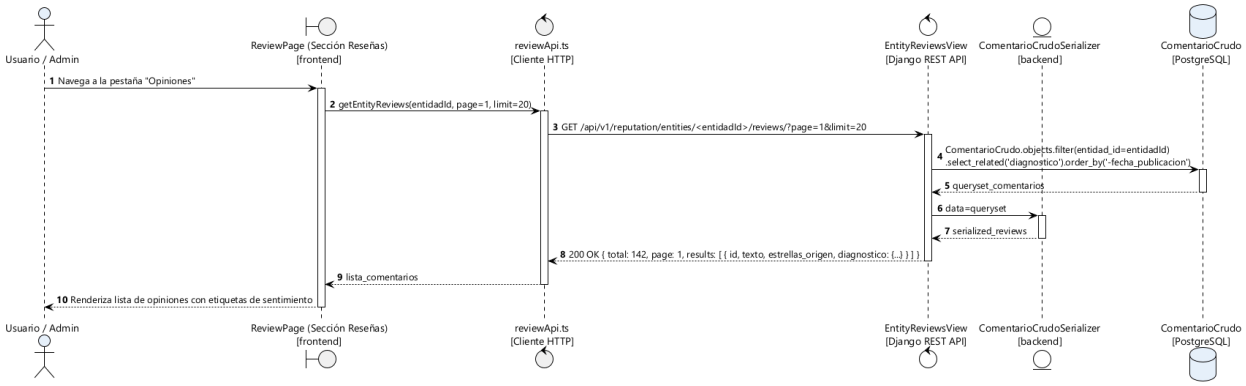


Figura 14. Análisis de casos de uso de Consultar opiniones y reseñas de una aplicación

2.7.CU07 Escribir reseña y calificar una aplicación

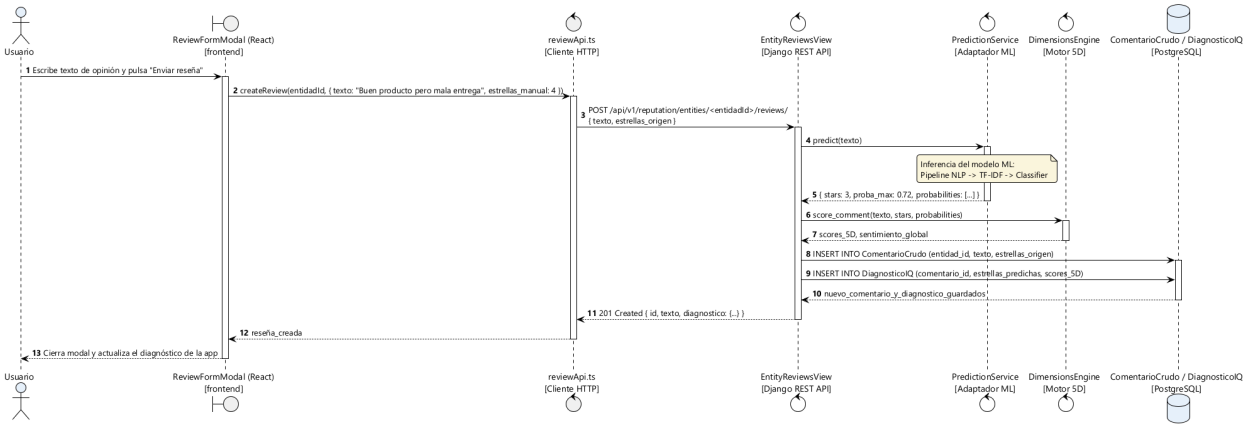


Figura 15. Análisis de casos de uso de Escribir reseña y calificar una aplicación

2.8.CU08 Comparar dos aplicaciones en modo Versus

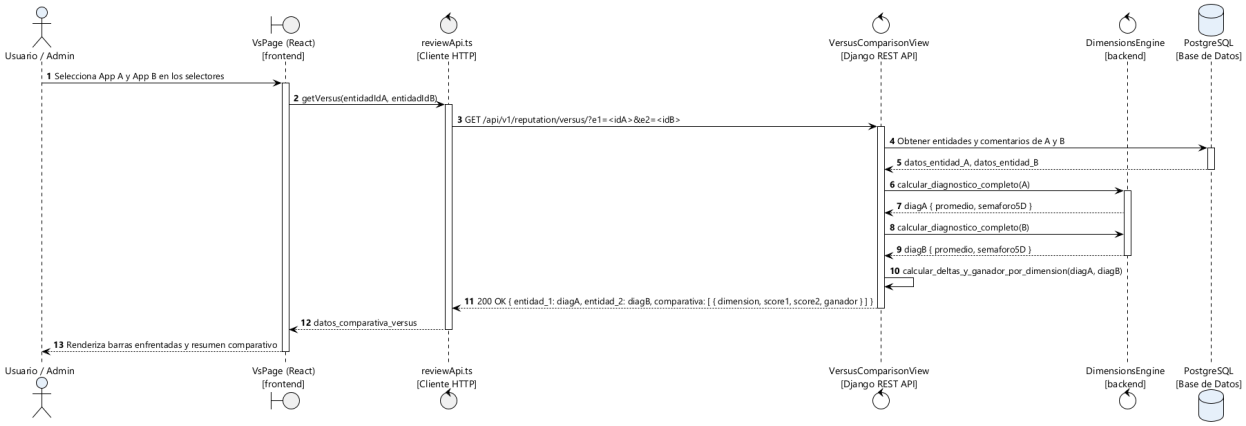


Figura 16. Análisis de casos de uso de Comparar dos aplicaciones en modo Versus

2.9.CU09 Buscar aplicaciones en Google Play en tiempo real

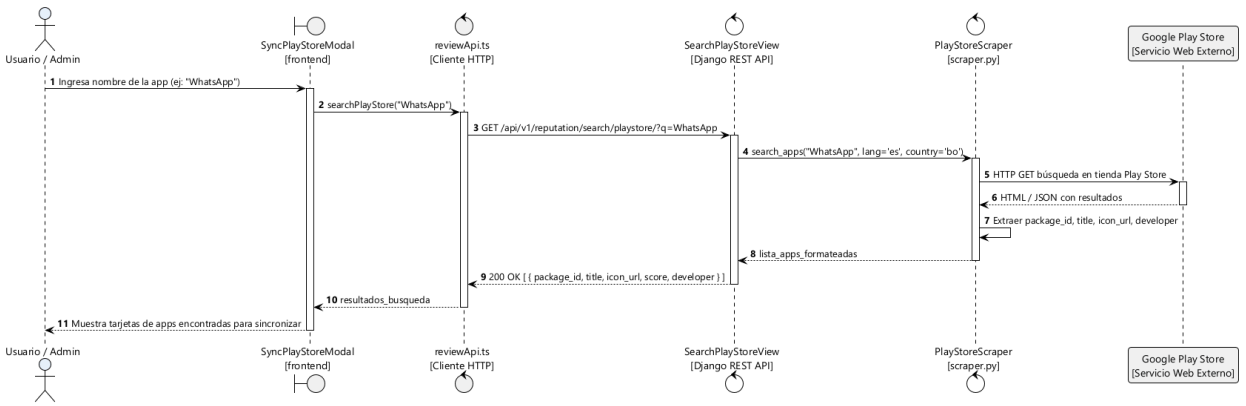


Figura 17. Análisis de casos de uso de Buscar aplicaciones en Google Play en tiempo real

2.10. CU10 Sincronizar opiniones desde Google Play

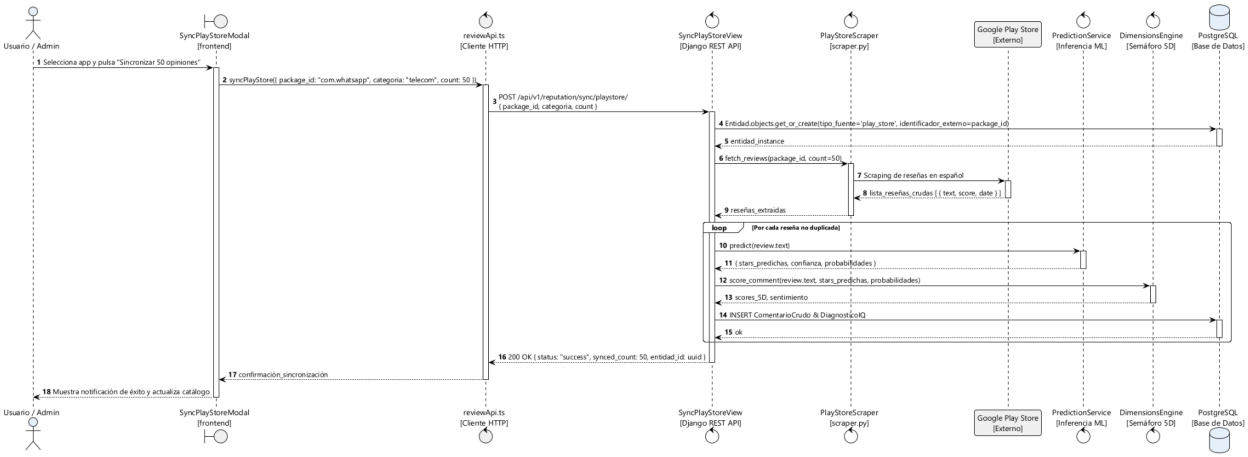


Figura 18. Análisis de casos de uso de Sincronizar opiniones desde Google Play

2.11. CU11 Estimar calificación de estrellas para una reseña libre

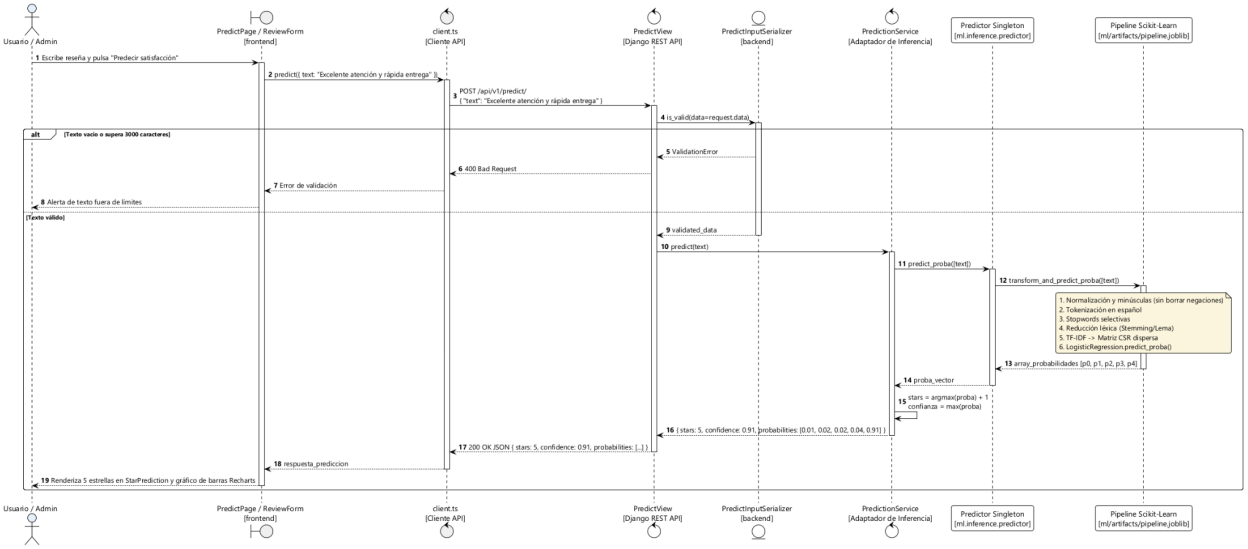


Figura 19. Análisis de casos de uso de Estimar calificación de estrellas para una reseña libre

2.12. CU12 Inspeccionar pipeline lingüístico en el Laboratorio NLP

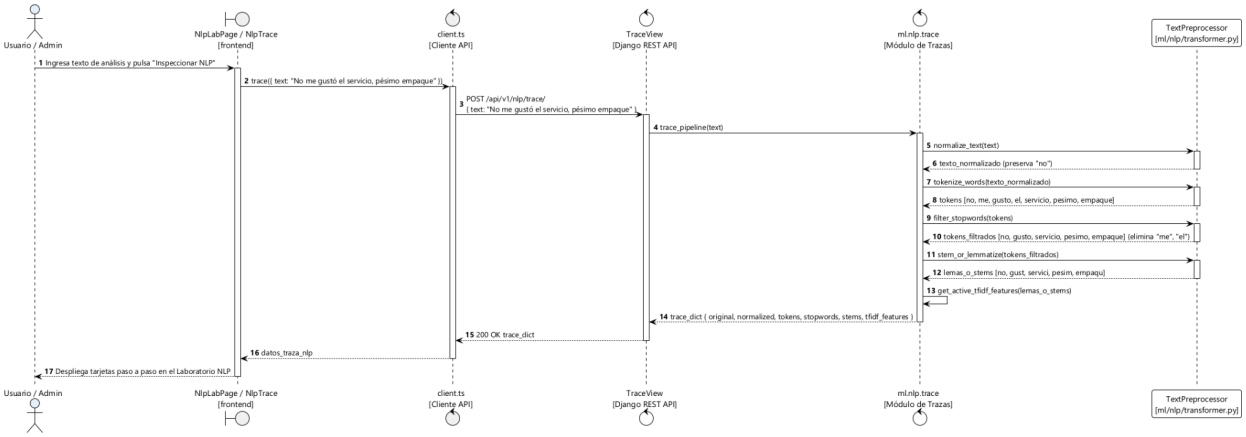


Figura 20. Análisis de casos de uso de Inspeccionar pipeline lingüístico en el Laboratorio NLP

2.13. CU13 Consultar métricas del modelo y matriz de confusión

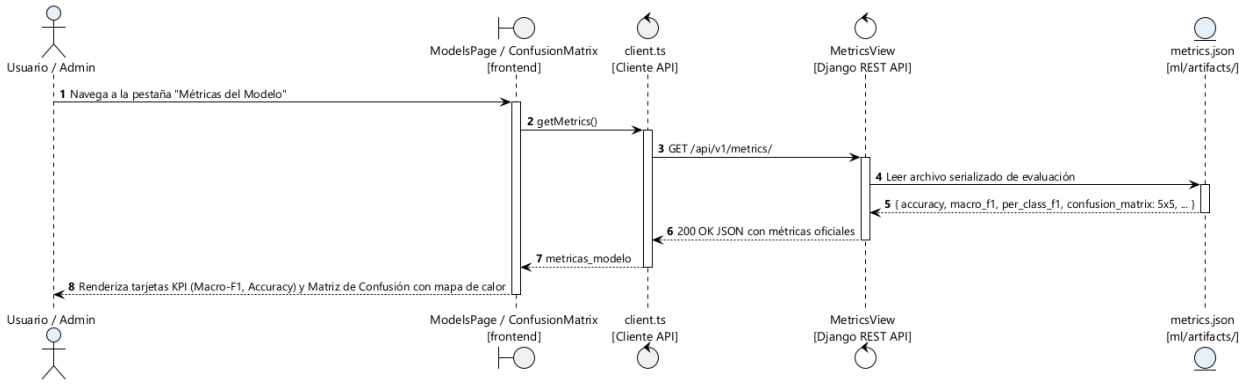


Figura 21. Análisis de casos de uso de Consultar métricas del modelo y matriz de confusión

2.14. CU14 Comparar modelo de Regresión Logística con Naive Bayes

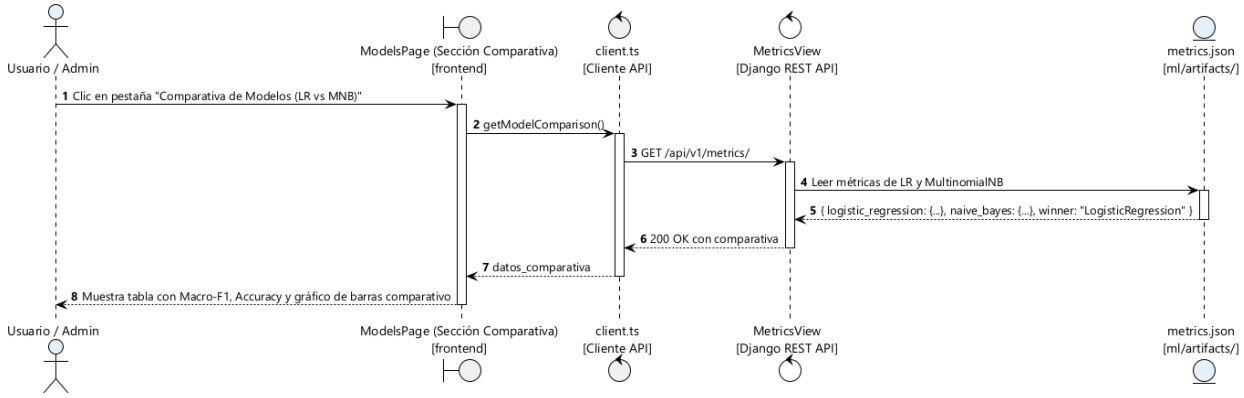


Figura 22. Análisis de casos de uso de Comparar modelo de Regresión Logística con Naive Bayes

2.15. CU15 Consultar metodología del corpus y particiones de datos

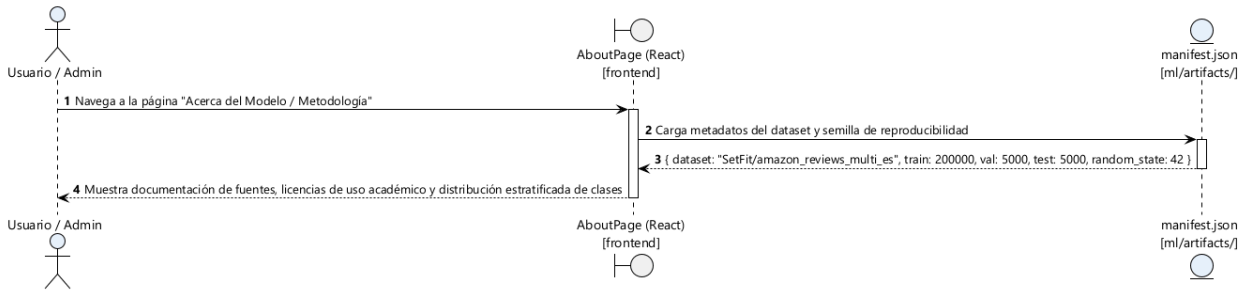


Figura 23. Análisis de casos de uso de Consultar metodología del corpus y particiones de datos

2.16. CU16 Consultar documentación interactiva de la API (Swagger)

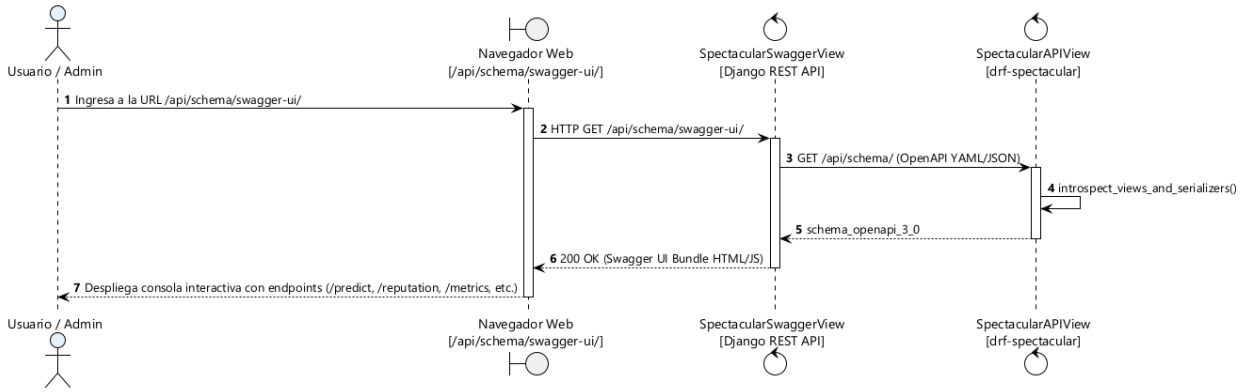


Figura 24. Análisis de casos de uso de Consultar documentación interactiva de la API (Swagger)

2.17. CU17 Procesar lote de reseñas de forma masiva

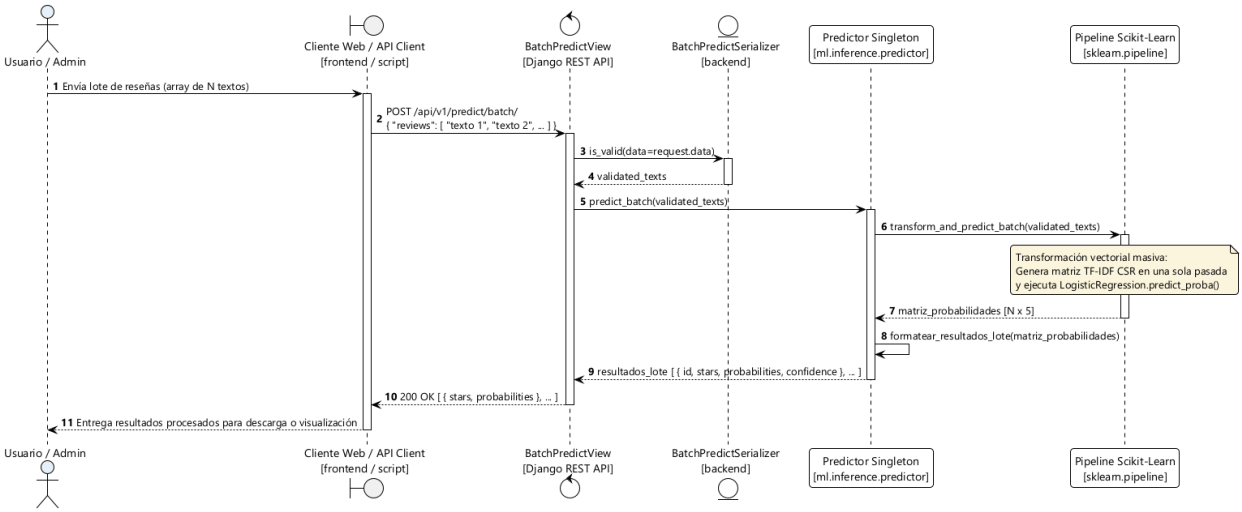


Figura 25. Análisis de casos de uso de Procesar lote de reseñas de forma masiva

2.18. CU18 Implementar pipeline de preprocesamiento de texto

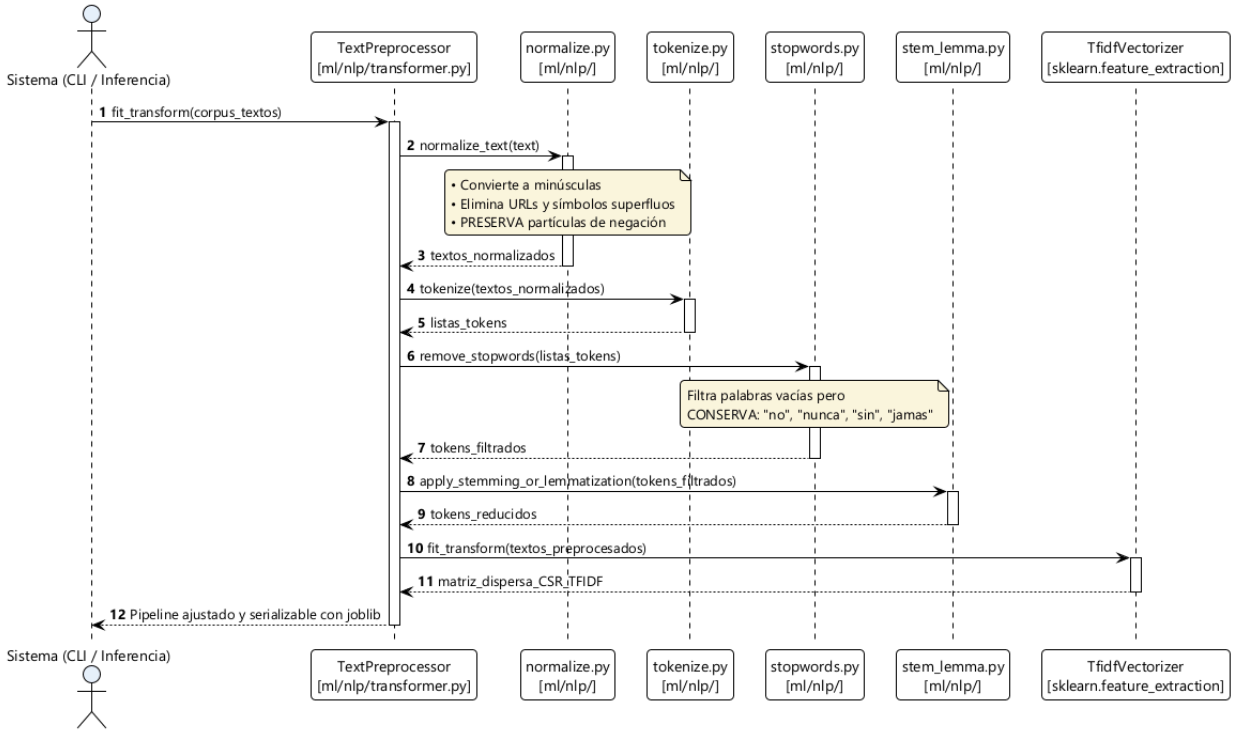


Figura 26. Análisis de casos de uso de Implementar pipeline de preprocesamiento de texto

2.19. CU19 Implementar flujo de entrenamiento del modelo de clasificación

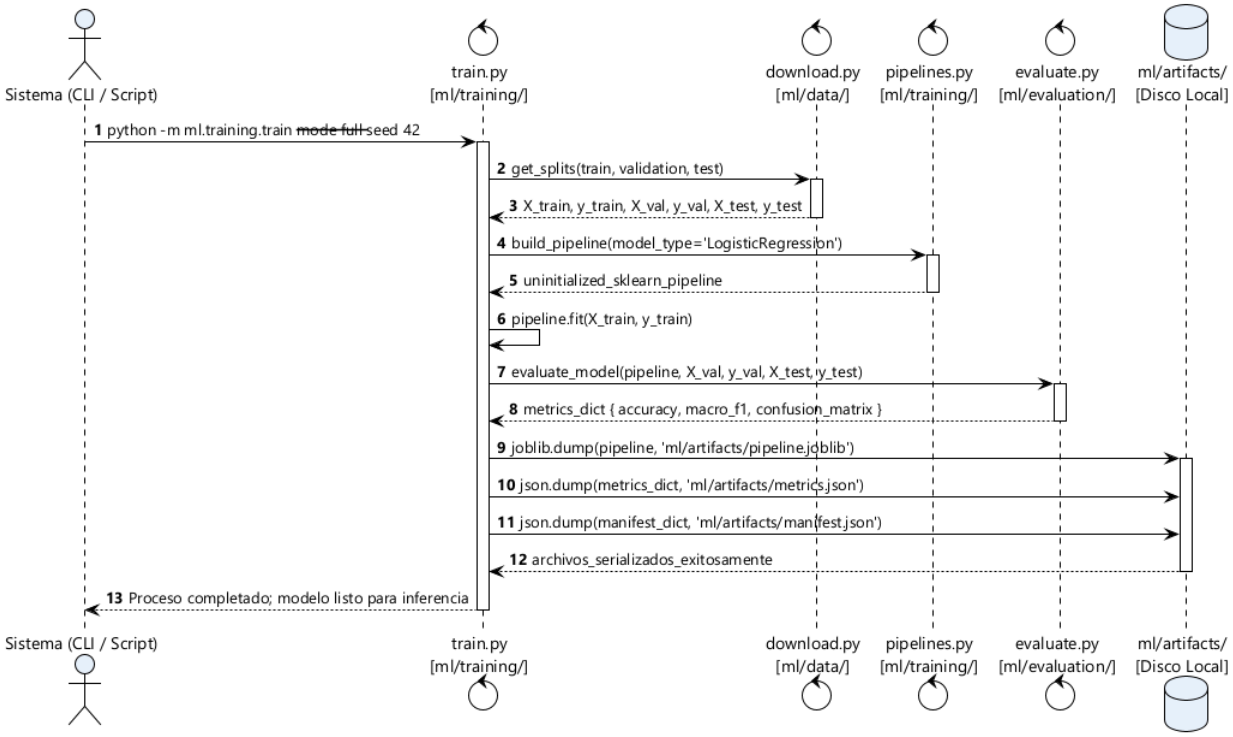


Figura 27. Análisis de casos de uso de Implementar flujo de entrenamiento del modelo de clasificación

2.20. CU20 Monitorear disponibilidad y estado del servicio

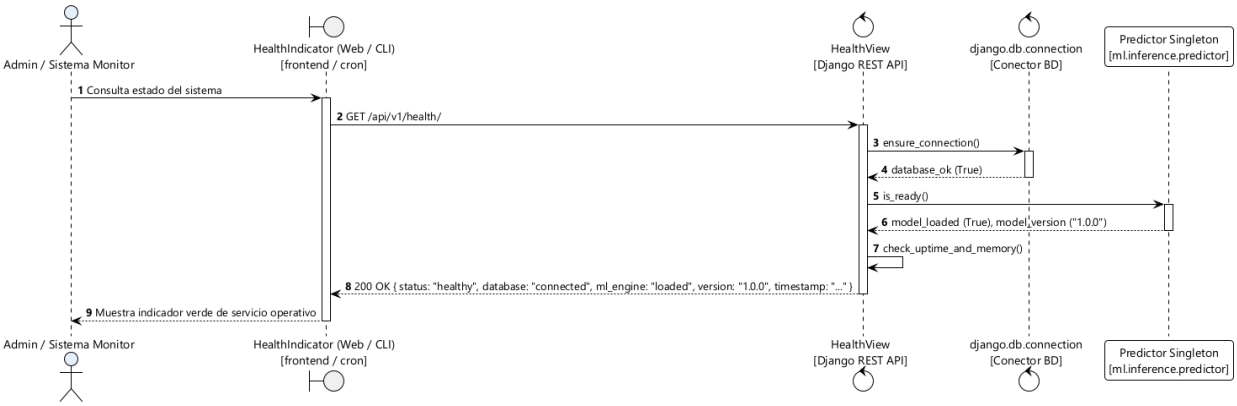


Figura 28. Análisis de casos de uso de Monitorear disponibilidad y estado del servicio

3. Análisis de paquete

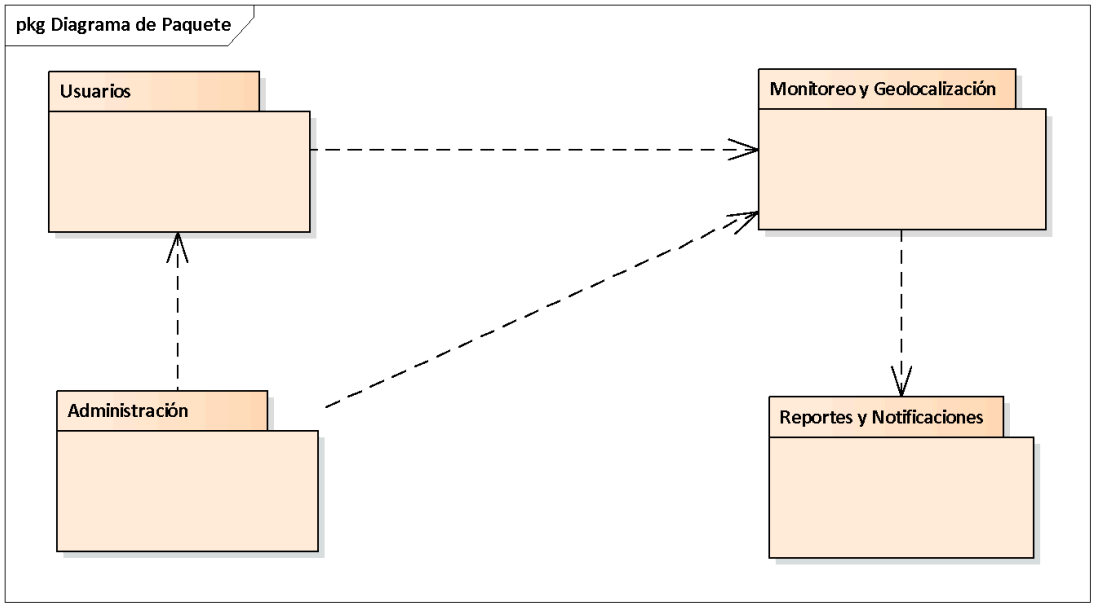


Figura 29. Análisis de paquete

2.2.3 FLUJO DE TRABAJO: DISEÑO

2.2.3.1 Diseño Físico (Diagrama de Despliegue)

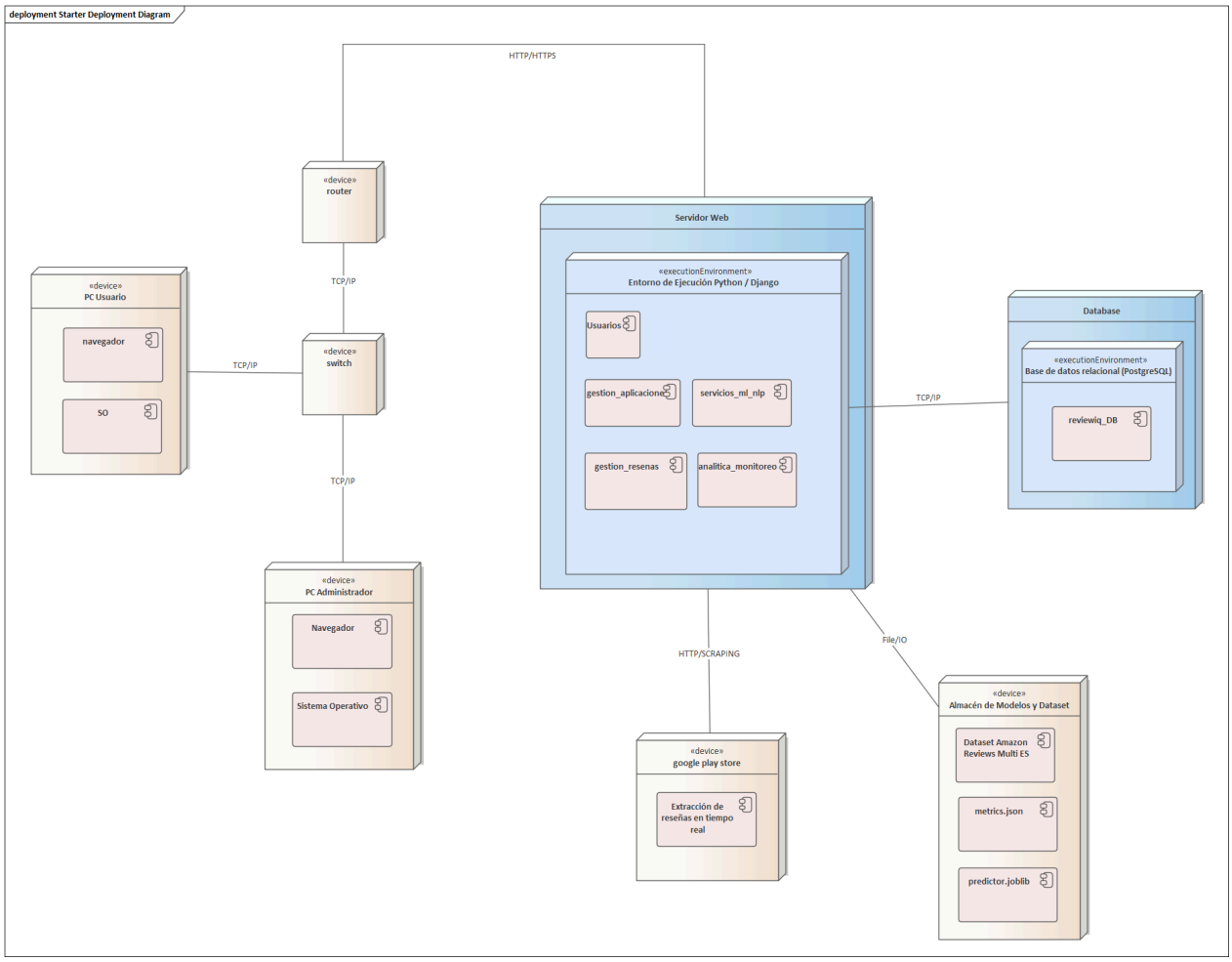
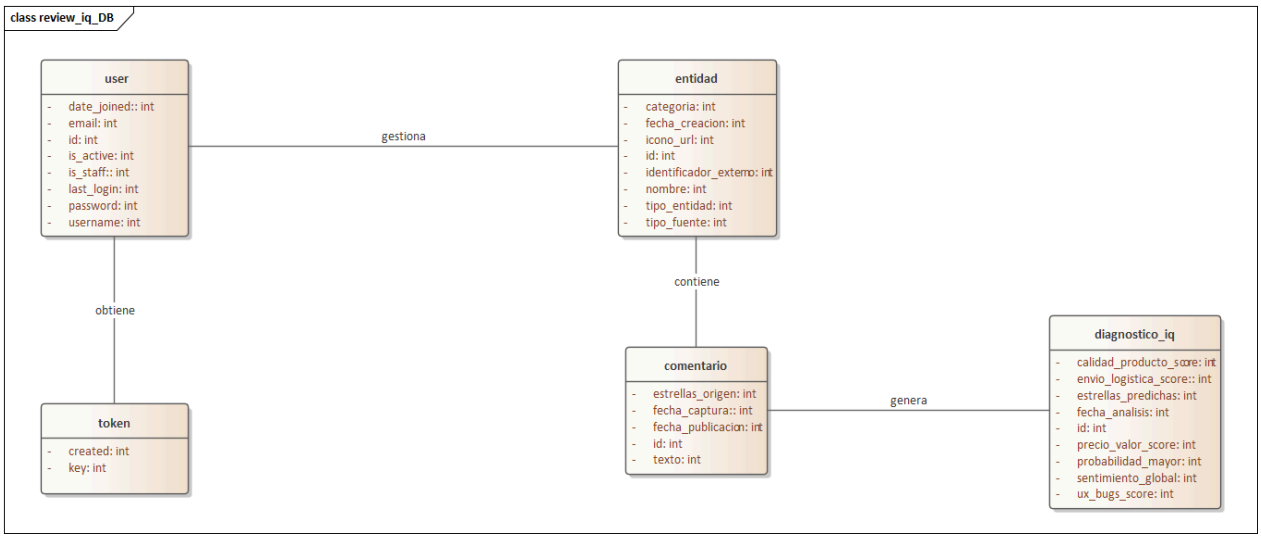


Diagrama de Clase

diseño conceptual



Diseño de Datos

Diseño Lógico

1.1.1. Mapeo

■ Llave Primaria ■ Llave Foránea

Users

id	username	email	password	is_staff	is_active	date_joined	last_login

Token

id	key	User_id

Entidad

id	nombre	tipo_entidad	categoria	tipo_fuente	identificador_externo	icono_url	fecha_creacion

Comentario

id	entidad_id	texto	estrellas_origen	fecha_publicacion	fecha_captura

diseño fisico

```
-- PROYECTO INTEGRADOR: ReviewIQ
-- SCRIPT DDL DE BASE DE DATOS RELACIONAL (POSTGRESQL 15+)
-- ================================================================================


-- 1. Habilitar extensión para identificadores únicos universales (UUID v4)
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE TABLE auth_user (
    id BIGSERIAL PRIMARY KEY,
    username VARCHAR(150) NOT NULL UNIQUE,
    email VARCHAR(254) NOT NULL,
    password VARCHAR(128) NOT NULL,
    is_staff BOOLEAN NOT NULL DEFAULT FALSE,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    date_joined TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    last_login TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_auth_user_username ON auth_user (username);
```

```
--
================================================================
=========
-- 2) TABLA: authtoken_token (Tokens de autenticación para la API REST)
--
================================================================
=========
CREATE TABLE authtoken_token (
    key VARCHAR(40) PRIMARY KEY,
    user_id BIGINT NOT NULL UNIQUE REFERENCES auth_user(id) ON DELETE
CASCADE,
    created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_authtoken_user ON authtoken_token (user_id);

--
================================================================
=========
-- 3) TABLA: reviewiq_entidad (Catálogo de aplicaciones móviles auditadas)
--
================================================================
=========
CREATE TABLE reviewiq_entidad (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    nombre VARCHAR(150) NOT NULL,
    tipo_entidad VARCHAR(20) NOT NULL DEFAULT 'app',
    categoria VARCHAR(50) NOT NULL,
    tipo_fuente VARCHAR(30) NOT NULL DEFAULT 'play_store',
    identificador_externo VARCHAR(255) NOT NULL,
```

```
    icono_url VARCHAR(200),
    fecha_creacion TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    -- Restricciones de unicidad y valores permitidos
    CONSTRAINT unique_entidad_por_fuente UNIQUE (tipo_fuente, identificador_externo),
    CONSTRAINT chk_tipo_entidad CHECK (tipo_entidad IN ('app', 'fisico')),
    CONSTRAINT chk_tipo_fuente CHECK (tipo_fuente IN ('play_store', 'google_maps', 'manual'))
);

CREATE INDEX idx_entidad_nombre ON reviewiq_entidad (nombre);
CREATE INDEX idx_entidad_categoria ON reviewiq_entidad (categoria);
CREATE INDEX idx_entidad_externo ON reviewiq_entidad (identificador_externo);

-- ================================================================================
-- 4) TABLA: reviewiq_comentariocrudo (Opiniones recolectadas desde Google Play)
-- ================================================================================
CREATE TABLE reviewiq_comentariocrudo (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    entidad_id UUID NOT NULL REFERENCES reviewiq_entidad(id) ON DELETE CASCADE,
    texto TEXT NOT NULL,
    estrellas_origen INTEGER NOT NULL,
    fecha_publicacion TIMESTAMP WITH TIME ZONE,
```

```
    fecha_captura TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    -- Validación de calificación original (1 a 5 estrellas)
    CONSTRAINT chk_estrellas_origen CHECK (estrellas_origen BETWEEN 1 AND 5)
);

CREATE INDEX idx_comentario_entidad ON reviewiq_comentariocrudo (entidad_id);
CREATE INDEX idx_comentario_fecha_pub ON reviewiq_comentariocrudo (fecha_publicacion);
CREATE INDEX idx_comentario_estrellas ON reviewiq_comentariocrudo (estrellas_origen);

--
=========================================================================
=========
-- 5) TABLA: reviewiq_diagnosticoiq (Diagnóstico multidimensional e inferencia ML)
--
=========================================================================
=========
CREATE TABLE reviewiq_diagnosticoiq (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    comentario_id UUID NOT NULL UNIQUE REFERENCES
reviewiq_comentariocrudo(id) ON DELETE CASCADE,

    -- Predicción del modelo de Machine Learning
    estrellas_predichas INTEGER NOT NULL,
    probabilidad_mayor DOUBLE PRECISION NOT NULL,
    sentimiento_global VARCHAR(20) NOT NULL,
```

```
    -- Puntuación de las 5 dimensiones (-1.0=Negativo, 0.0=Neutro/No aplica,
1.0=Positivo)
    atencion_cliente_score DOUBLE PRECISION NOT NULL DEFAULT 0.0,
    envio_logistica_score DOUBLE PRECISION NOT NULL DEFAULT 0.0,
    calidad_producto_score DOUBLE PRECISION NOT NULL DEFAULT 0.0,
    precio_valor_score DOUBLE PRECISION NOT NULL DEFAULT 0.0,
    ux_bugs_score DOUBLE PRECISION NOT NULL DEFAULT 0.0,

    fecha_analisis TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    -- Restricciones de validación
    CONSTRAINT chk_estrellas_predichas CHECK (estrellas_predichas BETWEEN 1
AND 5),
    CONSTRAINT chk_probabilidad CHECK (probabilidad_mayor BETWEEN 0.0 AND
1.0),
    CONSTRAINT chk_sentimiento CHECK (sentimiento_global IN ('positivo', 'neutro',
'negativo'))
);

CREATE INDEX idx_diagnostico_comentario ON reviewiq_diagnosticoiq
(comentario_id);
CREATE INDEX idx_diagnostico_estrellas ON reviewiq_diagnosticoiq
(estrellas_predichas);
CREATE INDEX idx_diagnostico_sentimiento ON reviewiq_diagnosticoiq
(sentimiento_global);
```

Diseño de Caso de Uso

1.2.CU01 Iniciar sesión

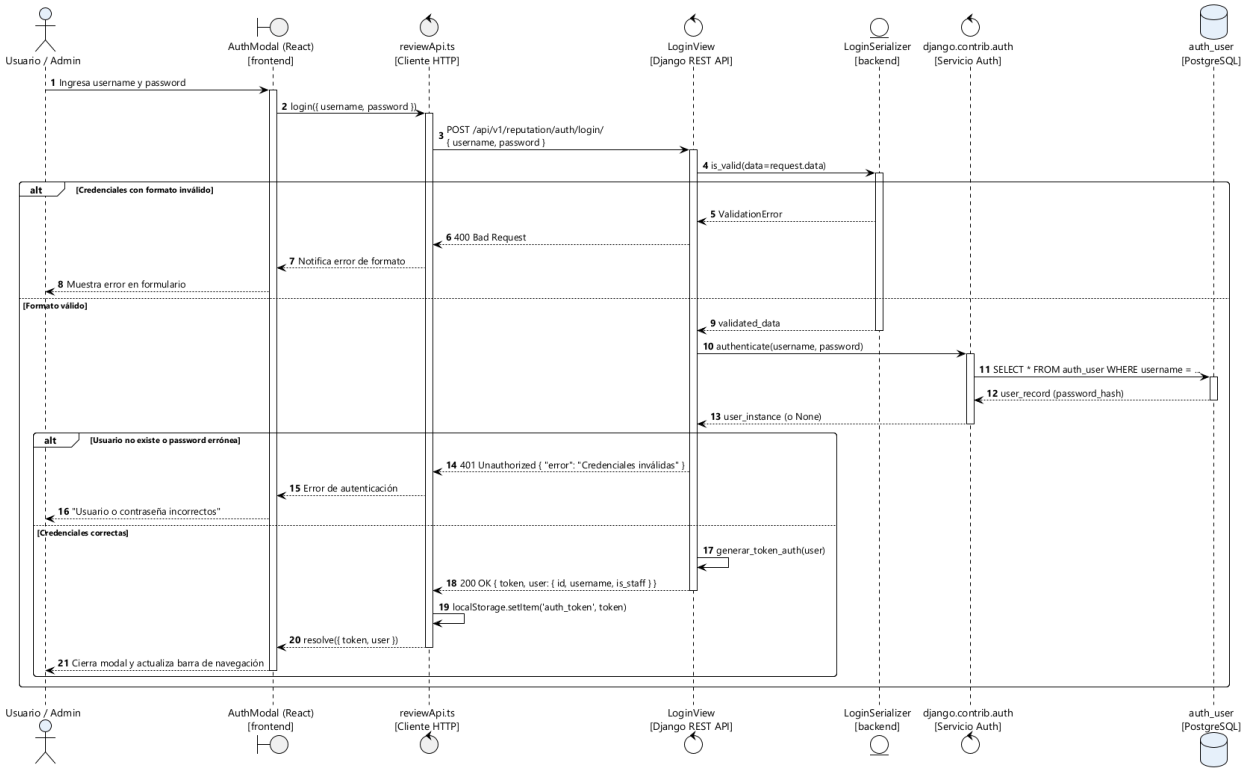


Figura 31. Análisis de casos de uso de Iniciar sesión

1.3.CU02 Cerrar sesión

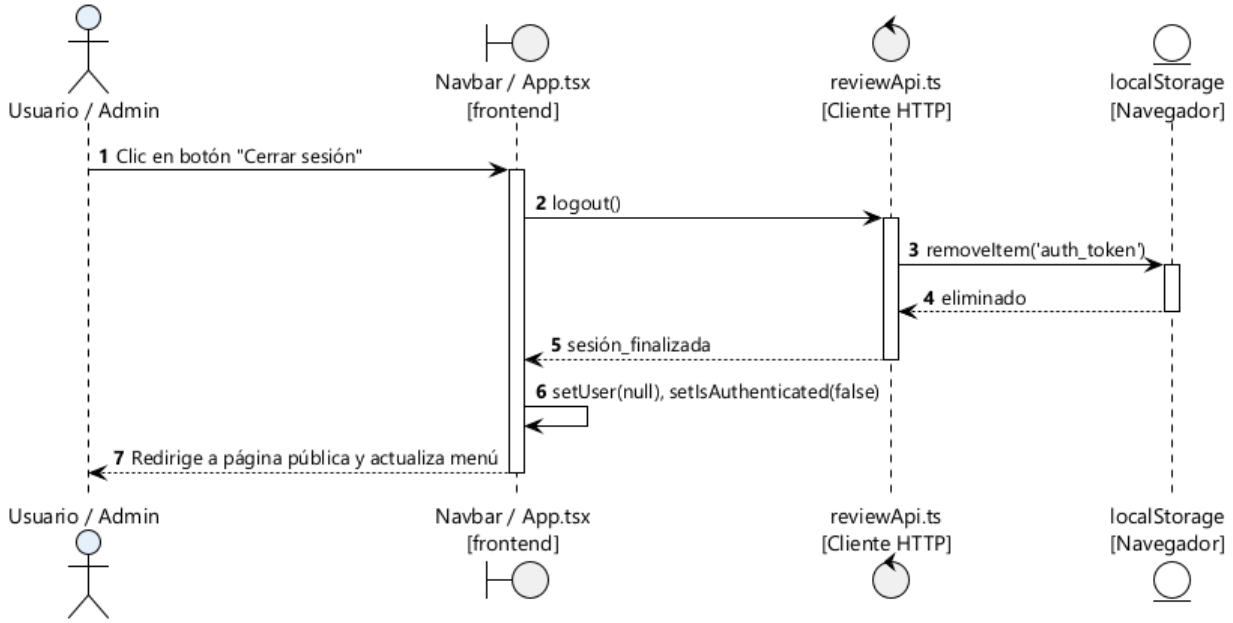


Figura 32. Análisis de casos de uso de Cerrar sesión

1.4.CU03 Registrar usuario

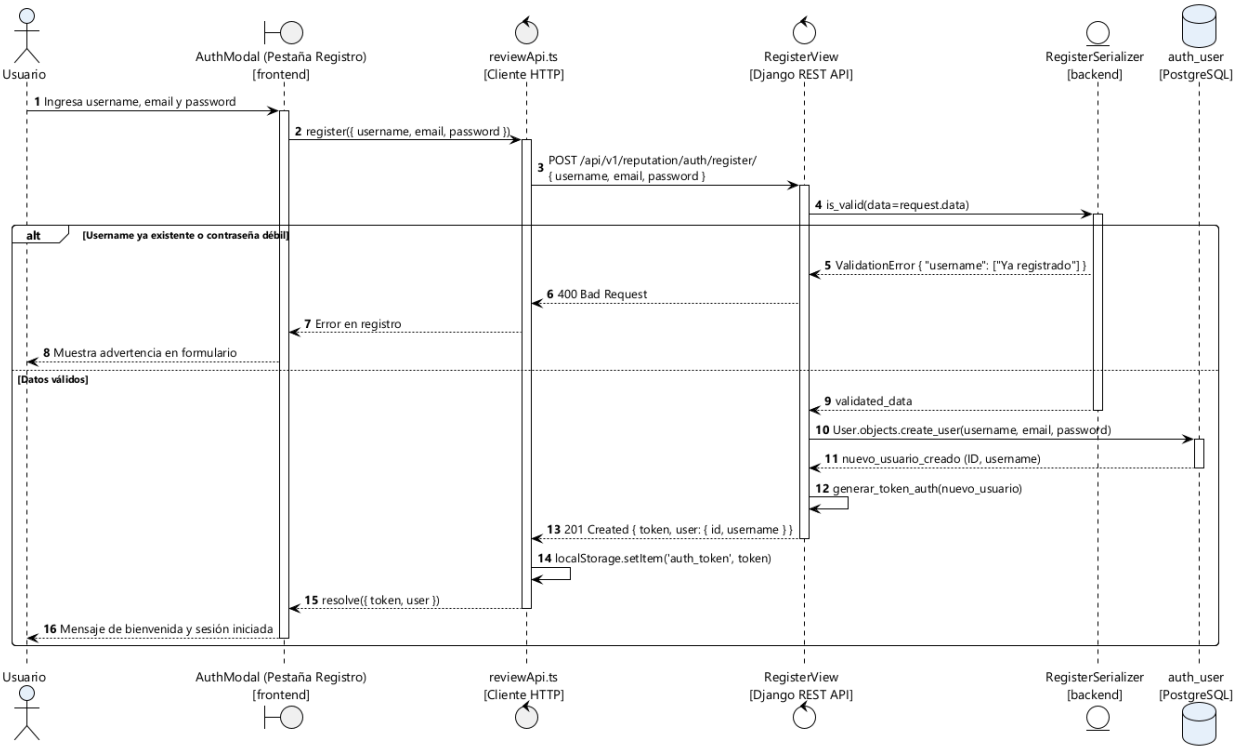


Figura 33. Análisis de casos de uso de Registrar usuario

1.5.CU04 Explorar catálogo de aplicaciones por categoría

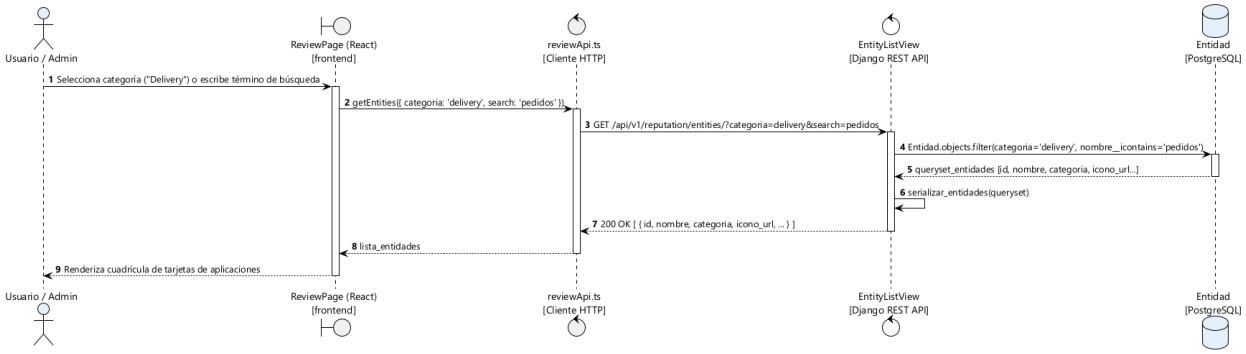


Figura 34. Análisis de casos de uso de Explorar catálogo de aplicaciones por categoría

1.6.CU05 Ver diagnóstico y calificación de una aplicación

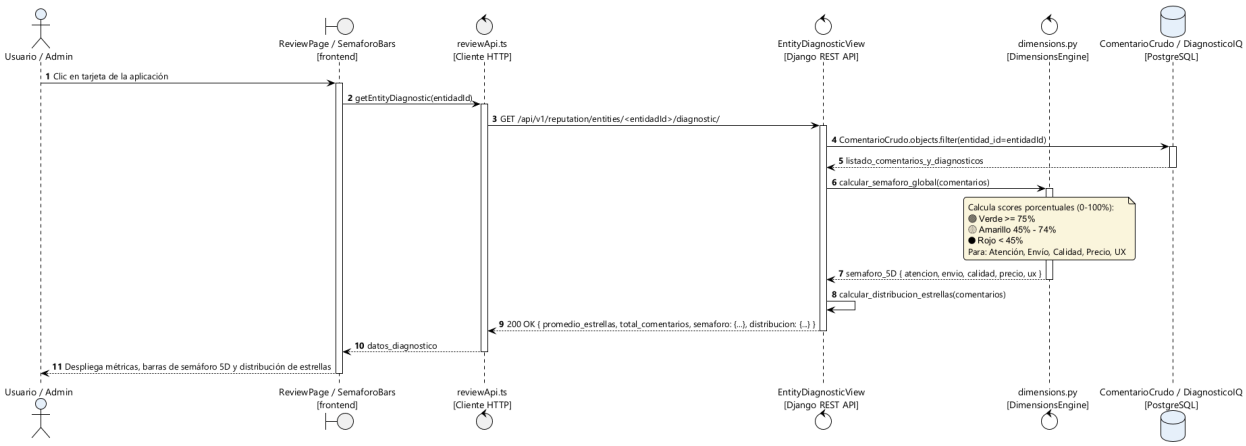


Figura 35. Análisis de casos de uso de Ver diagnóstico y calificación de una aplicación

1.7.CU06 Consultar opiniones y reseñas de una aplicación

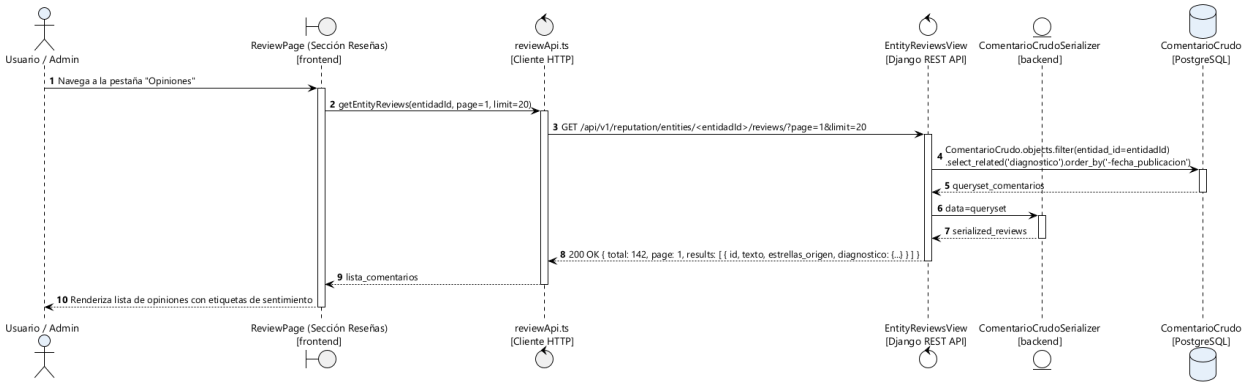


Figura 36. Análisis de casos de uso de Consultar opiniones y reseñas de una aplicación

1.8.CU07 Escribir reseña y calificar una aplicación

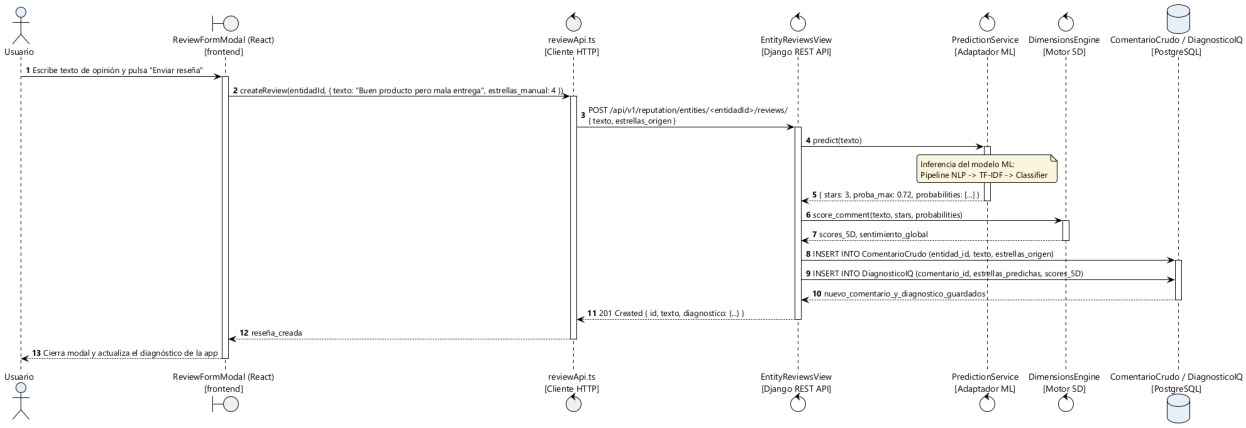


Figura 37. Análisis de casos de uso de Escribir reseña y calificar una aplicación

1.9.CU08 Comparar dos aplicaciones en modo Versus

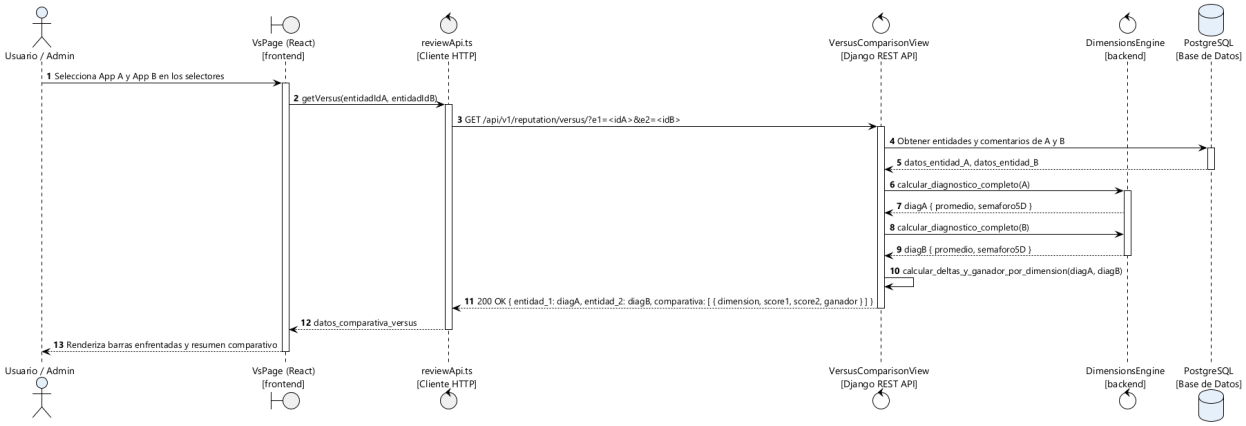


Figura 38. Análisis de casos de uso de Comparar dos aplicaciones en modo Versus

1.10. CU09 Buscar aplicaciones en Google Play en tiempo real

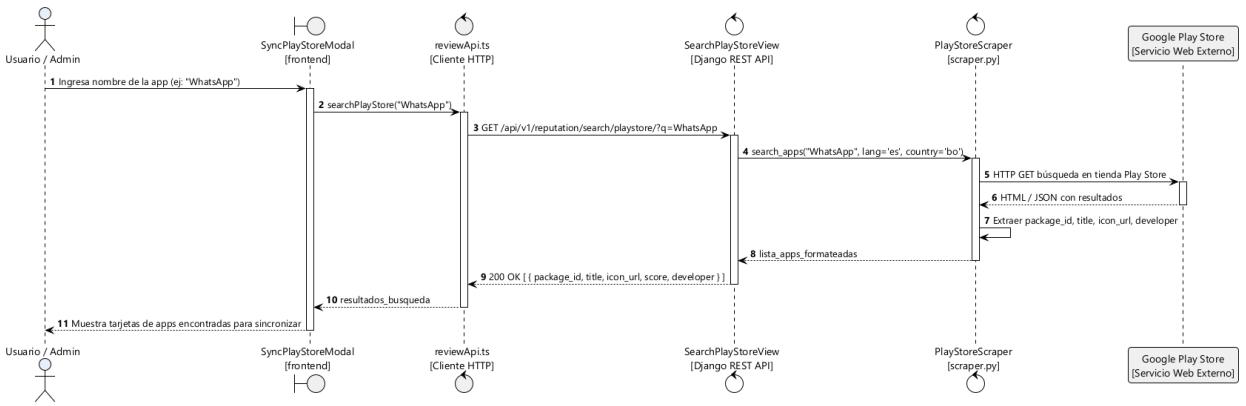


Figura 39. Análisis de casos de uso de Buscar aplicaciones en Google Play en tiempo real

1.11. CU10 Sincronizar opiniones desde Google Play

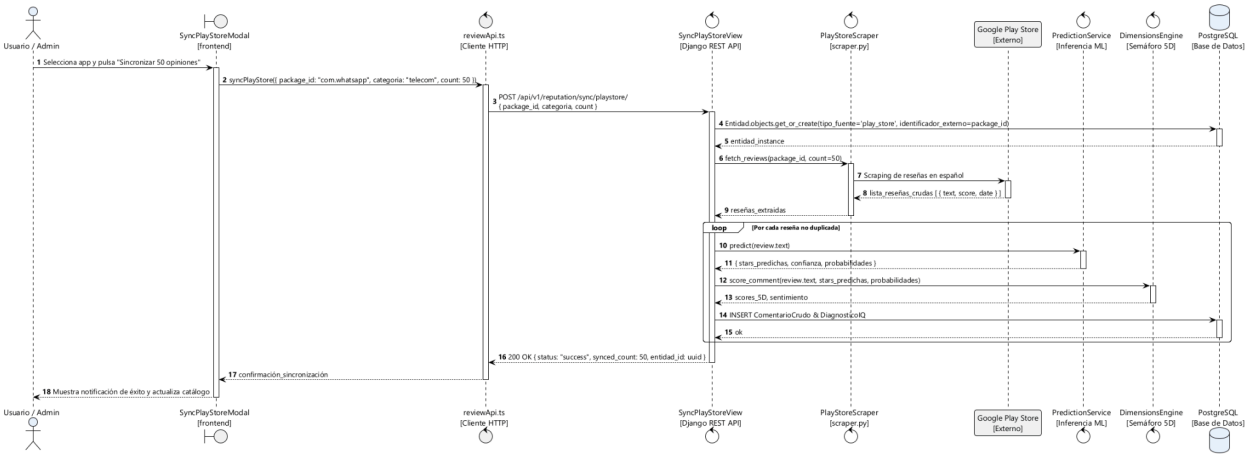


Figura 40. Análisis de casos de uso de Sincronizar opiniones desde Google Play

1.12. CU11 Estimar calificación de estrellas para una reseña libre

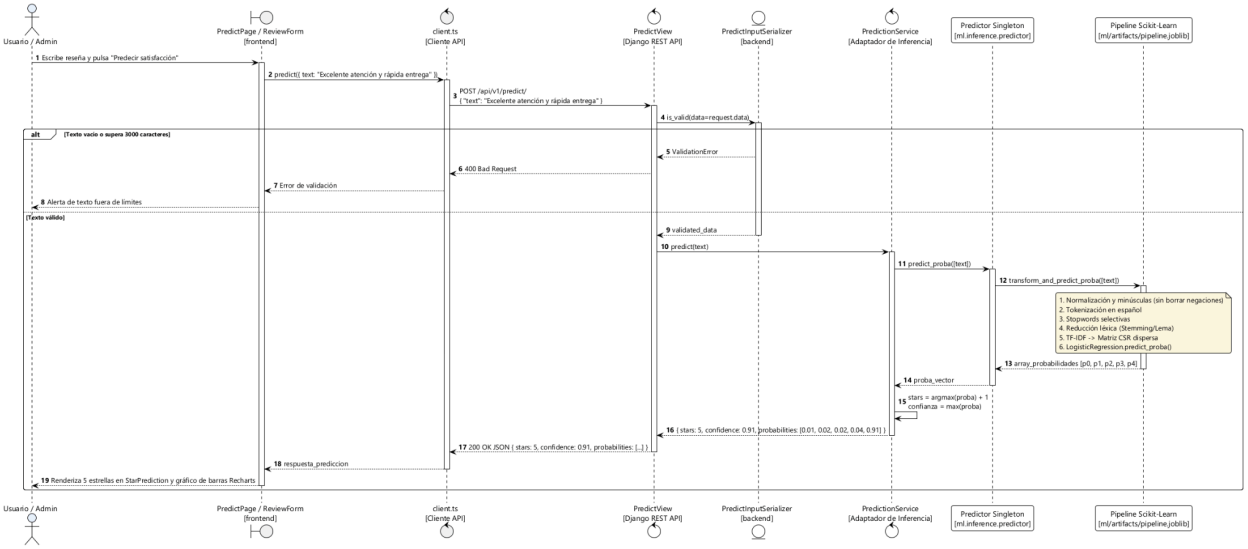


Figura 41. Análisis de casos de uso de Estimar calificación de estrellas para una reseña libre

1.13. CU12 Inspeccionar pipeline lingüístico en el Laboratorio NLP

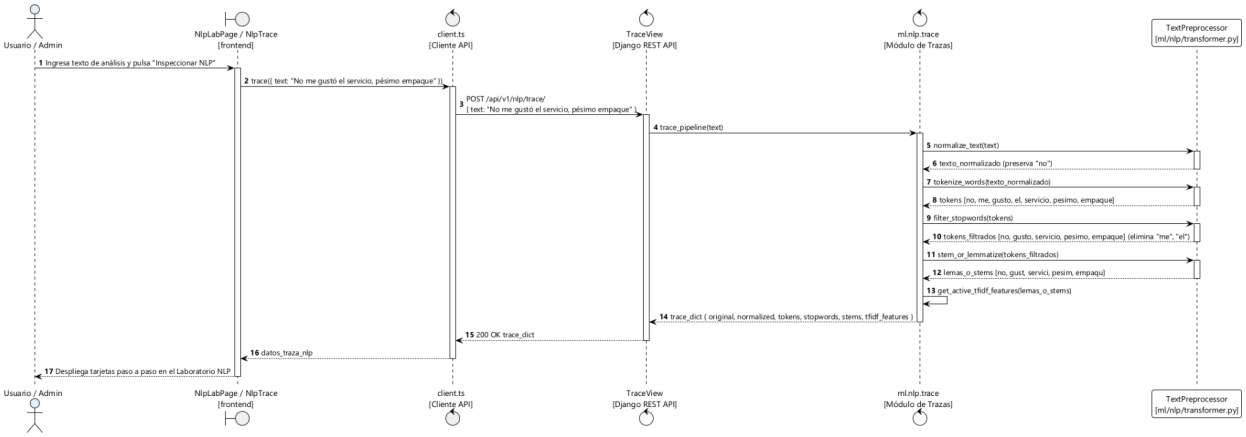


Figura 42. Análisis de casos de uso de Inspeccionar pipeline lingüístico en el Laboratorio NLP

1.14. CU13 Consultar métricas del modelo y matriz de confusión

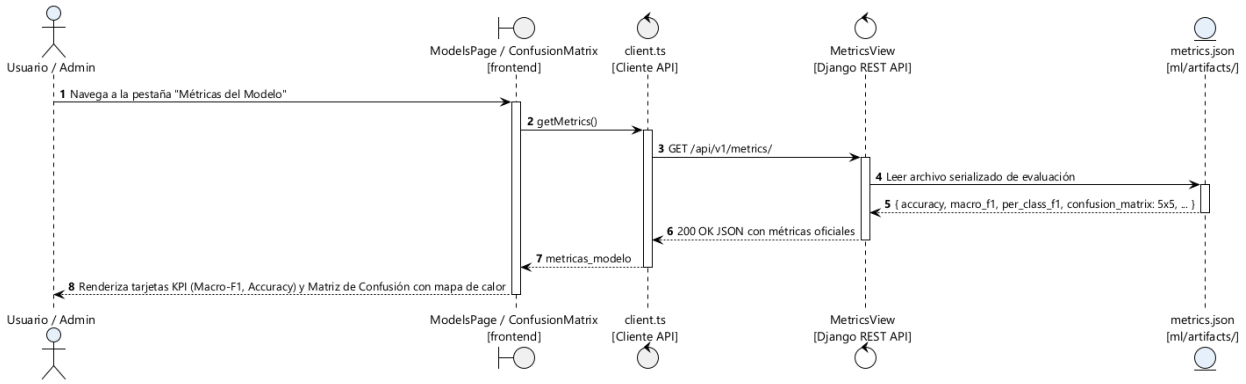


Figura 43. Análisis de casos de uso de Consultar métricas del modelo y matriz de confusión

1.15. CU14 Comparar modelo de Regresión Logística con Naive Bayes

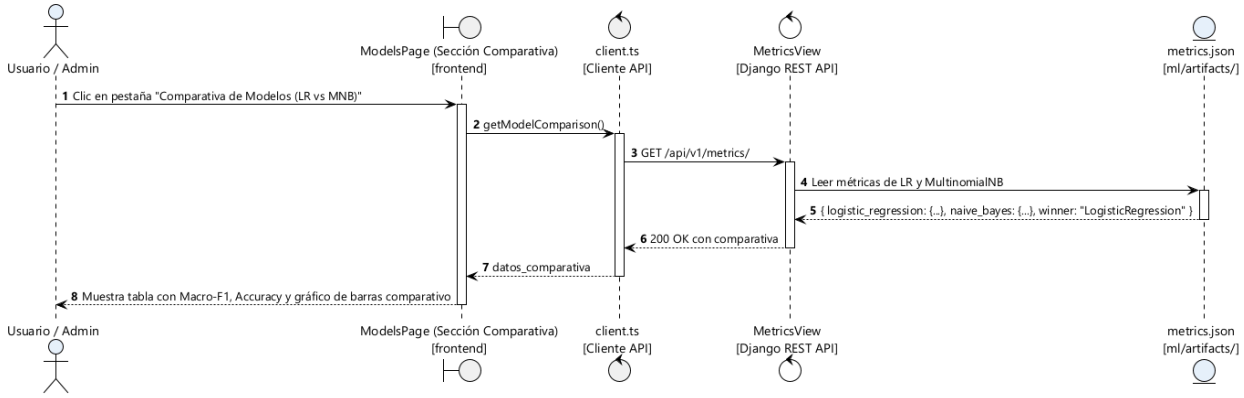


Figura 44. Análisis de casos de uso de Comparar modelo de Regresión Logística con Naive Bayes

1.16. CU15 Consultar metodología del corpus y particiones de datos

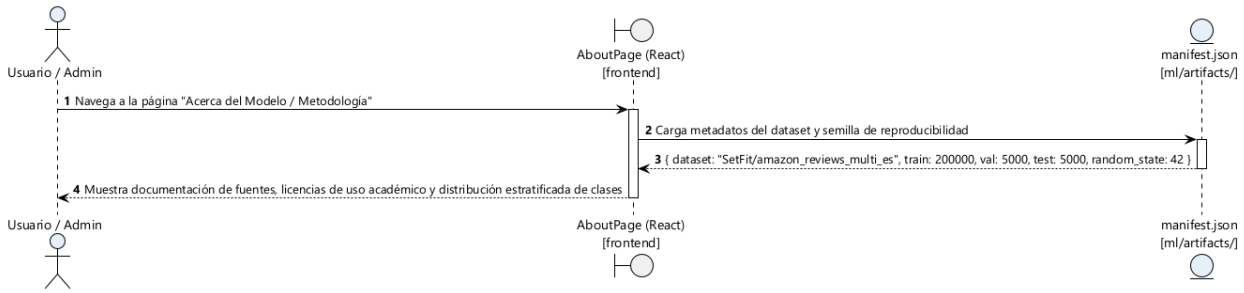


Figura 45. Análisis de casos de uso de Consultar metodología del corpus y particiones de datos

1.17. CU16 Consultar documentación interactiva de la API (Swagger)

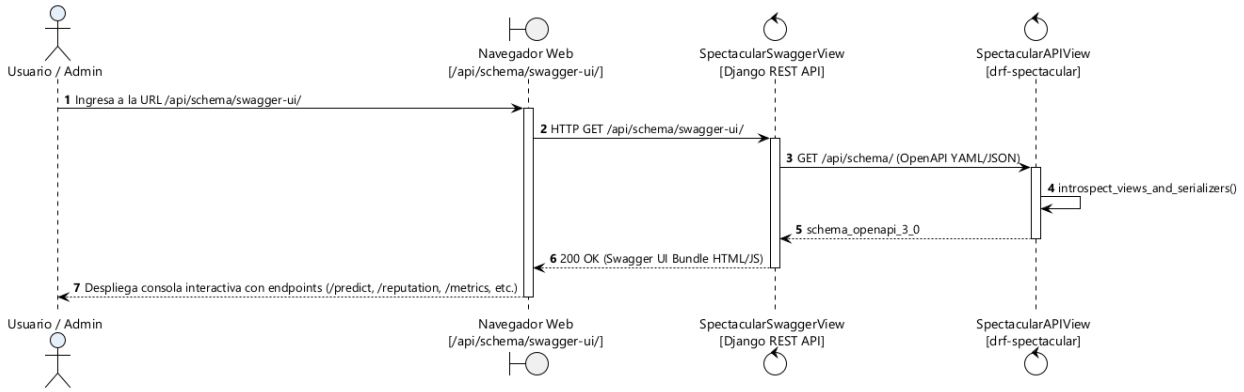


Figura 46. Análisis de casos de uso de Consultar documentación interactiva de la API (Swagger)

1.18. CU17 Procesar lote de reseñas de forma masiva

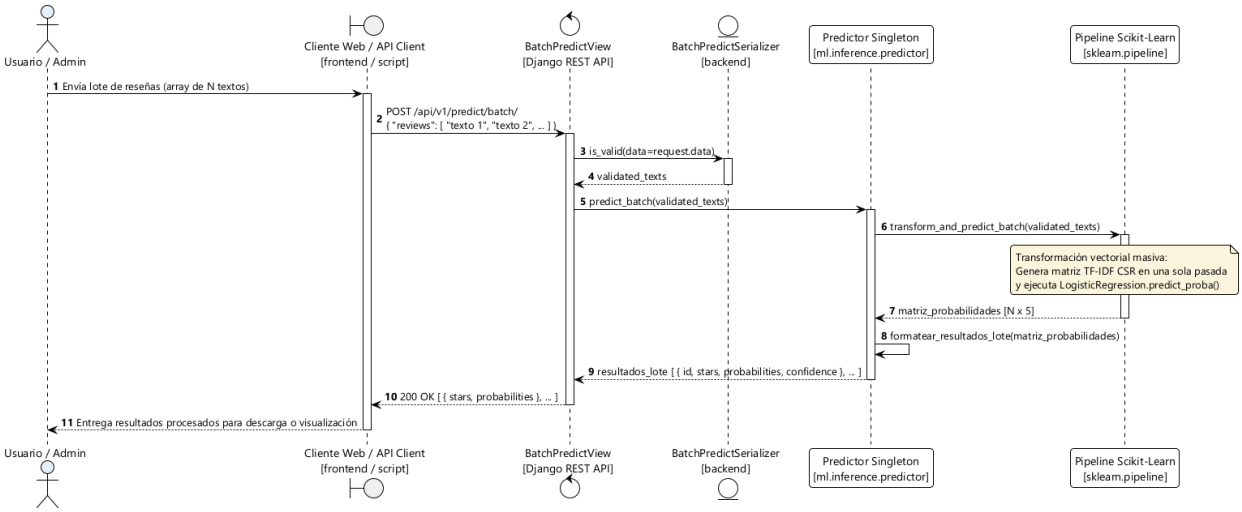


Figura 47. Análisis de casos de uso de Procesar lote de reseñas de forma masiva

1.19. CU18 Implementar pipeline de preprocesamiento de texto

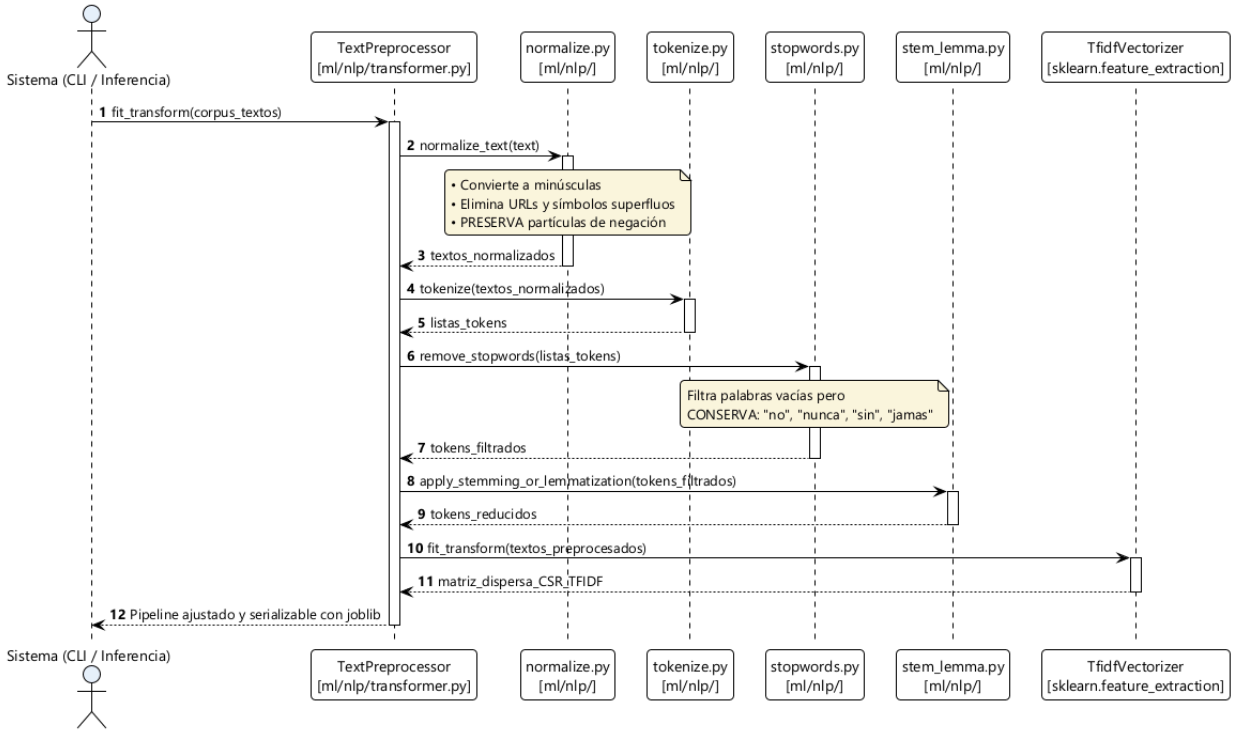


Figura 48. Análisis de casos de uso de Implementar pipeline de preprocesamiento de texto

1.20. CU19 Implementar flujo de entrenamiento del modelo de clasificación

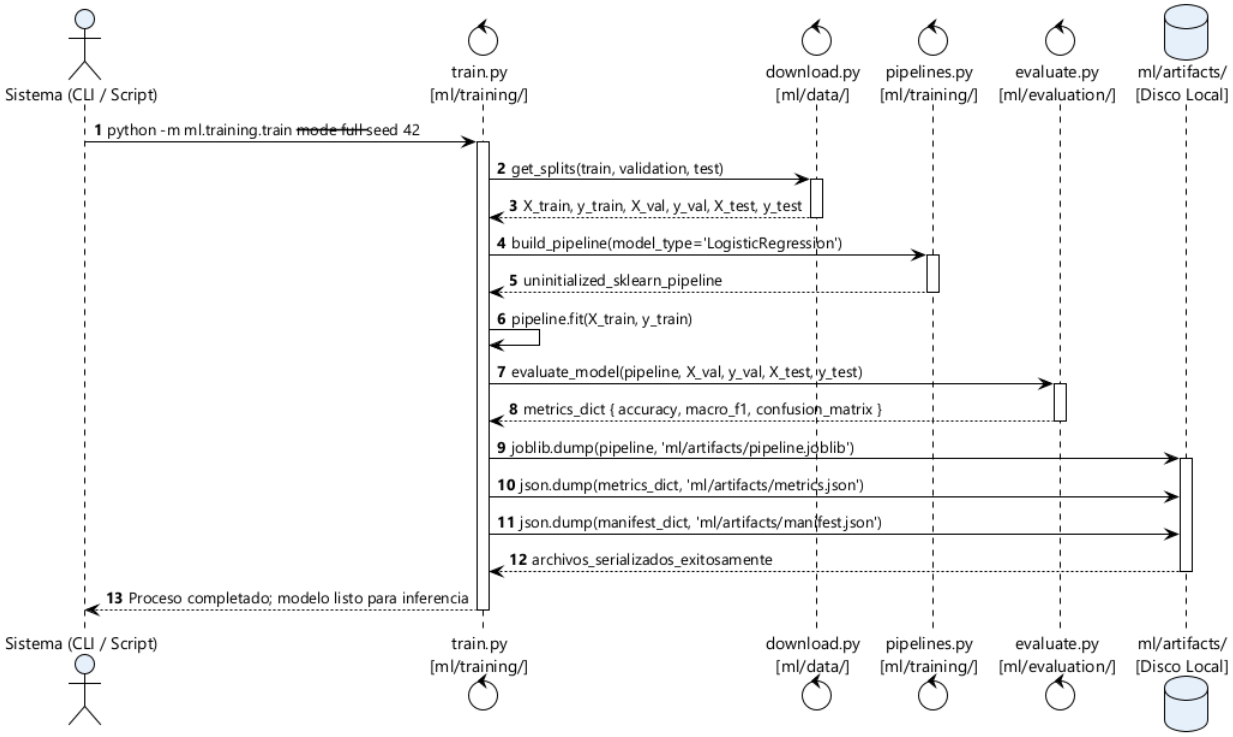


Figura 49. Análisis de casos de uso de Implementar flujo de entrenamiento del modelo de clasificación

1.21. CU20 Monitorear disponibilidad y estado del servicio

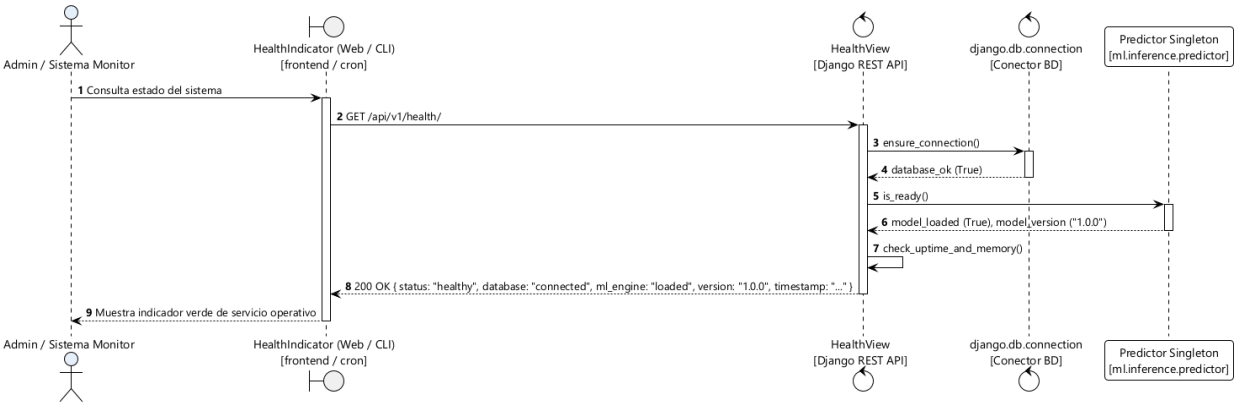
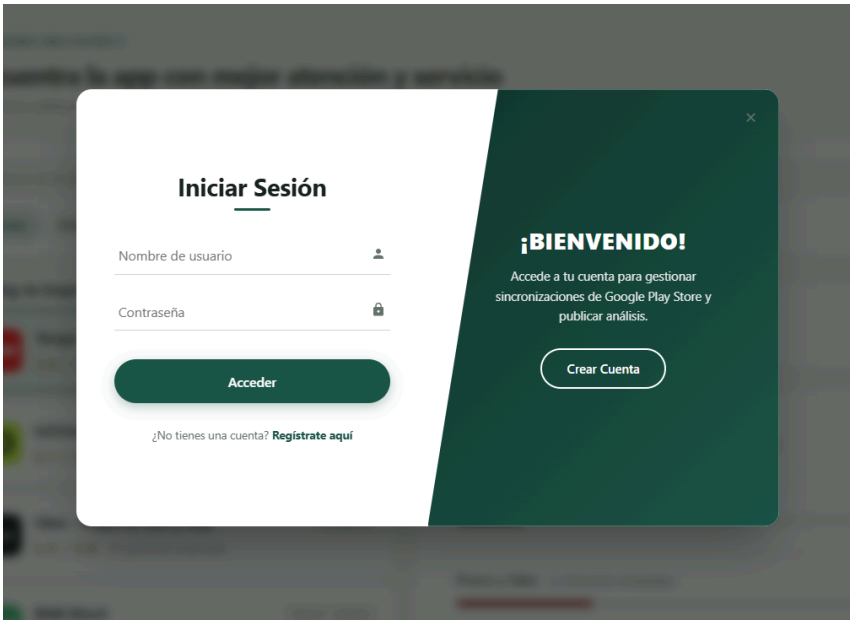


Figura 50. Análisis de casos de uso de Monitorear disponibilidad y estado del servicio

4. Diseño de Interfaces

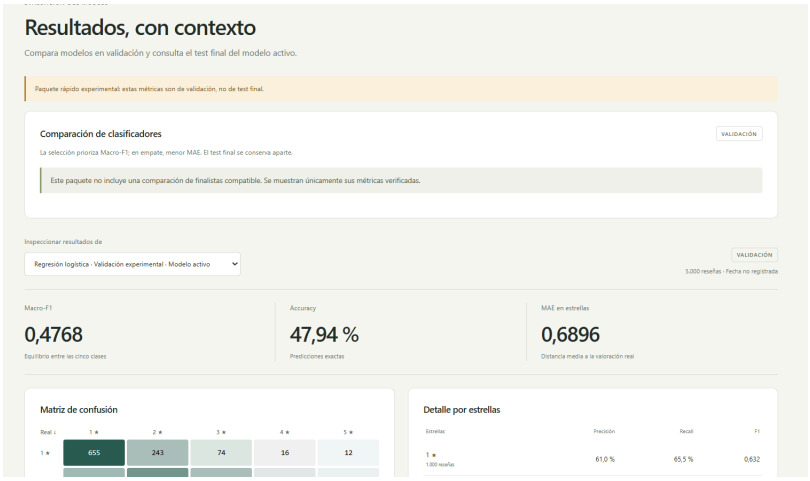
Diseño de Interfaz Iniciar sesión



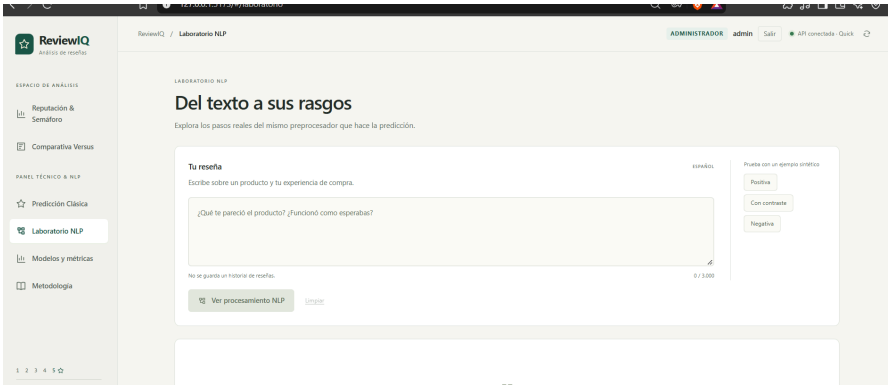
Pantalla principal



Diseño de Interfaz de metodos y metricas del modelo.



Diseño de Interfaz de pruebas individuales



5. Procesamiento De Lenguaje Natural Y Modelos De Machine Learning

El procesamiento analítico de reseñas de aplicaciones móviles presenta retos derivados del lenguaje informal del usuario: uso de emoticonos y emojis, modismos coloquiales, errores ortográficos, puntuación reiterativa y expresiones con polaridades mixtas (e.g., elogio al soporte vs. crítica al producto). Para transformar este texto desestructurado en representaciones matemáticas aptas para modelos supervisados, ReviewIQ implementa un pipeline secuencial, determinista y reproducible compuesto por cinco fases: normalización canónica, traducción léxica de emojis, tokenización con filtrado de stopwords y protección de negaciones, reducción morfológica por derivación léxica (stemming) y vectorización numérica ponderada mediante TF-IDF con n-gramas.

5.1 Pipeline de Procesamiento de Lenguaje Natural

5.1.1 Configuración de Parámetros de Ingeniería del Pipeline

Componente	Parámetro / Técnica	Configuración en ReviewIQ	Justificación Técnica
Normalización	Estándar Unicode	Forma de Compatibilidad NFKC	Estandariza caracteres compuestos, compatibilidad tipográfica y minúsculas continuas.
Limpieza Léxica	Expresiones Regulares	Reemplazo de URLs y Dígitos	Sustituye enlaces por urltoken y cifras por numerotoken para evitar dispersión léxica irrelevante.
Emojis	Diccionario Semántico	Mapeo emoji -> texto	Traduce símbolos gestuales como 😡 a emojitoken_enraged_face, preservando la carga afectiva del usuario.
Stopwords	Lista NLTK en Español	Negaciones Protegidas	Se protegen estrictamente: "no", "sin", "nunca", "jamás", "tampoco". Evita invertir la polaridad crítica.
Morfología	Snowball Stemmer	Derivación léxica en español	Colapsa variantes flexivas a la raíz común con alta velocidad y bajo consumo de memoria.
Espacio N-gramas	Rango de tokens	Unigramas y Bigramas (1, 2)	Captura combinaciones contextuales críticas como no_funciona o muy_bueno.
Ponderación TF-IDF	Filtro Frecuencia	min_df=5, max_df=0.8	Elimina vocabulario ruidoso (<5 apariciones) y términos ubicuos sin valor discriminante (>80%).

5.1.2 Trazabilidad Paso a Paso del Pipeline (Ejemplo Antes / Después)

Fase	Entrada / Salida en ReviewIQ	Descripción del Cambio y Justificación
1. Texto Original	"No me gustó!!! Llegó roto 😡. Sin garantía tras 2 días: https://ejemplo.invalid"	Texto en bruto extraído por scraping de la tienda de aplicaciones. Contiene puntuación, emojis, URLs y negaciones.

2. Normalización y Emojis	no me gustó llegó roto emojitoken_enraged_face sin garantía tras numerotoken días urltoken	Conversión a minúsculas, reemplazo de URLs por urltoken, números por numerotoken y traducción del emoji 😡.
3. Tokenización y Stopwords	Tokens retenidos: ['no', 'gustó', 'llegó', 'roto', 'emojitoken_enraged_face', 'sin', 'garantía', 'tras', 'numerotoken', 'días', 'urltoken'] Eliminados: ['me']	Segmentación léxica. La lista de stopwords retiene las partículas de negación para proteger el sentido crítico de la oración.
4. Reducción Morfológica	['no', 'gust', 'lleg', 'rot', 'emojitoken_enraged_face', 'sin', 'garanti', 'tras', 'numerotoken', 'dias', 'urltoken']	Aplicación de Snowball Stemmer. Raíces léxicas comunes: gustó -> gust, garantía -> garanti, reduciendo la dimensionalidad.
5. N-gramas (1, 2)	Unigramas: no, gust, lleg, rot... Bigramas: no_gust, gust_lleg, lleg_rot, rot_emojitoken_enraged_face, sin_garanti...	Los bigramas preservan la sintaxis local y asocian la negación al verbo o adjetivo adyacente (no_gust vs gust).
6. Vectorización TF-IDF	Vector esparso x in R^36851 (float32): - TF-IDF(no_gust) = 0.412 - TF-IDF(rot) = 0.385 - TF-IDF(emojitoken_enraged_face) = 0.354	Asignación de pesos matemáticos según frecuencia local e inversa en el corpus completo de 200,000 reseñas.

5.1.3 Comparativa Empírica: Stemming (Snowball) frente a Lematización

Enfoque Morfológico	N-gramas	Macro-F1 (Val)	Exactitud	Vocabulario	Tiempo Preproc.	Tiempo Vectoriz.	RAM Pico
Stemming (Snowball)	(1, 1)	0.4467	44.90%	5,710 términos	13.38 s	0.25 s	272.9 MiB
Stemming (Snowball)	(1, 2)	0.4783	48.12%	36,851 términos	13.38 s	1.01 s	272.9 MiB
Lematización (spaCy)	(1, 1)	0.4373	43.90%	8,291 términos	81.63 s	0.25 s	409.6 MiB
Lematización (spaCy)	(1, 2)	0.4689	47.18%	36,737 términos	81.63 s	1.04 s	409.6 MiB

Conclusiones:

- Eficiencia Temporal (Latencia): Snowball Stemmer es 6.1 veces más rápido que spaCy (13.38 s vs 81.63 s), requisito fundamental para procesar lotes de reseñas en tiempo real desde Google Play.
- Consumo de Memoria: Snowball opera con 272.9 MiB de memoria RAM pico frente a 409.6 MiB de spaCy (+50% de sobrecarga debida a la red neuronal del pipeline sintáctico).
- Rendimiento Predictivo: Stemming supera a la lematización por +0.94% de Macro-F1 (0.4783 vs 0.4689), debido a que el texto coloquial presenta faltas de concordancia que confunden al lematizador sintáctico.
- Aporte de Bigramas: La inclusión de n-gramas (1, 2) aportó una ganancia decisiva de +3.16% en Macro-F1 en ambos enfoques.

5.2 Modelos de Machine Learning, Comparativa y Análisis de Errores

Para resolver la clasificación multiclase ordinal (predicción de 1 a 5 estrellas), se entrenaron y optimizaron dos familias de modelos supervisados sobre el corpus Amazon Reviews Multi Spanish (N = 200,000 reseñas): Multinomial Naive Bayes (MNB) como línea base probabilística y Regresión Logística Multiclase regularizada.

5.2.1 Comparativa de Modelos Supervisados Multiclase

Algoritmo	Hiperparámetro Óptimo	Macro-F1 (Val)	Exactitud (Val)	MAE (Val)	Macro-F1 (Test)	Exactitud (Test)	MAE (Test)
Multinomial Naive Bayes	alpha = 2.0 (Lidstone)	0.5088	51.38%	0.6350	-	-	-
Regresión Logística (Campeón)	C = 1.0 (Regularización L2)	0.5150	51.82%	0.6040	0.5131	51.54%	0.6128

5.2.2 Desempeño Detallado por Clase

Clase / Calificación	Precisión (Precision)	Cobertura (Recall)	F1-Score	Soporte Evaluado	Comportamiento en ReviewIQ
1 Estrella (1)	64.97%	67.70%	0.6631	1,000 reseñas	Alta discriminación: Detección precisa de fallos severos y rechazo crítico.

2 Estrellas (2)	42.88%	44.60%	0.4373	1,000 reseñas	Zona de transición: Frecuente adyacencia de error con 1 y 3.
3 Estrellas (3)	40.42%	38.80%	0.3959	1,000 reseñas	Mayor ambigüedad: Clase neutra con lenguaje balanceado pro/contra.
4 Estrellas (4)	45.35%	41.00%	0.4307	1,000 reseñas	Zona de transición: Frecuente adyacencia de error con 3 y 5.
5 Estrellas (5)	62.24%	65.60%	0.6388	1,000 reseñas	Alta discriminación: Detección de satisfacción plena y elogios directos.
Promedio Macro	51.17%	51.54%	0.5131	5,000 reseñas	Evaluación final sobre conjunto ciego e independiente.

5.2.3 Matriz de Confusión Multiclase

Calificación Real	Predicho: 1	Predicho: 2	Predicho: 3	Predicho: 4	Predicho: 5	Total Real	Recall (%)
1 Estrella (1)	677	246	61	6	10	1,000	67.70%
2 Estrellas (2)	247	446	231	46	30	1,000	44.60%
3 Estrellas (3)	85	248	388	217	62	1,000	38.80%
4 Estrellas (4)	19	64	211	410	296	1,000	41.00%
5 Estrellas (5)	14	36	69	225	656	1,000	65.60%

5.2.4 Distribución de Errores por Distancia Ordinal

Distancia de Error	Cantidad de Casos	Porcentaje del Total	Interpretación Técnica en ReviewIQ

Distancia 0 (Exactos)	2,577	51.54%	Predicción exacta de la estrella otorgada por el usuario.
Distancia 1 (Adyacente)	1,921	38.42%	Desviación subjetiva mínima (e.g. predecir 4 en lugar de 5).
Distancia 2 (Moderada)	428	8.56%	Confusión entre extremos y neutralidad (e.g. 3 predicho como 1).
Distancia 3 (Severa)	50	1.00%	Desviación de rango amplio.
Distancia 4 (Inversión polar)	24	0.48%	Errores catastróficos aislados (1 predicho como 5 o viceversa).
Total Evaluado	5,000	100.00%	Error Absoluto Medio (MAE): 0.6128 estrellas.

Diagnóstico de Robustez: El 89.96% de todas las predicciones del modelo se ubican en la calificación exacta o a una sola estrella de distancia, lo que fundamenta un Error Absoluto Medio (MAE) de solo 0.61 estrellas y asegura una estimación reputacional sólida.

5.2.5 Análisis Cualitativo de Errores Reales Inspeccionados

Caso / Fila	Reseña Real del Usuario	Real	Pred.	Desv.	Patrón Lingüístico	Causa Técnica del Error en el Modelo
Caso 1 (Fila 198)	"Excelente si lo que buscas es proteger tu móvil con la consistencia de una servilleta fina"	1	5	+4	Sarcasmo e Ironía Retórica	El modelo lineal suma pesos positivos de "excelente", "proteger" y "fina". Sin atención contextual profunda (tipo Transformer), no detecta la intención irónica.
Caso 2 (Fila 730)	"La atención al cliente fue magnífica y el envío súper rápido, pero la aplicación no abre en absoluto"	1	5	+4	Reseña Bipolar / Múltiples Entidades	Contiene dos entidades evaluadas: Servicio (+) y Software (-). Los adjetivos positivos iniciales saturan el clasificador lineal diluyendo la queja final.

Caso 3 (Fila 33)	"Compré el producto y funcionó bien los primeros días, luego dejó de cargar y soporte no dio respuesta"	1	5	+4	Narrativa Temporal y Falla Administrativa	A pesar de proteger "no", los bigramas positivos iniciales ("funcionó_bien") compensan y mitigan el relato cronológico de la queja técnica posterior.

RECOMENDACIONES

Se recomienda continuar el desarrollo de ReviewIQ incorporando arquitecturas de aprendizaje profundo (*Deep Learning*) o modelos de lenguaje basados en transformadores ligeros (tales como BETO o RoBERTa preentrenados en español). Esto permitirá capturar sutilezas semánticas complejas en el lenguaje coloquial, tales como el sarcasmo, la ironía y dobles sentidos presentes en las reseñas informales de las tiendas móviles.

Es conveniente implementar colas de procesamiento asíncrono y tareas en segundo plano mediante herramientas como Celery y Redis. Esto mejorará significativamente la escalabilidad del sistema durante la ingesta masiva de comentarios desde Google Play Store y el procesamiento de grandes volúmenes de inferencia por lotes (*Batch Processing*), evitando la saturación de los *workers* de Django REST Framework en escenarios de alta concurrencia.

Se sugiere extender la capa de recolección de datos hacia nuevas fuentes de opiniones públicas, incorporando conectores para Apple App Store, Google Maps y plataformas de comercio electrónico. De esta manera, las organizaciones podrán obtener una visión integral y omnicanal de su reputación de marca.

Para entornos de producción, es recomendable fortalecer las políticas de seguridad mediante el uso de tokens JWT con rotación periódica de claves de firma, autenticación multifactor (MFA) para analistas administradores y la implementación de limitación de tasa de peticiones (*rate limiting*) sobre los endpoints de predicción libre para mitigar potenciales ataques de denegación de servicio (DoS).

Se aconseja enriquecer el módulo de analítica y diagnóstico con herramientas de visualización de series temporales que permitan observar la evolución histórica de cada una de las cinco dimensiones de calidad a lo largo de semanas o meses, complementando el sistema con la exportación automática de reportes ejecutivos de auditoría en formato PDF.

Finalmente, se recomienda establecer un protocolo de monitoreo continuo de deriva de datos (*Data Drift*) y degradación del modelo en producción. Esto permitirá alertar a los ingenieros de Machine Learning cuando el vocabulario o los modismos locales comiencen a desviarse de la distribución del corpus original de Amazon Reviews, disparando un ciclo automatizado de reentrenamiento y calibración de hiperparámetros.

Bibliografía

- GBooch, G., Rumbaugh, J., & Jacobson, I. (2006). *El Lenguaje Unificado de Modelado* (2.ª ed.). Pearson Educación.
- Django Software Foundation. (2024). *Django: The web framework for perfectionists with deadlines* (Versión 5.0). https://docs.djangoproject.com
- ECMA International. (2023). *ECMAScript® 2023 Language Specification* (14.ª ed.). https://www.ecma-international.org
- Jacobson, I., Booch, G., & Rumbaugh, J. (1999). *El Proceso Unificado de Desarrollo de Software*. Addison Wesley Longman.
- Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition* (3.ª ed. borrador). Stanford University. https://web.stanford.edu/~jurafsky/slp3/
- Keung, P., Lu, Y., Szarvas, G., & Smith, N. A. (2020). The Multilingual Amazon Reviews Corpus. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 4563–4568. https://doi.org/10.18653/v1/2020.emnlp-main.369
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. https://doi.org/10.1017/CBO9780511809071
- Object Management Group. (2017). *Unified Modeling Language (UML) Specification Version 2.5.1*. https://www.omg.org/spec/UML

- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- PostgreSQL Global Development Group. (2024). *PostgreSQL 15 Documentation*. https://www.postgresql.org/docs/15/
- React Documentation Group. (2024). *React: The library for web and native user interfaces*. Meta Open Source. https://react.dev
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.

DECLARACIÓN DE USO DE ASISTENTES DE INTELIGENCIA ARTIFICIAL

Dimensión de Aplicación	Herramienta Utilizada	Alcance y Propósito Académico	Nivel de Responsabilidad del Equipo
Soporte de Programación	Google Gemini / Antigravity	Asistencia en scripts de benchmarking para librerías NLP (Snowball vs spaCy) y soporte sintáctico en Django REST Framework.	Autoría y Validación: El equipo diseñó la arquitectura, depuró y probó todo el código en el repositorio.
Análisis de Métricas	Google Gemini / Antigravity	Apoyo en la formulación de tablas comparativas y diagnóstico cualitativo de la matriz de confusión multiclase.	Decisión Técnica: La elección de la Regresión Logística y el preprocesamiento fue tomada exclusivamente por el equipo con base en resultados experimentales.

1. Anexos

QR de la nube de la web

https://cnflq5t9-5173.brs.devtunnels.ms/

QR del github de la web



https://github.com/Gerstep360/ReviewIQ